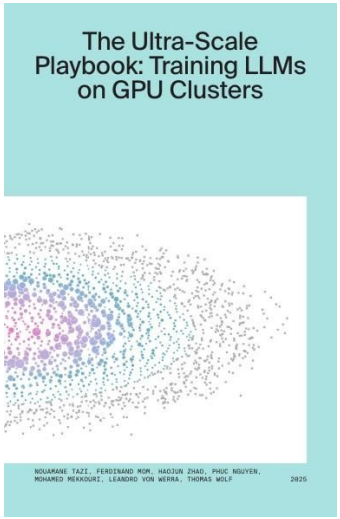
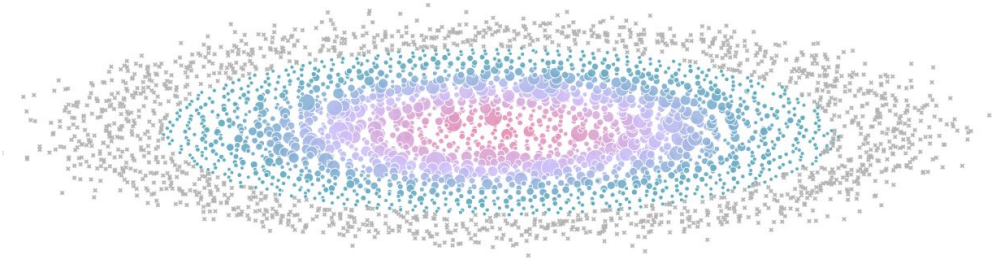
Scaling Training to *Thousands* of GPUs

Nouamane Tazi
Research Engineer @ Hugging Face

Stanford University



The Ultra-Scale Playbook: Training LLMs on GPU Clusters



We ran over 4,000 scaling experiments on up to 512 GPUs and measured throughput (size of markers) and GPU utilization (color of markers). Note that both are normalized per model size in this visualization.

ORDER BOOK GET PDF

AUTHORS
Nouamane Tazi, Ferdinand Mom, Haojun Zhao, Phuc Nguyen, Mohamed Mekkouri, Leandro Werra, Thomas Wolf

AFFILIATION
Hugging Face

PUBLISHED
Feb 19, 2025

Table of Contents ▼

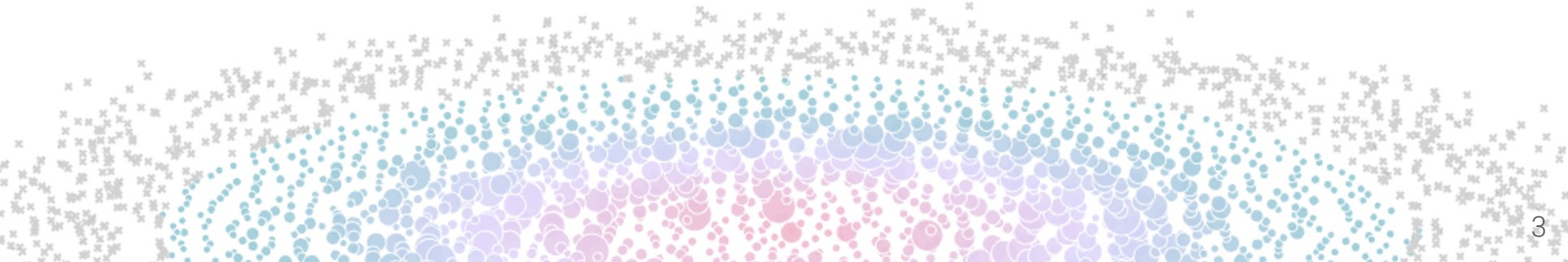
Thousands of GPUs humming in perfect harmony. That's what it takes to train today's most powerful AI models – a symphony of computing power that until recently was the exclusive domain of elite research labs. Open source has transformed this landscape, but not completely. Yes, you can download the latest Llama or DeepSeek models. Yes, you can read their technical and experiment reports. But the most challenging part – the training code, the knowledge and techniques necessary to coordinate GPUs to train these massive systems – remains shrouded in complexity and spread around in a series of disconnected papers and often private codebases.

This open source book is here to change that. Starting from the basics, we'll walk you through the knowledge necessary to scale the training of large language models (LLMs) from one GPU to tens, hundreds, and even thousands of GPUs, illustrating theory with practical code examples

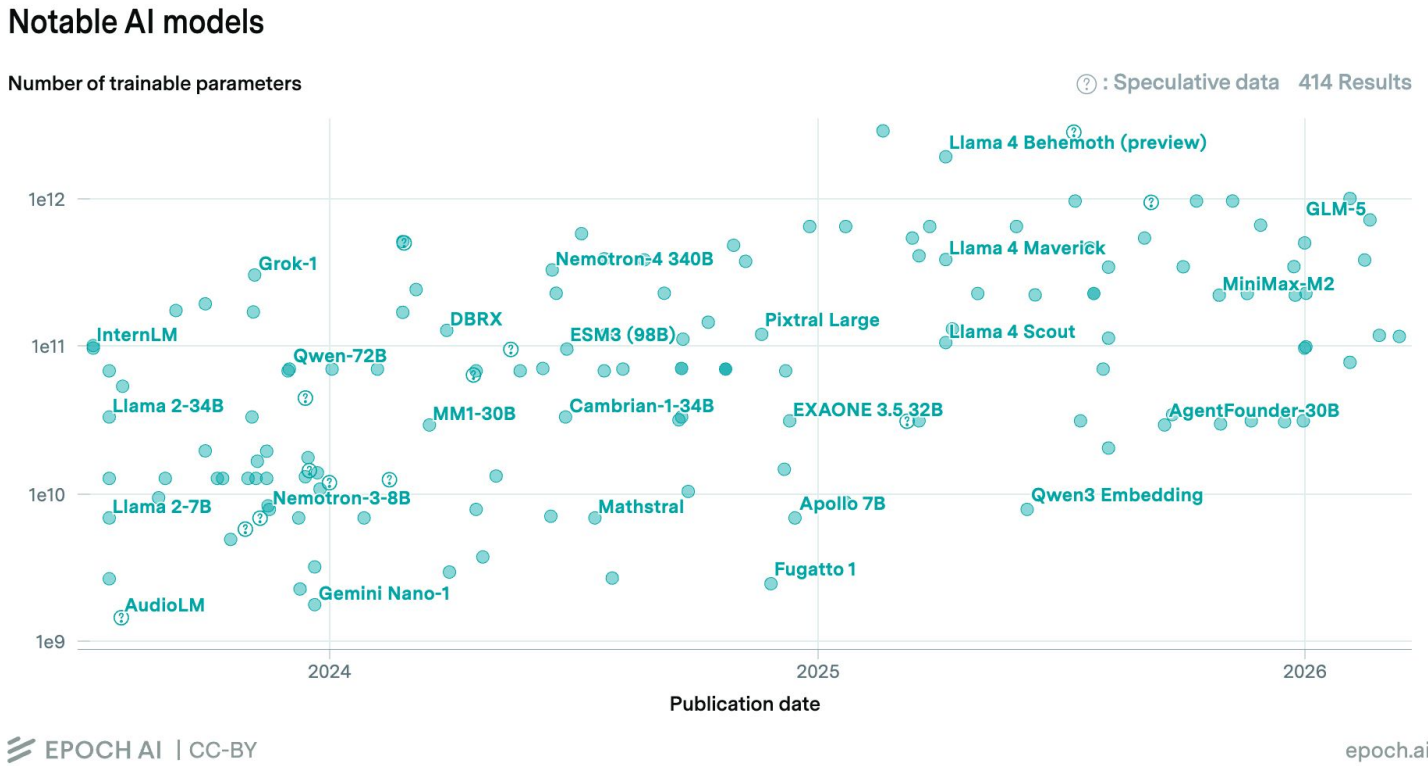
Reading time: 2-4 days. For the best reading experience, we recommend not using a mobile phone.

Why does scaling matter?

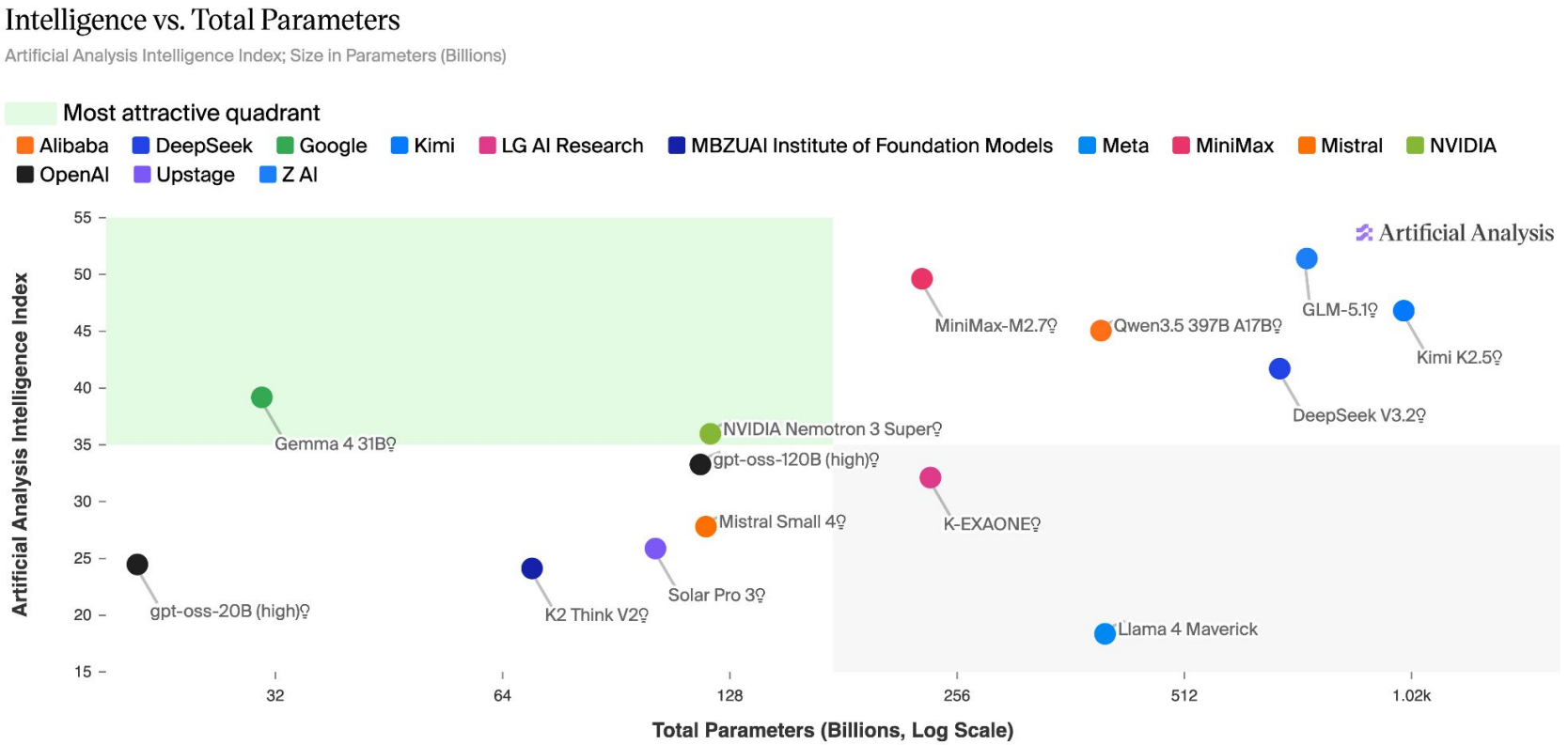
For real though, why?



Trends: train large



Trends: train large



Trends: train large

1 T

Parameters

Frontier MoE models

15 T

Training Tokens

Chinchilla-era and beyond

1 M

Context Length

Long-doc and code agents

How can we train this large?

1T params large!

LLM Training is troublesome for infrastructure

We need to deal with:

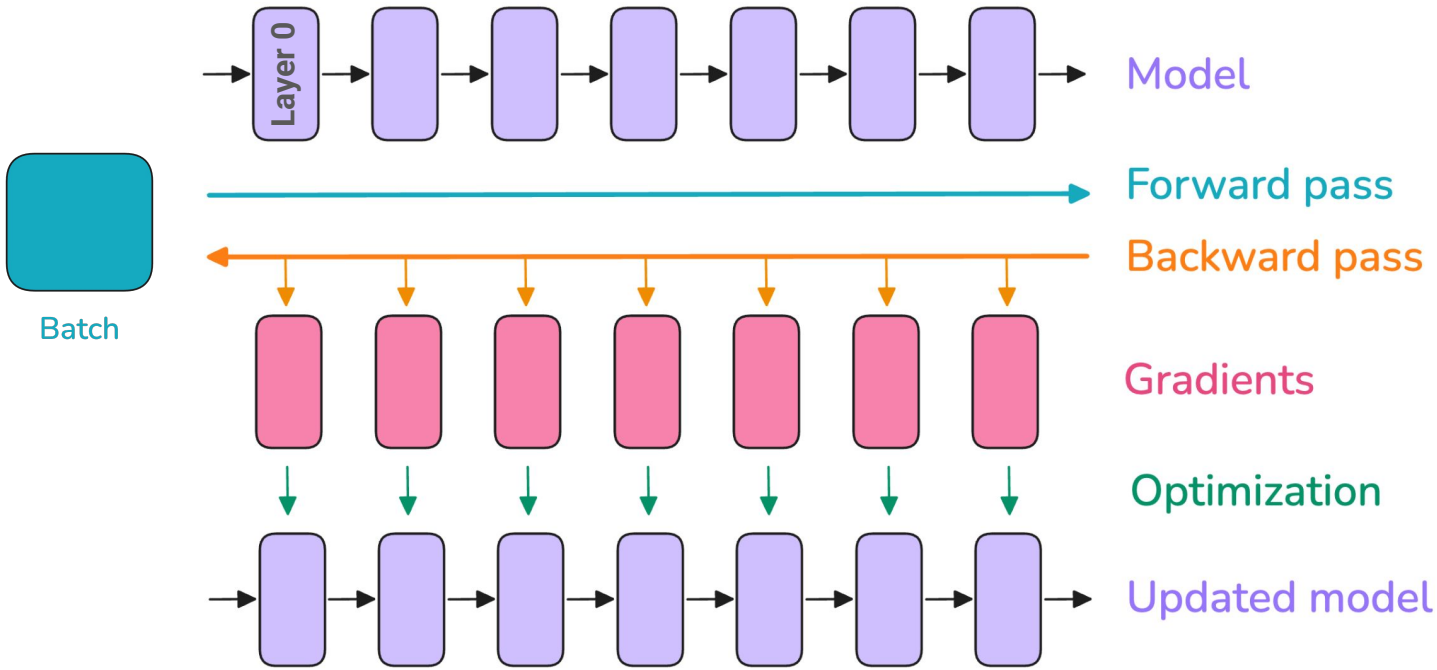
1. Loading data *(**~15T** tokens read from **storage**)*
2. Training iteration *(**~1s/iter** while handling **~80GB** per **GPU memory**)*
3. Saving checkpoints (**~1TBs** written to **storage**)
4. ….

LLM Training is troublesome for infrastructure

We need to deal with:

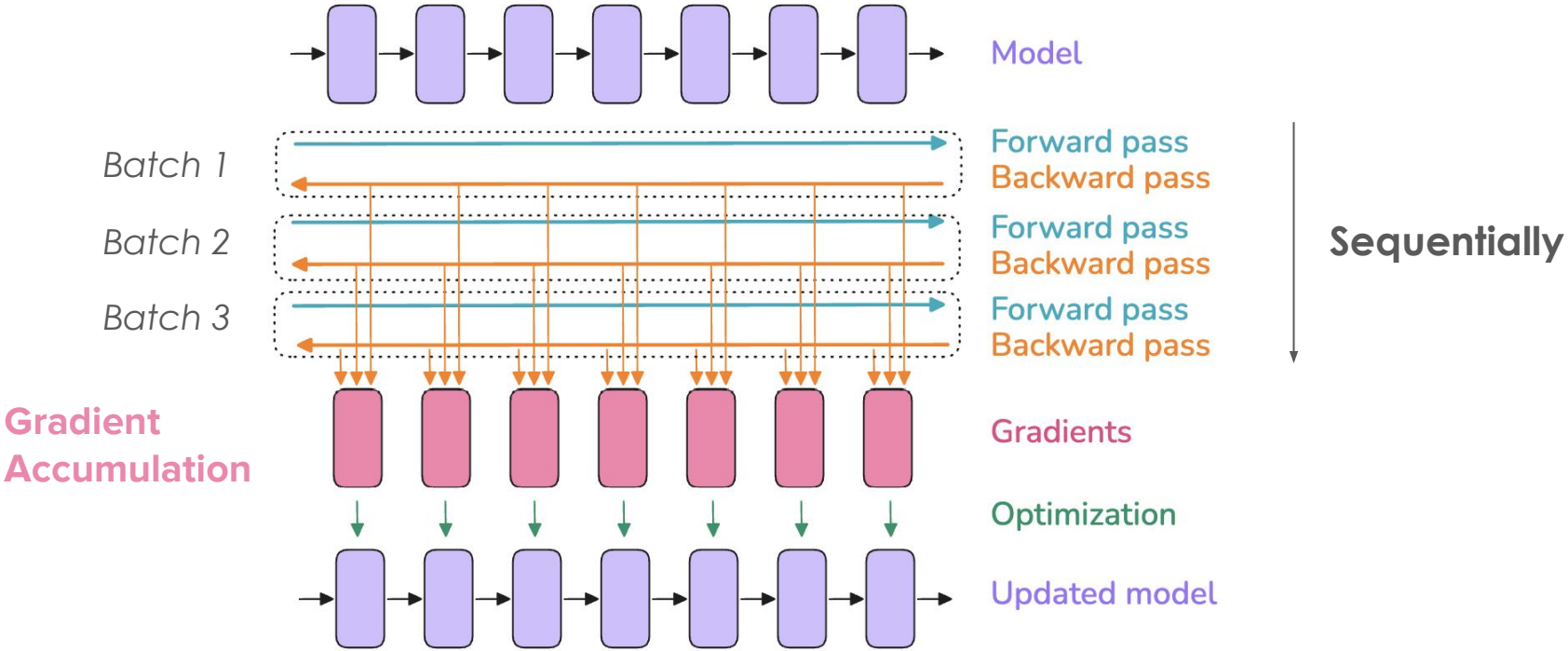
1. Loading data *(**~15T** tokens read from **storage**)*
2. Training iteration *(**~1s/iter** while handling **~80GB** per **GPU memory**)*
3. Saving checkpoints (**~1TBs** written to **storage**)
4. ….

Training: on 1 GPU

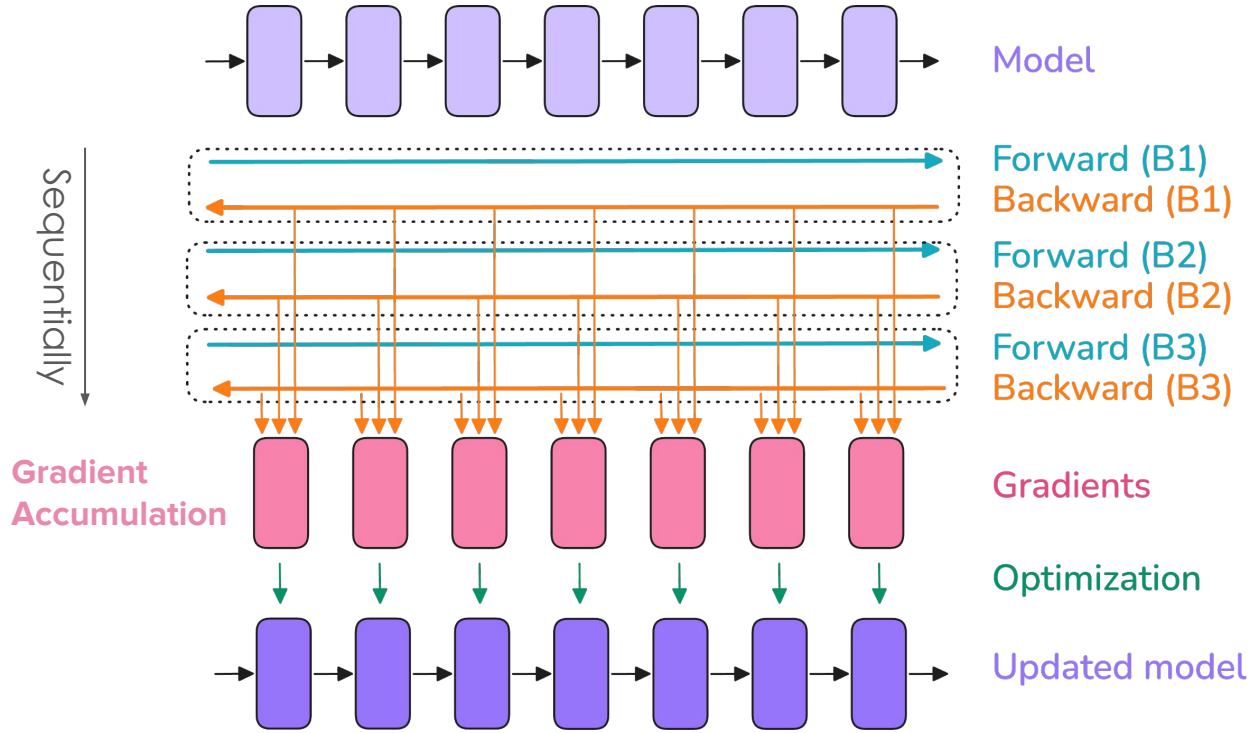


Training: on 1 GPU with a bigger GBS

We train on a **Global Batch Size** of several **millions of tokens**



Training: on 1 GPU with a bigger GBS



This assumes:

- Training **fits in memory**
- Patient enough to **wait** for **all the batches**

How can we solve this with more GPUs?

Plan for today

Demystifying 5D parallelism

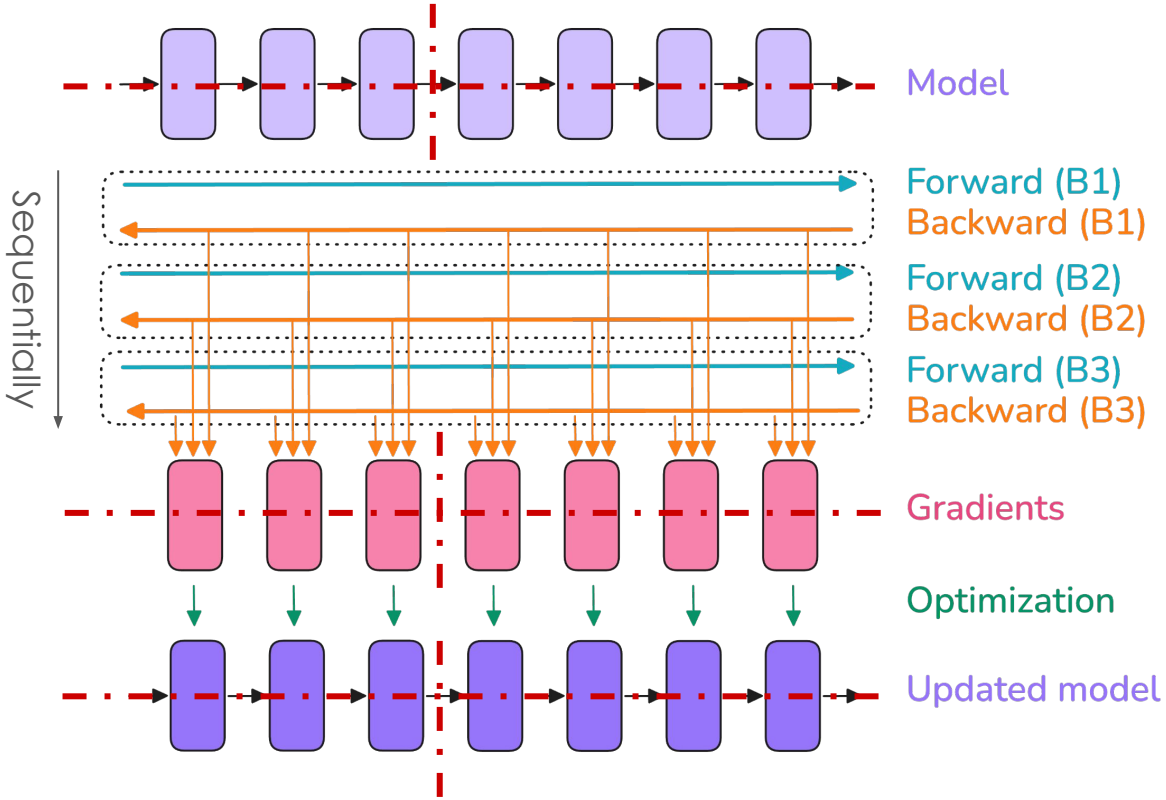
1. Data Parallelism (ZeRO1/2/3)
2. Tensor Parallelism (Sequence Parallelism)
3. Pipeline Parallelism
4. Context Parallelism
5. Expert Parallelism

Preamble

Methods shared here:

- Can be applied to any accelerator (GPUs/TPUs..)
- Useful for both training and inference
- Applicable for > 1 accelerator

What we'd like to do



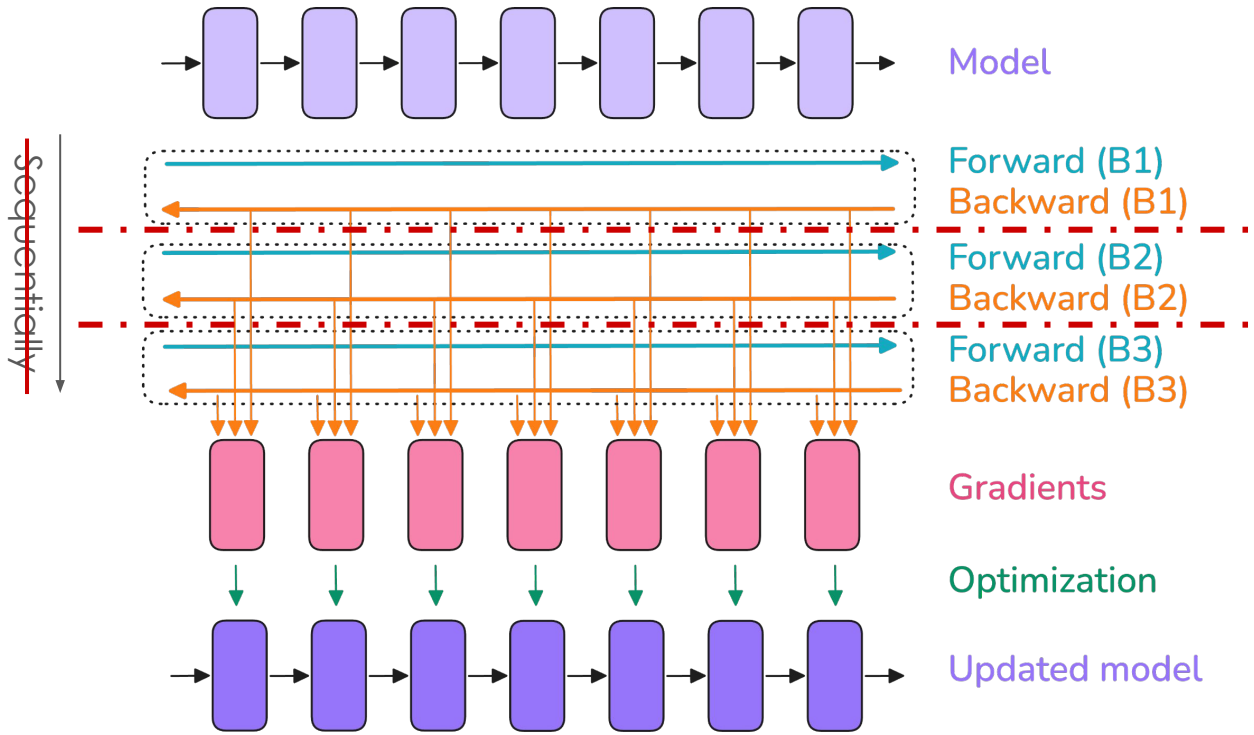
1 **Shard model & gradients**

Every GPU keeps:

- **part** of the **model**
- **part** of the **gradients**
- **part** of the **optimizer states**

Training fits in memory

What we'd like to do

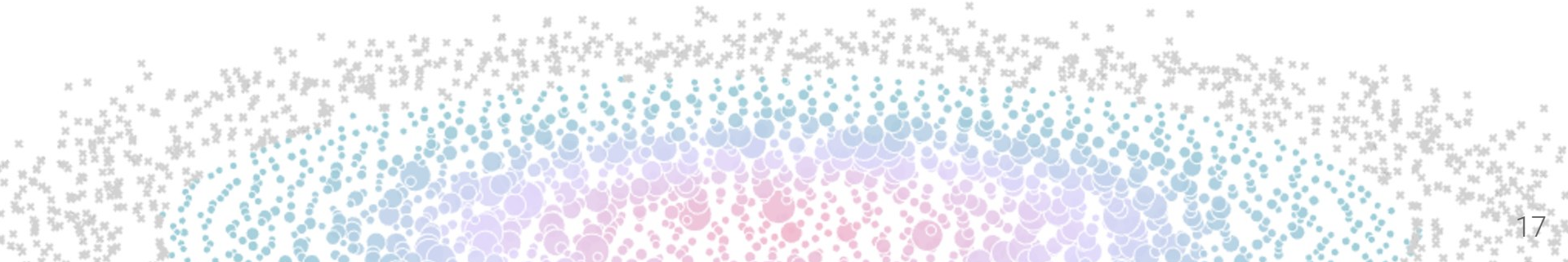


2 **Shard data**

We **duplicate** our **sharded model** and feed **each GPU** a **shard of data**

Let’s deep dive into scaling

Scaling 101 here we go!

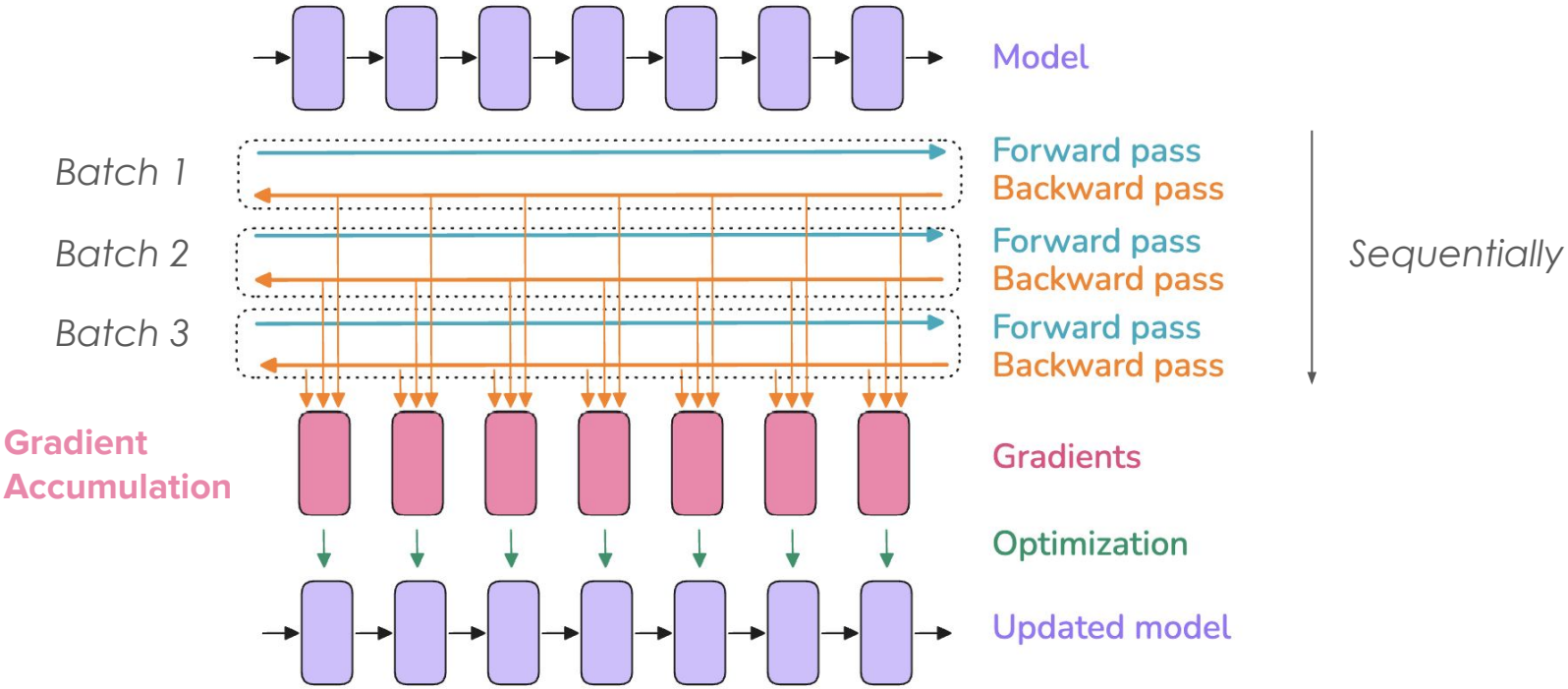


Data Parallelism (DP)

Sharding the batch dimension

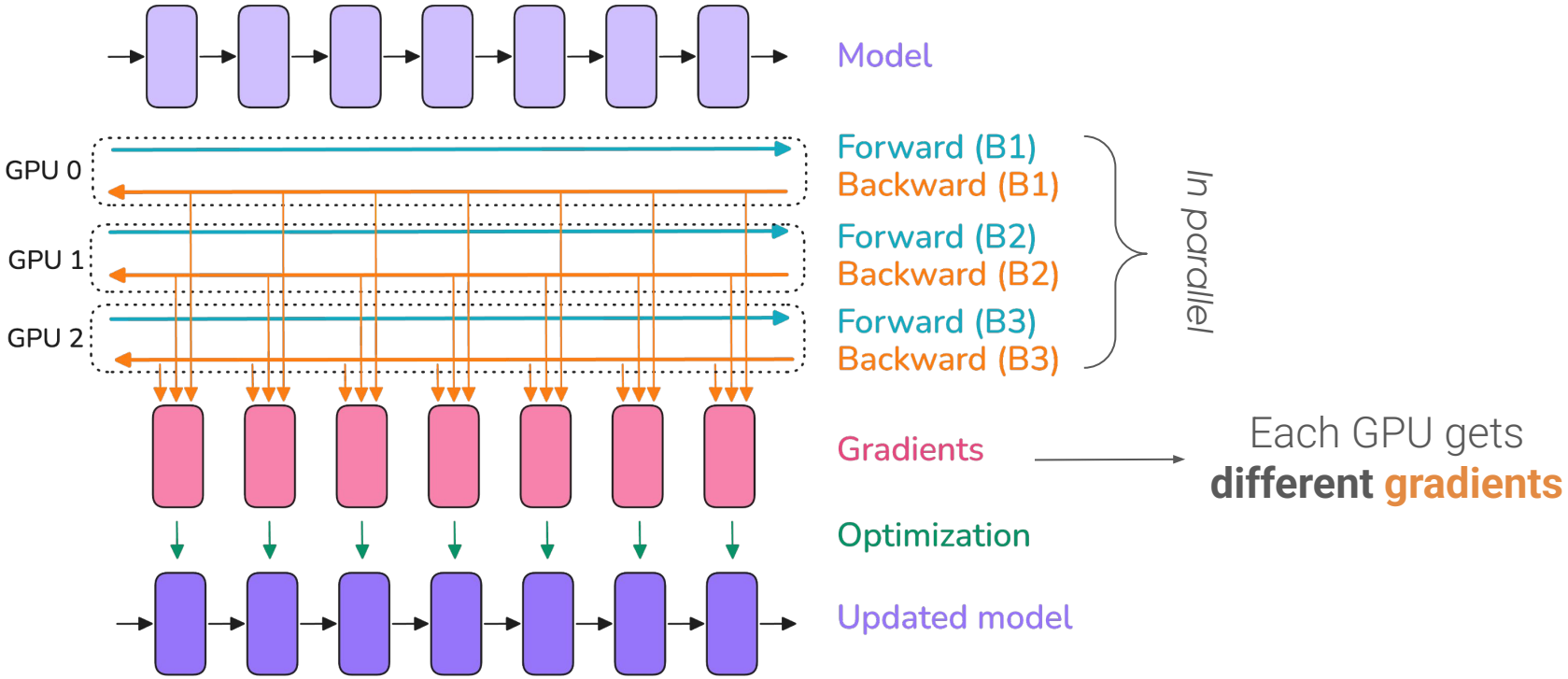
Intuition behind Data Parallelism

Instead of passing batches sequentially and accumulating the gradient on 1GPU,

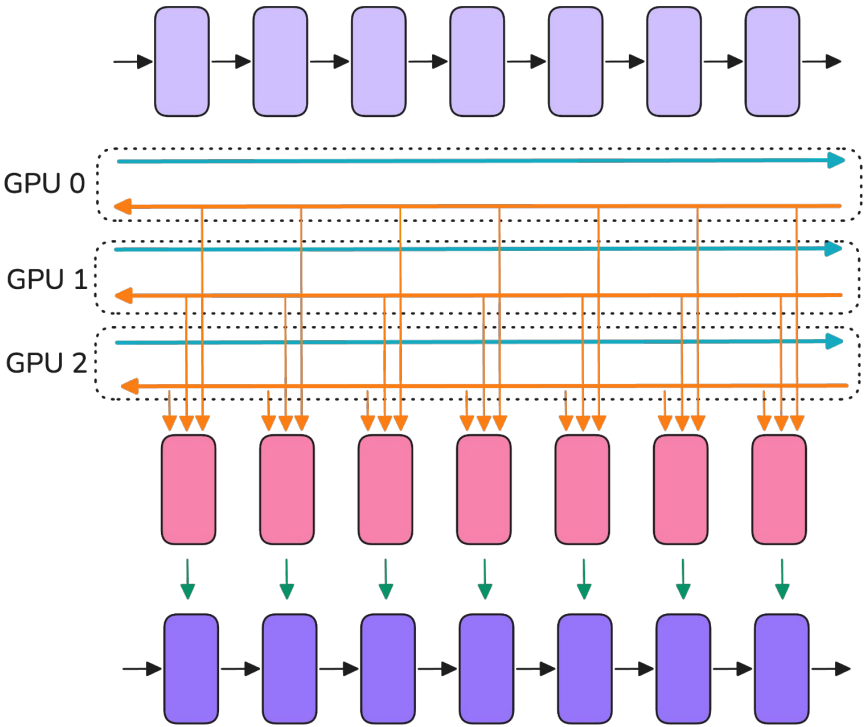


Intuition behind Data Parallelism

We **partition** data and replicate the model across our **GPUs.**



Intuition behind Data Parallelism



Model

Forward (B1)
Backward (B1)

Forward (B2)
Backward (B2)

Forward (B3)
Backward (B3)

In parallel

Gradients

Optimization

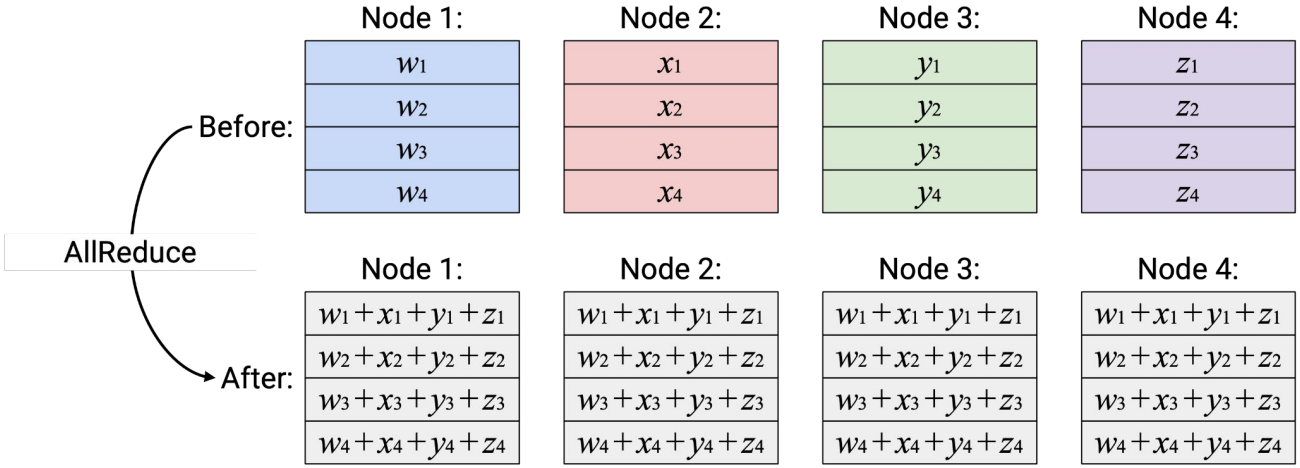
Updated model

Each GPU gets
different gradients

How do we **accumulate gradients** if they exist on **different GPUs**?

Intuition behind Data Parallelism

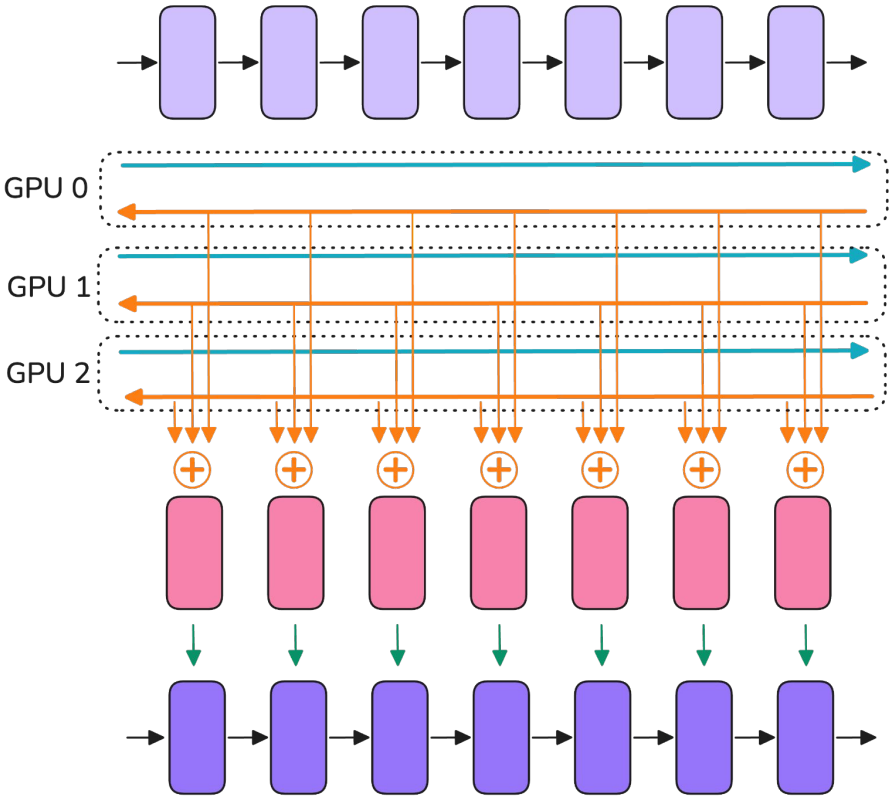
Meet our first **collective operation**: **AllReduce**



** Node = GPU*

Can be seen as a **distributed sum**

Intuition behind Data Parallelism



Model

Forward (B1)
Backward (B1)
Forward (B2)
Backward (B2)
Forward (B3)
Backward (B3)

In parallel

AllReduce

Gradients

Optimization

Updated model

Data is sharded across **GPUs**

↓

Each **GPU** gets **different grads**

↓

AllReduce helps **accumulating grads**

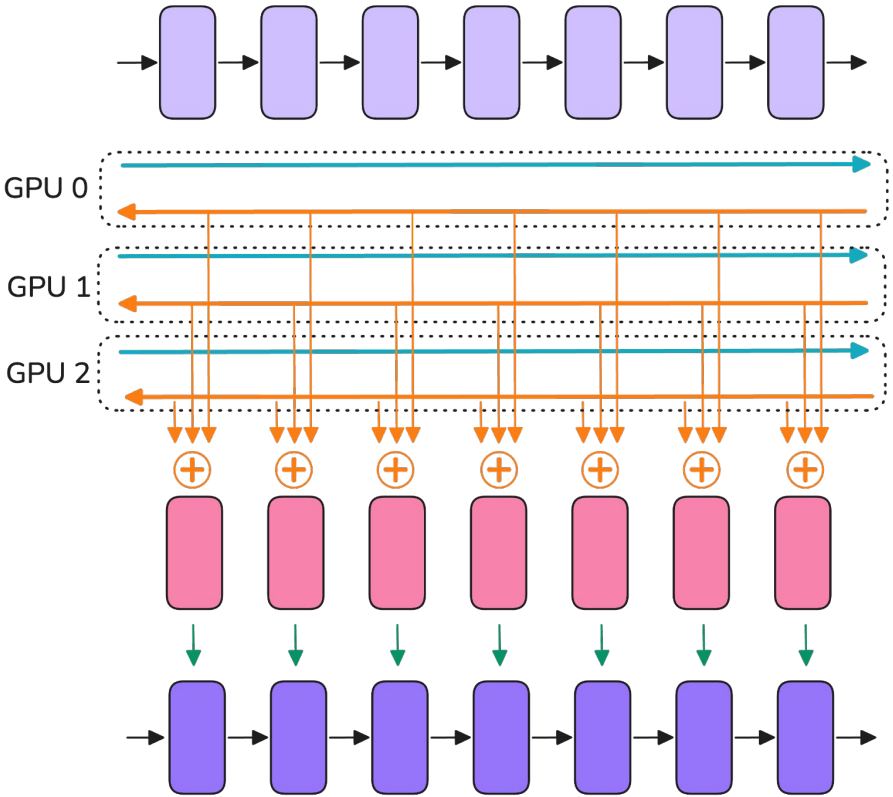
Example code

This is done via
DistributedDataParallel (DDP)

DDP AllReduces grads in background

```python
class ToyModel(nn.Module):
    def __init__(self):
        super(ToyModel, self).__init__()
        self.net1 = nn.Linear(10, 10)
        self.relu = nn.ReLU()
        self.net2 = nn.Linear(10, 5)

    def forward(self, x):
        return self.net2(self.relu(self.net1(x)))


def demo_basic(rank, world_size):
    print(f"Running basic DDP example on rank {rank}.")
    setup(rank, world_size)

    # create model and move it to GPU with id rank
    model = ToyModel().to(rank)
    ddp_model = DDP(model, device_ids=[rank])

    loss_fn = nn.MSELoss()
    optimizer = optim.SGD(ddp_model.parameters(), lr=0.001)

    optimizer.zero_grad()
    outputs = ddp_model(torch.randn(20, 10))
    labels = torch.randn(20, 5).to(rank)
    loss_fn(outputs, labels).backward()
    optimizer.step()

    cleanup()
    print(f"Finished running basic DDP example on rank {rank}.")


def run_demo(demo_fn, world_size):
    mp.spawn(demo_fn,
             args=(world_size,),
             nprocs=world_size,
             join=True)
```

Intuition behind Data Parallelism



Model

Forward (B1)
Backward (B1)
Forward (B2)
Backward (B2)
Forward (B3)
Backward (B3)

In parallel

AllReduce

Gradients

Optimization

Updated model

Data is sharded across **GPUs**

↓

Each **GPU** gets **different grads**

↓

AllReduce helps **accumulating grads**

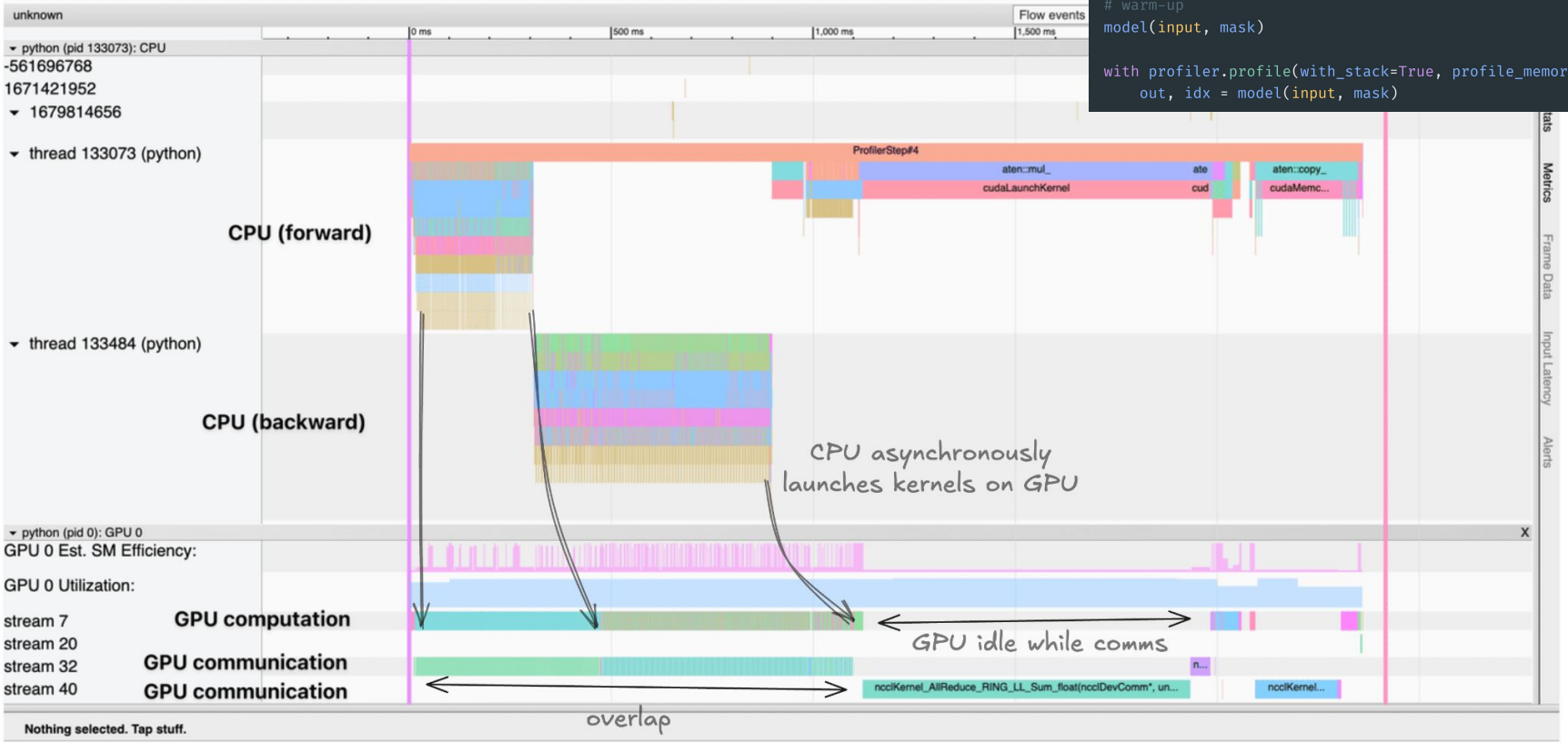
⇩

Do GPUs stay **idle** when **running comms**?

** comms = communications*

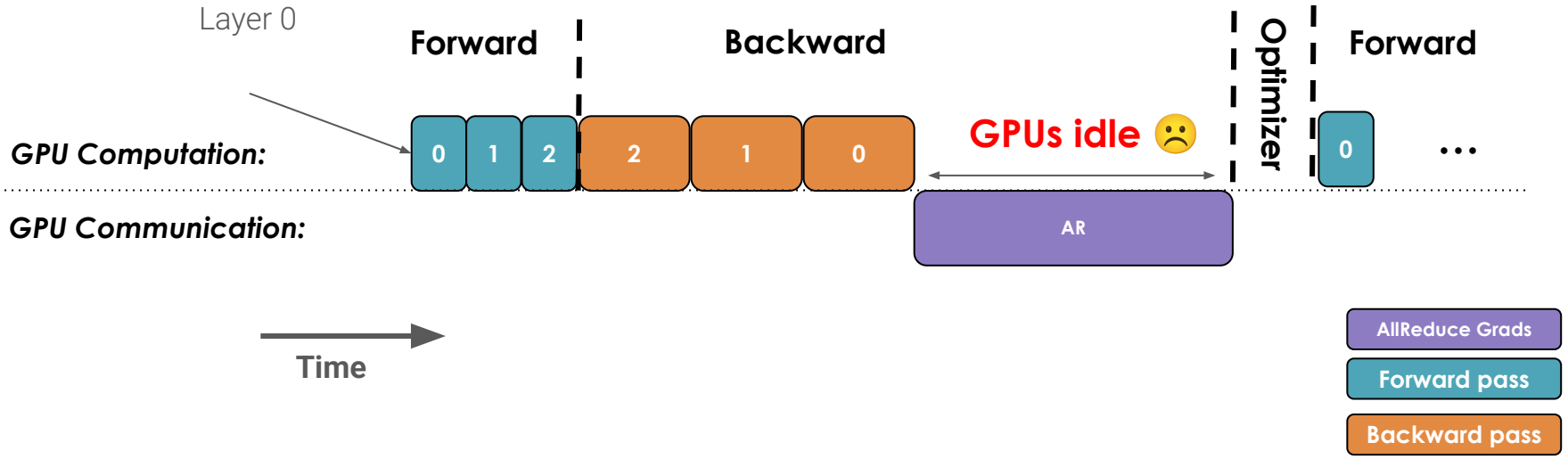
torch.profiler

```python
import torch
import numpy as np
from torch import nn
from torch import profiler

model = MyModule(500, 10).cuda()
input = torch.rand(128, 500).cuda()
mask = torch.rand((500, 500, 500), dtype=torch.double).cuda()

# warm-up
model(input, mask)

with profiler.profile(with_stack=True, profile_memory=True) as prof:
    out, idx = model(input, mask)
```



DP: comms/compute timelines

A naive DP implementation:

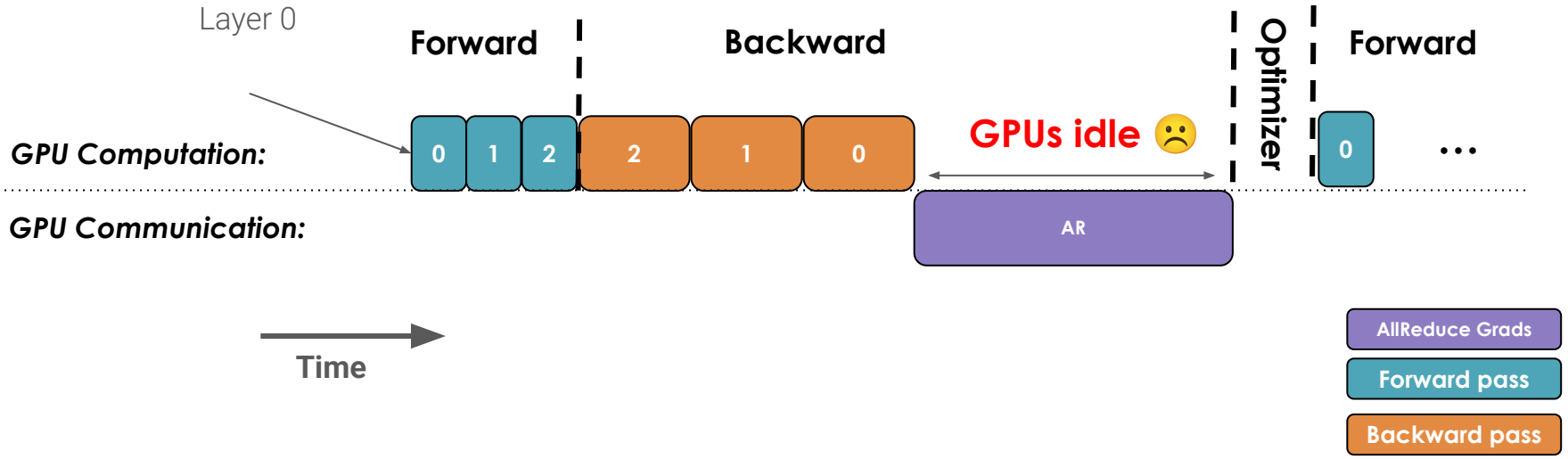


GPUs are **idle** **waiting** for **gradients** to be **AllReduced !**

Can we **overlap comms with compute ?**

DP: comms/compute timelines

A naive DP implementation:

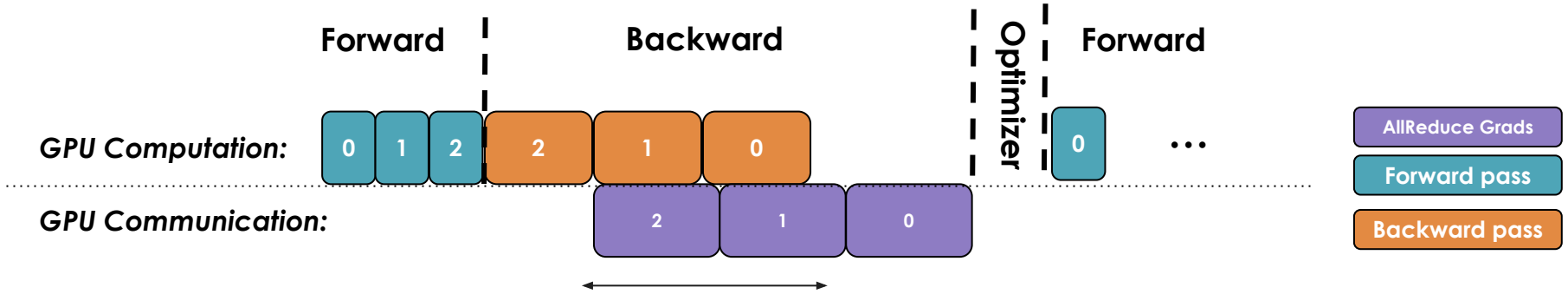


GPUs are **idle** **waiting** for **gradients** to be **AllReduced !**

Can we **overlap comms with compute ?** **YES**

DP: comms/compute timelines

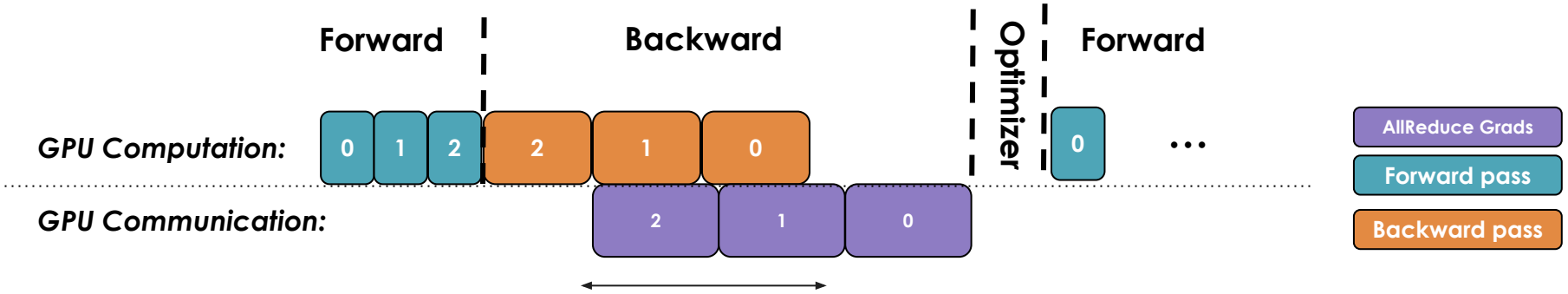
Once a **bucket's gradient** is **computed**, we can already start **communicating** it.
A bucket can be any number of parameters.



Grads AllReduce is overlapped with backward 👌

DP: comms/compute timelines

Once a **bucket's gradient** is **computed**, we can already start **communicating** it



Grads AllReduce is overlapped with backward 👌

Note: We control bucket size via DDP arg:

- **bucket_cap_mb** – `DistributedDataParallel` will bucket parameters into multiple buckets so that gradient reduction of each bucket can potentially overlap with backward computation. `bucket_cap_mb` controls the bucket size in MebiBytes (MiB). If `None`, a default size of 25 MiB will be used. (default: `None`)

DP: Pros/Cons

- \+ Easy to implement
- \+ Little idle time in compute (**Efficient overlap**)
- \+ **Model-agnostic** since we're just duplicating the model

- \- Optimizer step is **duplicated** across GPUs
- \- **GBS** scales with **number of GPUs** ⟶ can't scale indefinitely
- \- Needs **training step** to **fit in memory**

DP: Pros/Cons

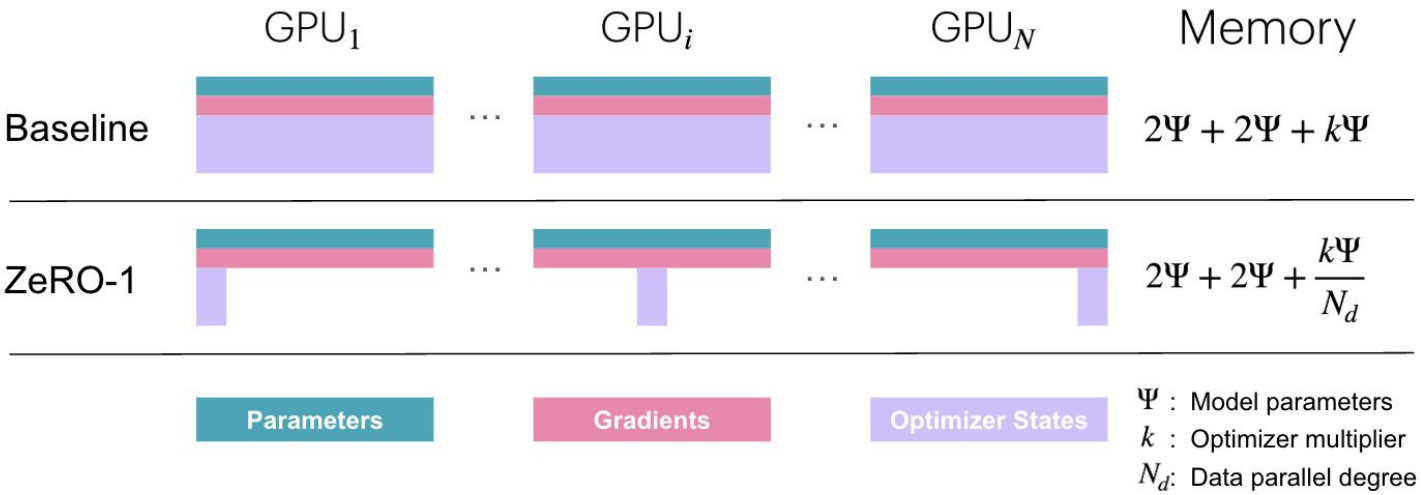
+ Easy to implement
+ Little idle time in compute (**Efficient overlap**)
+ **Model-agnostic** since we're just duplicating the model

- Optimizer step is **duplicated** across GPUs
- **GBS** scales with **number of GPUs** ⟶ can't scale indefinitely
- Needs **training step** to **fit in memory**

To **reduce memory usage** let's look at **DP-ZeRO!**

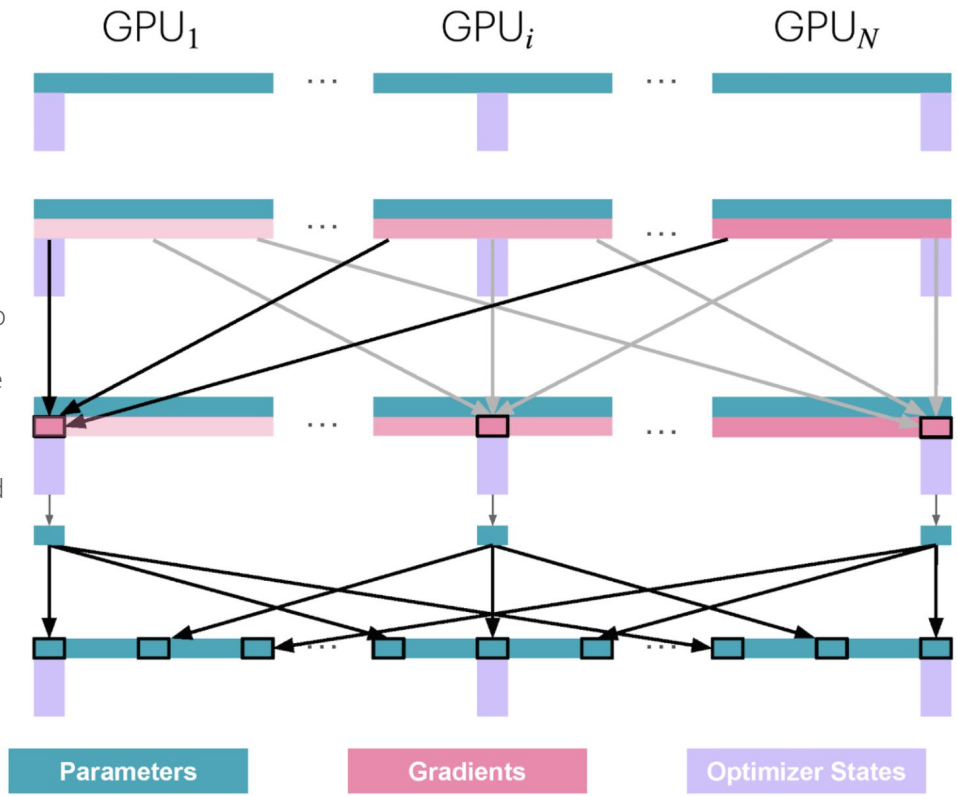
DP: ZeRO1

Problem with DP: All GPUs do the same **optimizer step** -> We need **Zero Redundancy Optimizer**



We want **each GPU** to only keep a **shard of optimizer states** !

DP: ZeRO1



Forward pass

Backward pass

Reduce-scatter

Optimizer step

All-gather

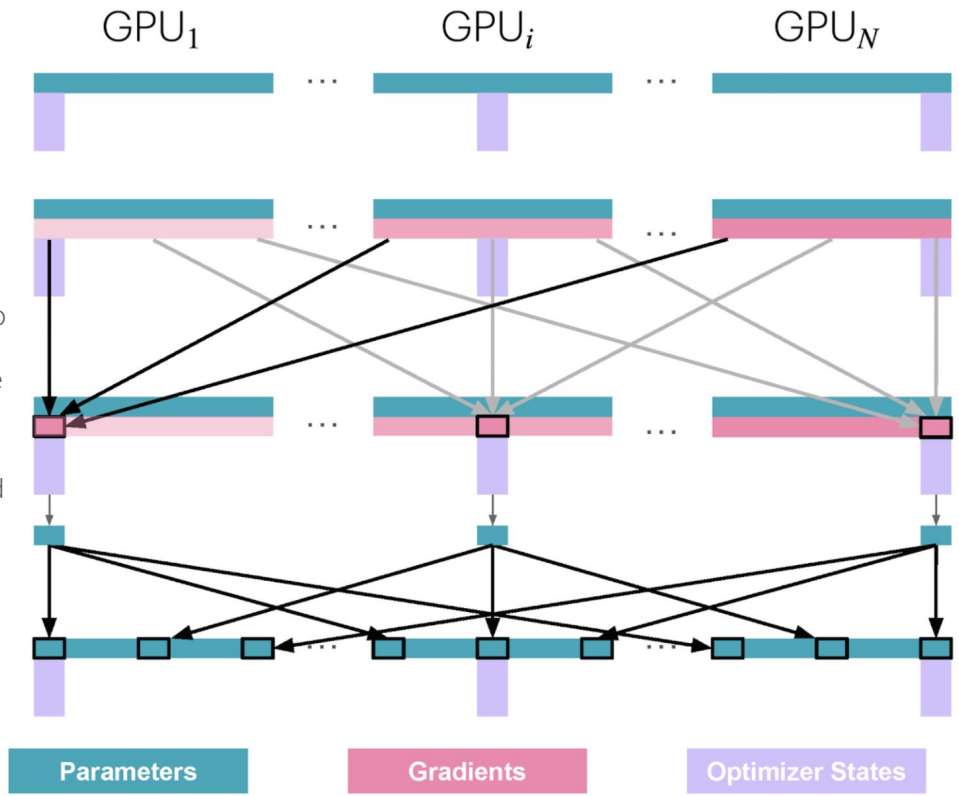
Forward pass (next)

Each rank owns **1/N of the optim states** and **updates only this shard of parameters**

We **ReduceScatter** grads because we only need a shard

We need to **AllGather** the updated params

DP: ZeRO1



Forward pass

Backward pass

Reduce-scatter

Optimizer step

All-gather

Forward pass (next)

Each rank owns **1/N of the optim states** and **updates only this shard of parameters**

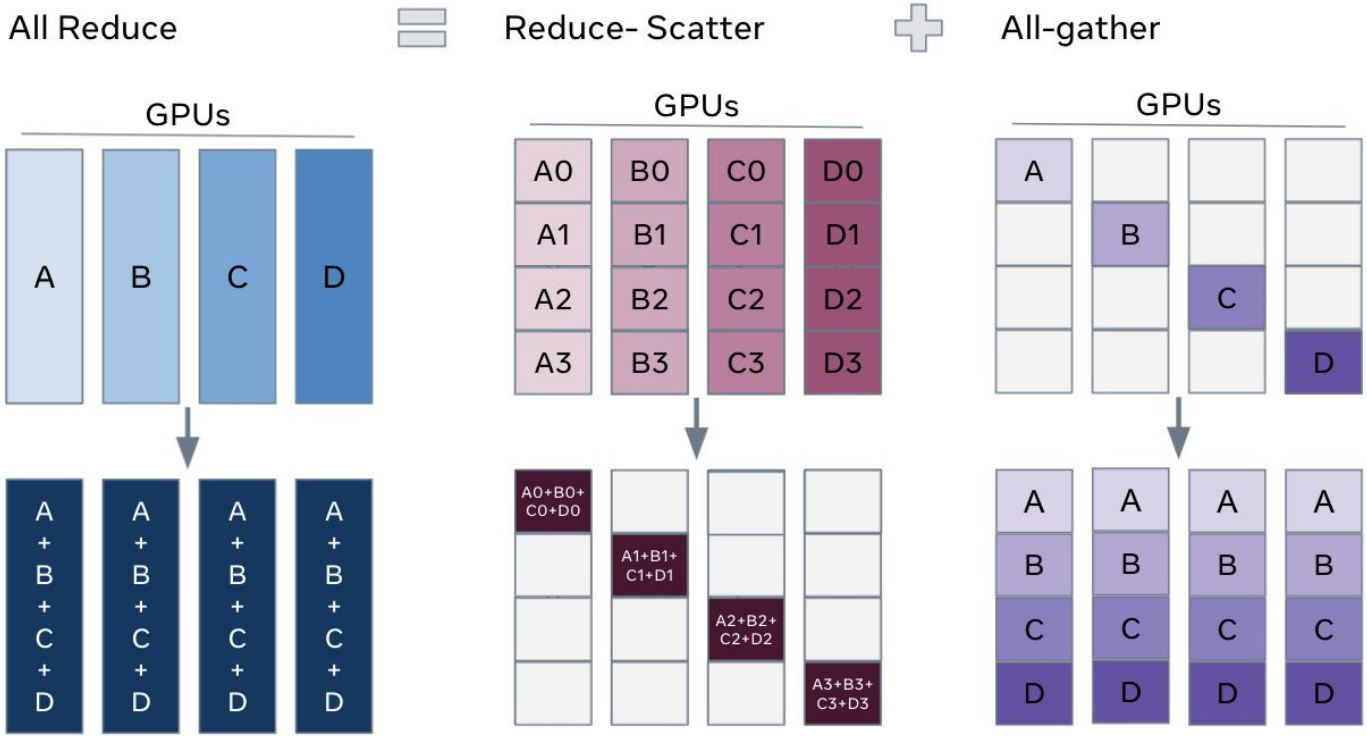
We **ReduceScatter** grads because we only need a shard

We need to **AllGather** the updated params

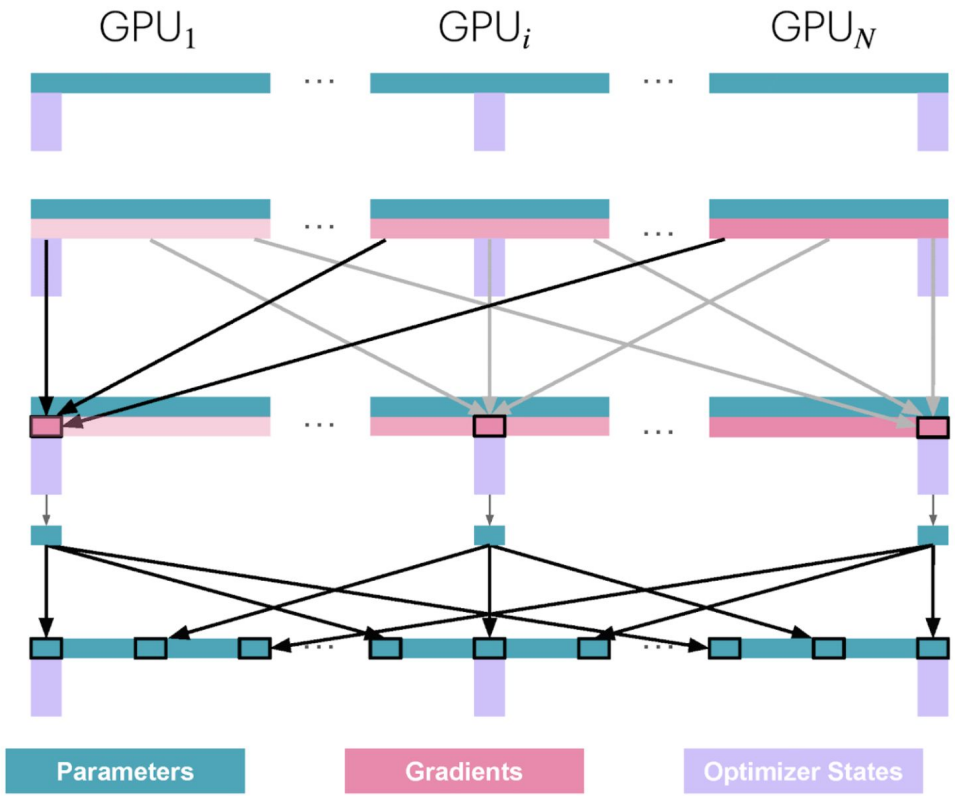
We replaced AllReduce with RS + AG to avoid duplicated work and memory (optim states)

DP: ZeRO1

We didn't add any comms, we just decomposed AllReduce!



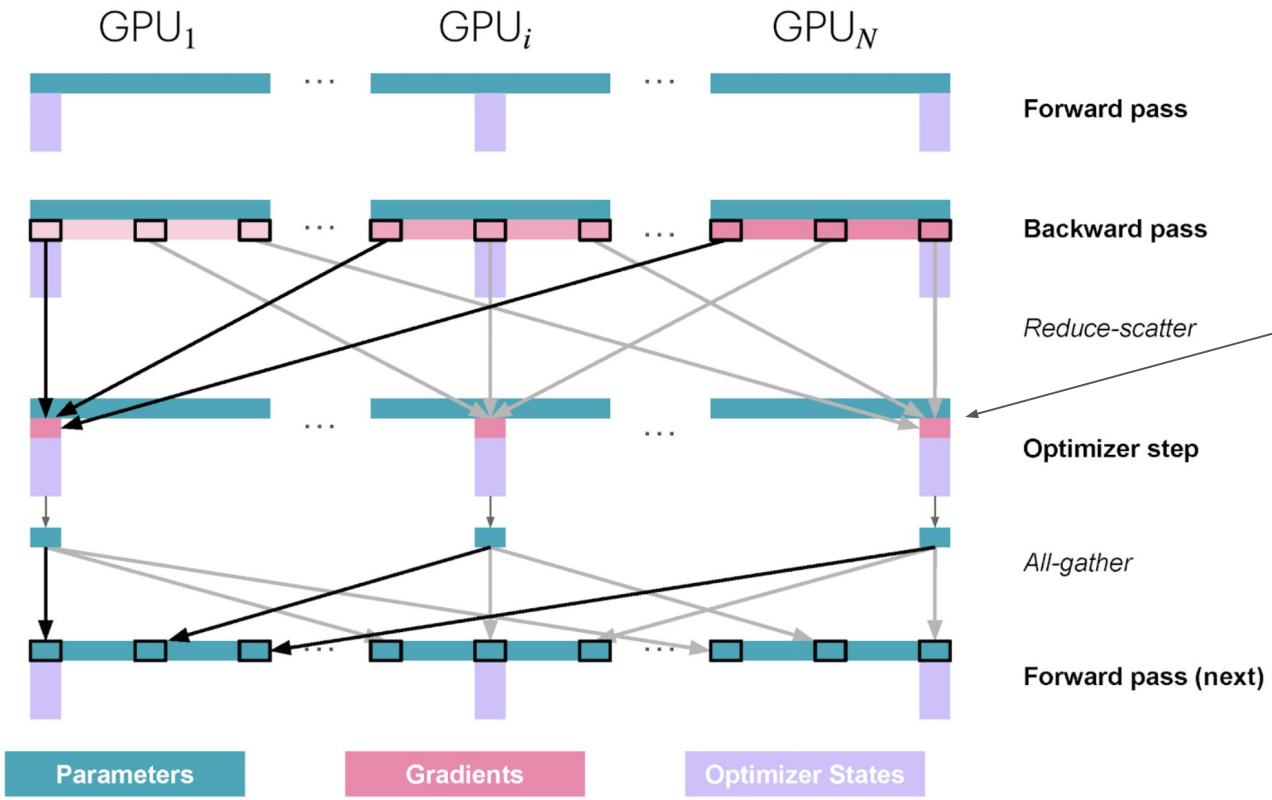
DP: ZeRO1



Do we need to keep the full **grads** in memory?

No, and that's called **DP-Zero2**

DP: ZeRO2

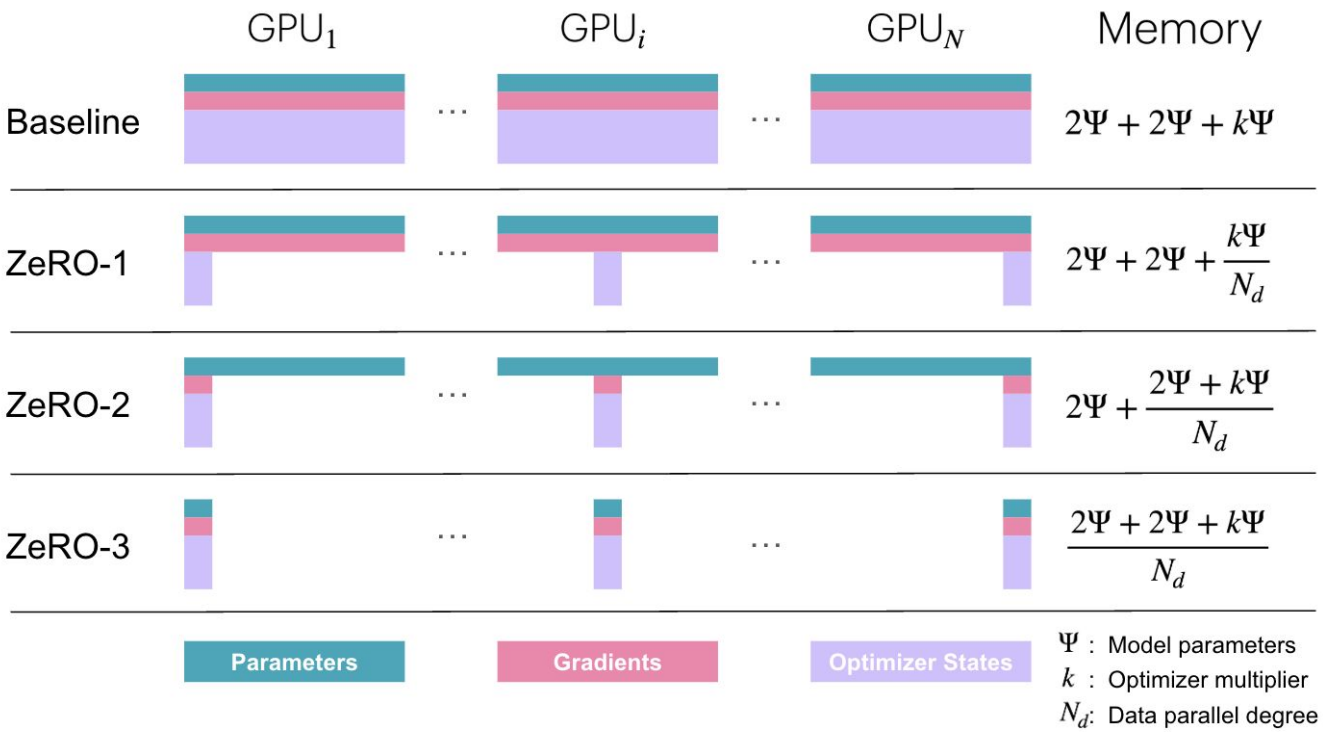


In **ZeRO2** we only keep useful grads -> less memory needed

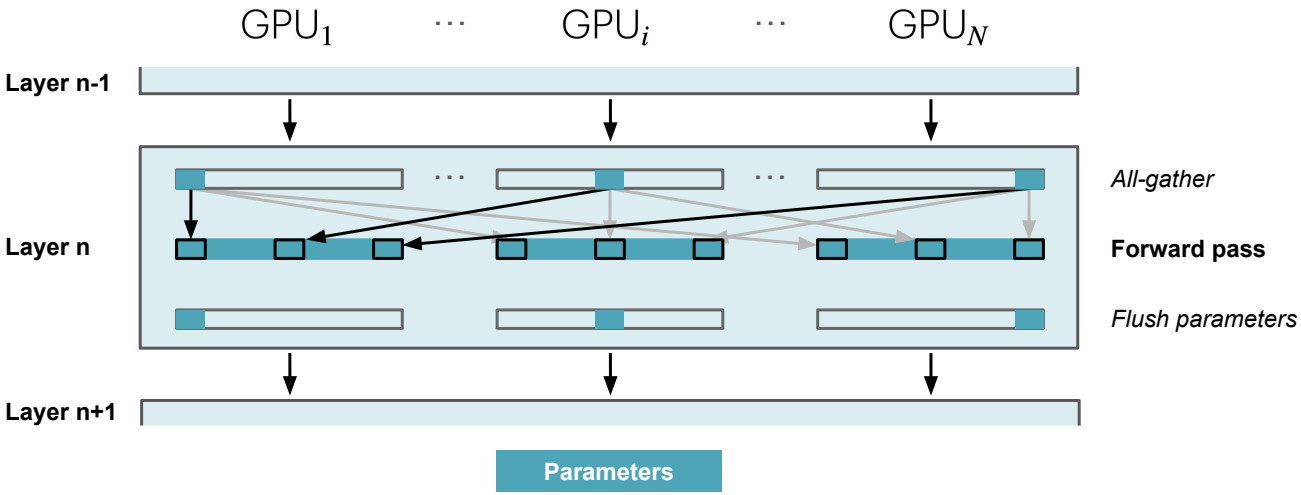
Note: **Muon** optimizer requires full tensors grads, so when sharding our opt_states/grads we ideally want each rank to have **full weights**

DP: ZeRO-3 (aka FSDP)

For **ZeRO-3** we also want to shard our **model parameters memory**, but how would forward work?



DP: ZeRO-3 forward

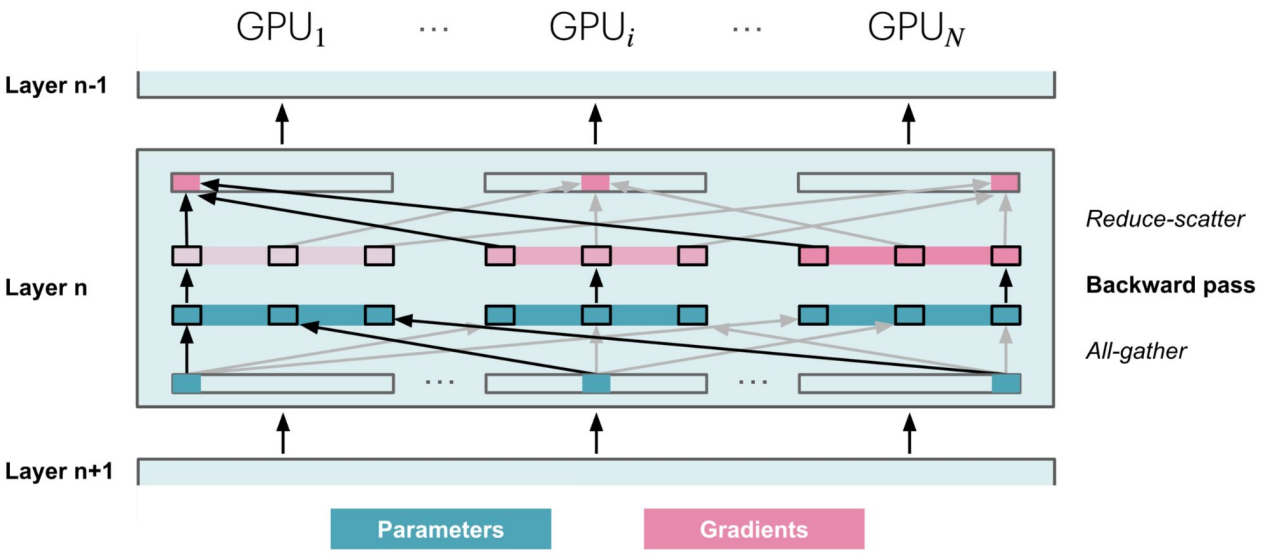


In **ZeRO3** we never materialize the full model

We **All-Gather** the missing params from other ranks when needed then we flush them right after

Each GPU computes forward as if it had the full params (note that **activations** are unsharded)

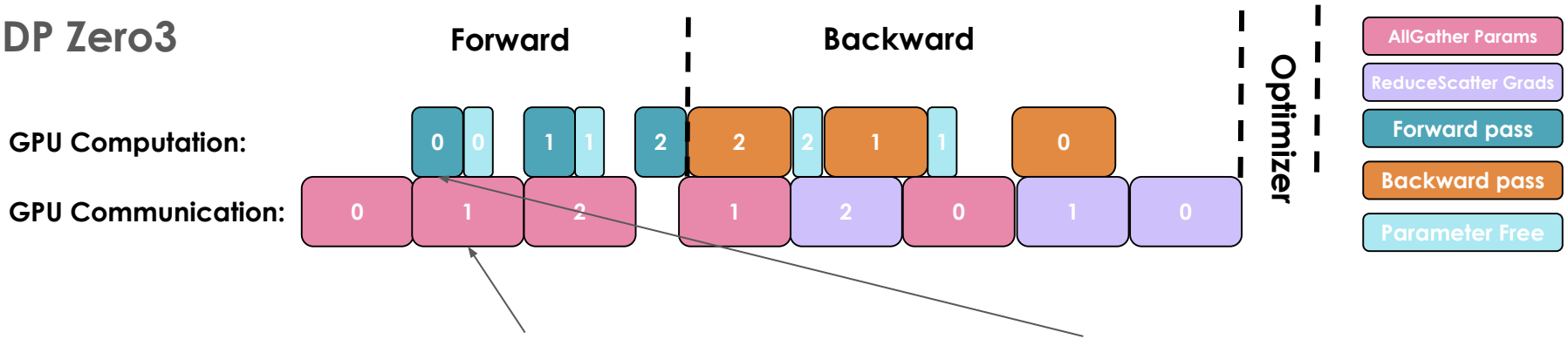
DP: ZeRO-3 backward



Similarly, GPU computes backward as if it had full params (upstream **grads** are unsharded)

We **All-Gather** the missing params to compute grads then we flush them right after.

DP: ZeRO-3 comms/compute timelines



To avoid idle GPU time, we **prefetch** Layer i+1's parameters while computing Layer i's forward

Similar to DP's bucket_size, we can decide what should be the FSDPModule using:

- **auto_wrap_policy** (*Optional[Union[Callable[[nn.Module, bool, int], bool], ModuleWrapPolicy, CustomPolicy]]*) –

  This specifies a policy to apply FSDP to submodules of `module`, which is needed for communication and computation overlap and thus affects performance. If `None`, then FSDP only applies to `module`, and users should manually apply FSDP to parent modules themselves (proceeding bottom-up). For convenience, this accepts `ModuleWrapPolicy` directly, which allows users to specify the module classes to wrap (e.g. the transformer block).

Example code for Zero3 (aka FSDP)

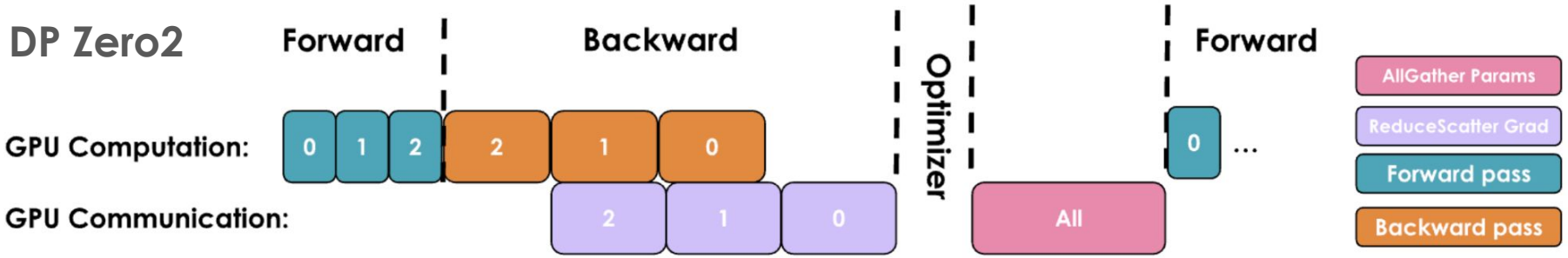
FSDP1 example is done via **FSDP**

```
my_auto_wrap_policy = functools.partial(
        size_based_auto_wrap_policy, min_num_params=20000
    )
torch.cuda.set_device(rank)
model = Net().to(rank)

model = FSDP(model,
    auto_wrap_policy=my_auto_wrap_policy)
```

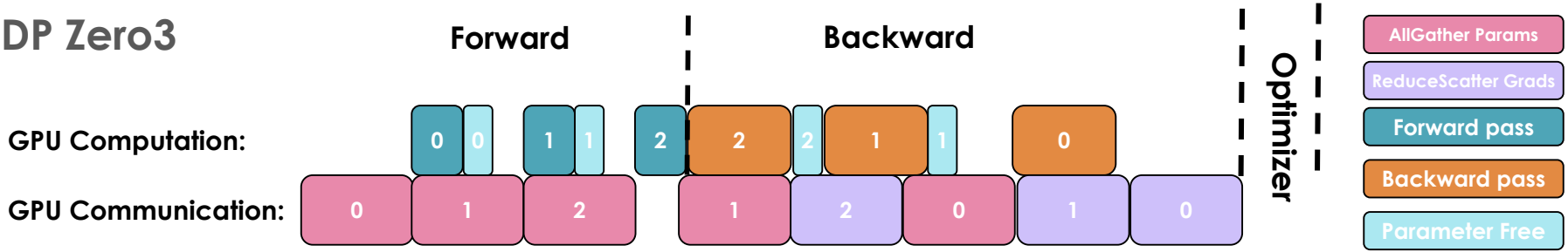
FSDP2 example is done via **fully_shard**

```
from torch.distributed.fsdp import fully_shard, FSDPModule
model = Transformer()
for layer in model.layers:
    fully_shard(layer)
fully_shard(model)

assert isinstance(model, Transformer)
assert isinstance(model, FSDPModule)
print(model)
#  FSDPTransformer(
#    (tok_embeddings): Embedding(...)
#    ...
#    (layers): 3 x FSDPTransformerBlock(...)
#    (output): Linear(...)
#  )
```

DP: Comparing Zero-2 vs ZeRO-3

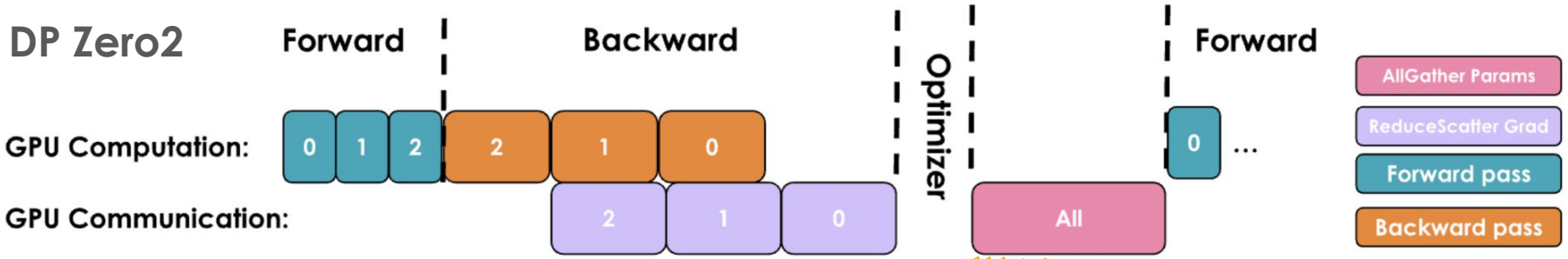


Instead of waiting for **AllGather of the whole model,** we **AllGather it in parts** and **overlap** with compute

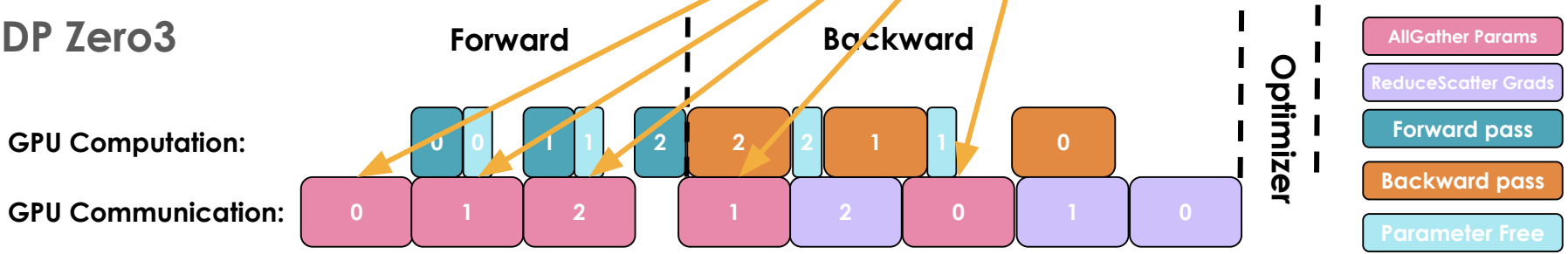


ZeRO-3 helps better overlap comms with compute 👌

DP: Comparing Zero-2 vs ZeRO-3



Instead of waiting for **AllGather of the whole model,** we **AllGather it in parts** and **overlap** with compute



ZeRO-3 helps better overlap comms with compute 👌

DP-ZeRO: Pros/Cons

- \+ Trades memory for comms
- \+ Great overlap of comms with compute
- \+ **Model-agnostic** since we compute fwd/bwd as if we have full tensors

- \- At scale, even ZeRO-3's overlap isn't enough to hide comms ⟶ Hybrid Sharding
- \- **GBS** scales with **number of GPUs** ⟶ can't scale indefinitely

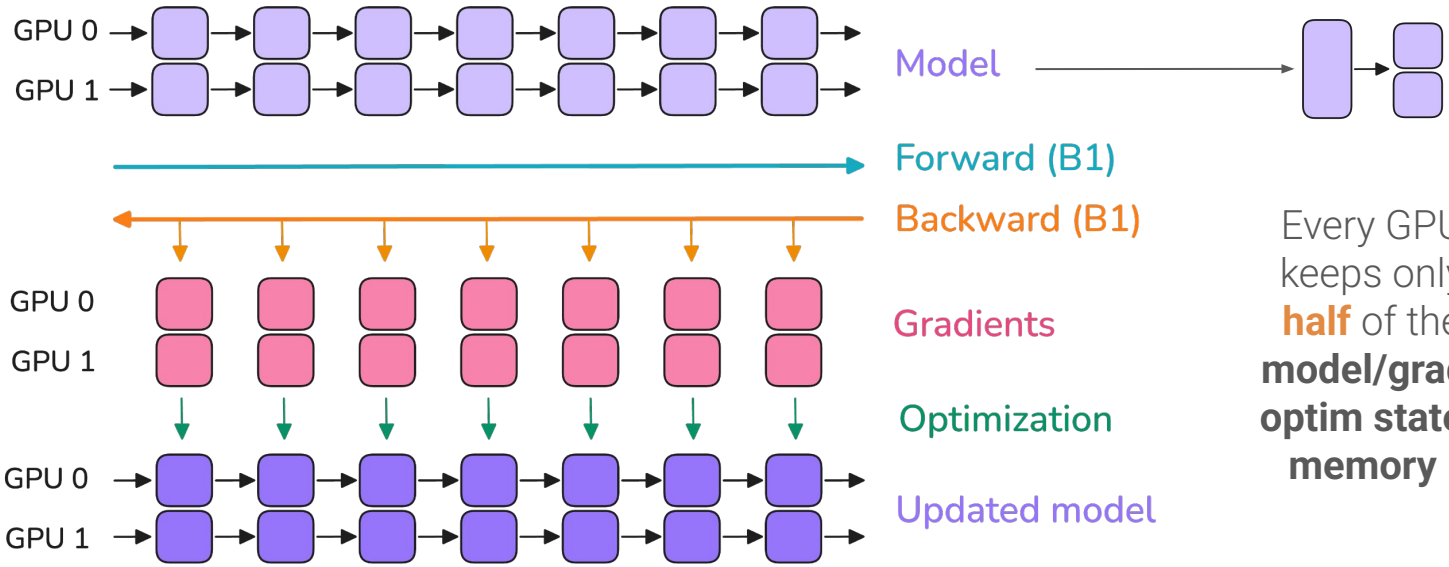
To **avoid increasing GBS** let's look at **Model Parallelism**!

Tensor Parallelism (TP)

Sharding the hidden dimension

Motivation behind Tensor Parallelism

Idea: we want to shard our **model** across GPUs but keep same **data**



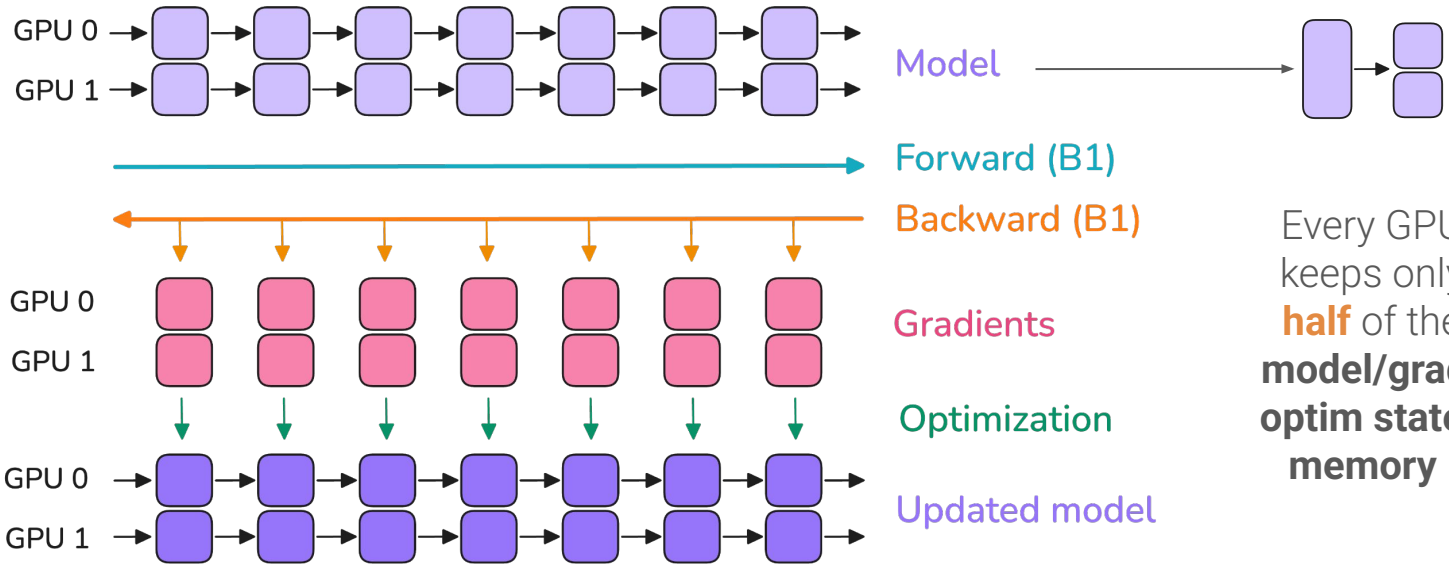
Same batch for both GPUs

Every GPU keeps only **half** of the **model/grad/ optim states memory** !

**Example of TP=2*

Motivation behind Tensor Parallelism

Idea: we want to shard our **model** across GPUs but keep same **data**



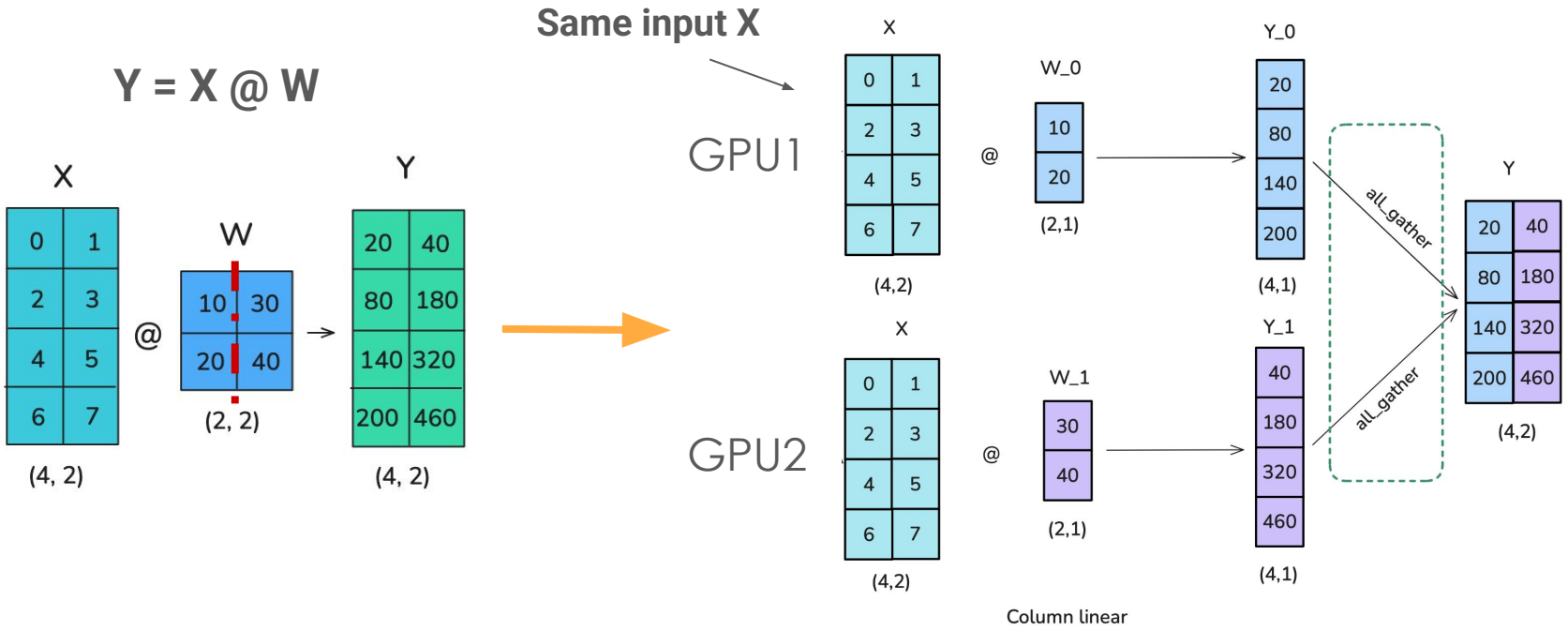
Same batch for both GPUs

Every GPU keeps only **half** of the **model/grad/ optim states memory** !

**Example of TP=2*

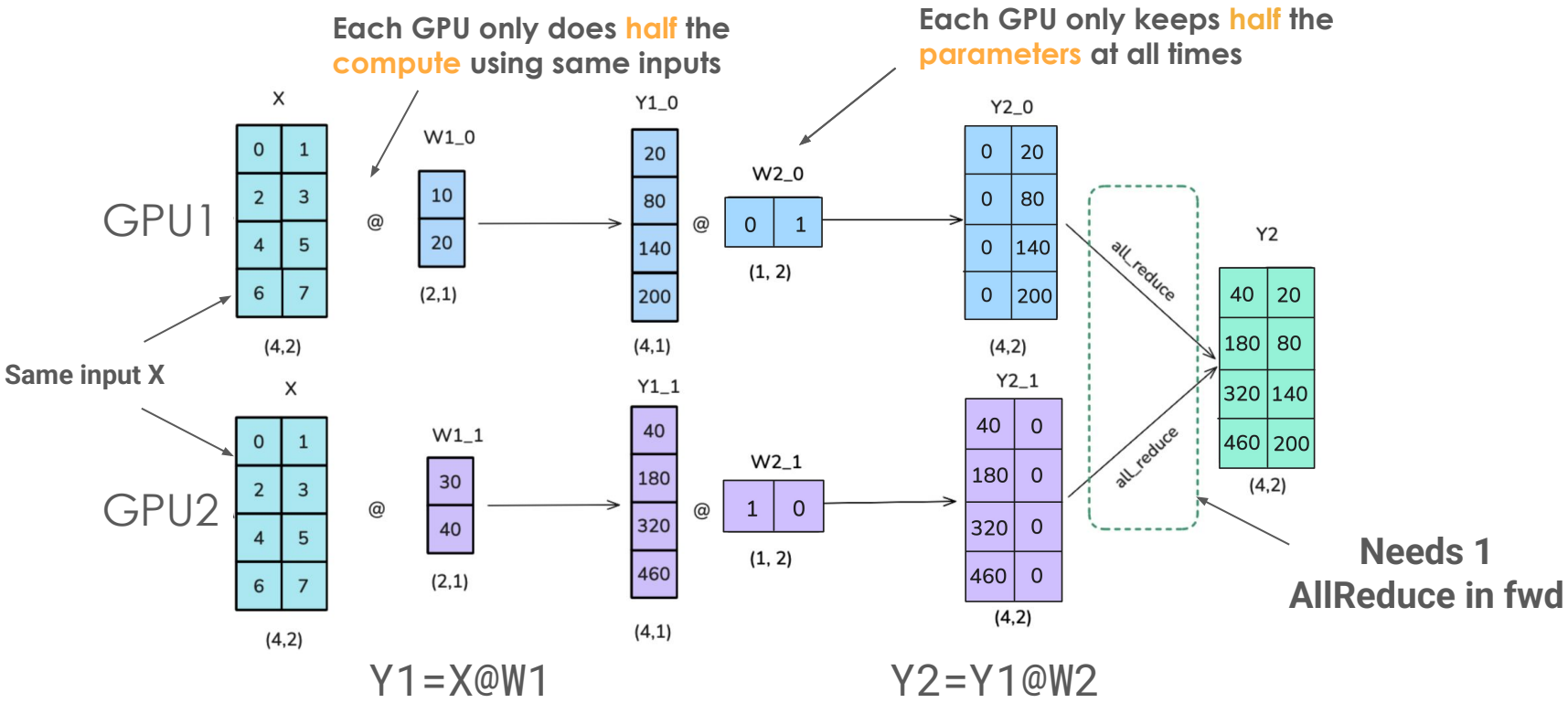
Do we keep full or half activations?

TP: Matmul example



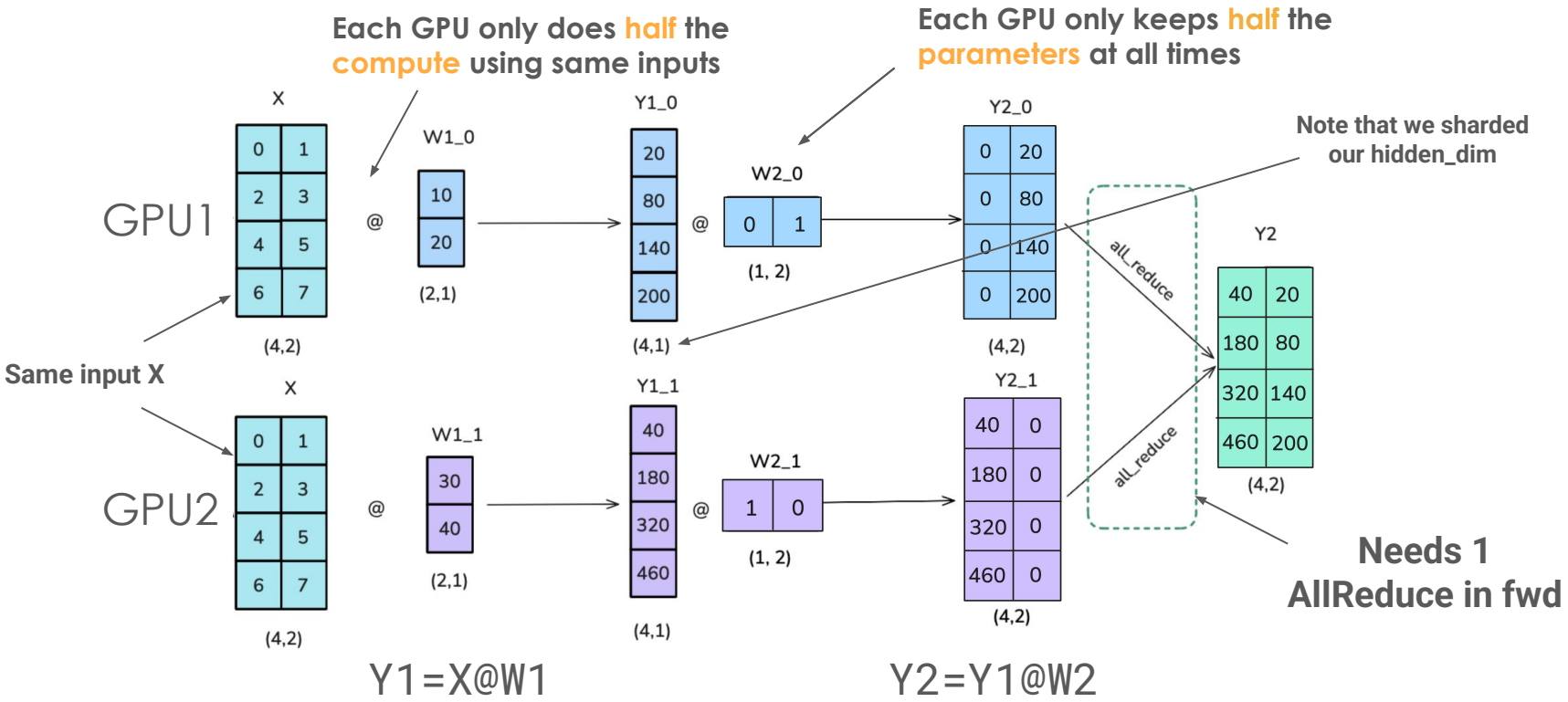
A **matrix multiplication** can be **parallelized** across **GPUs !**

TP: 2 matmuls example



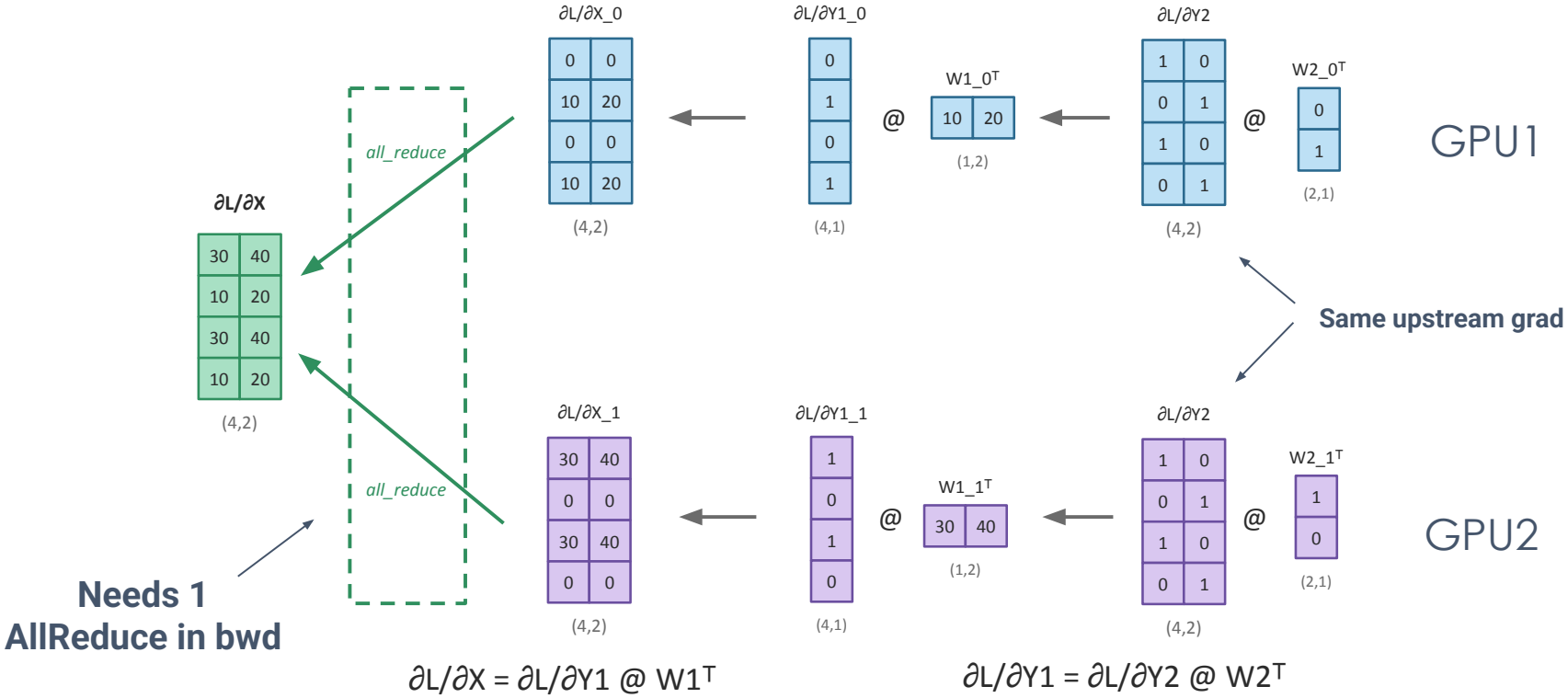
Still works nicely with **2 matrix multiplications !** Note that we assume **same inputs**

TP: 2 matmuls example



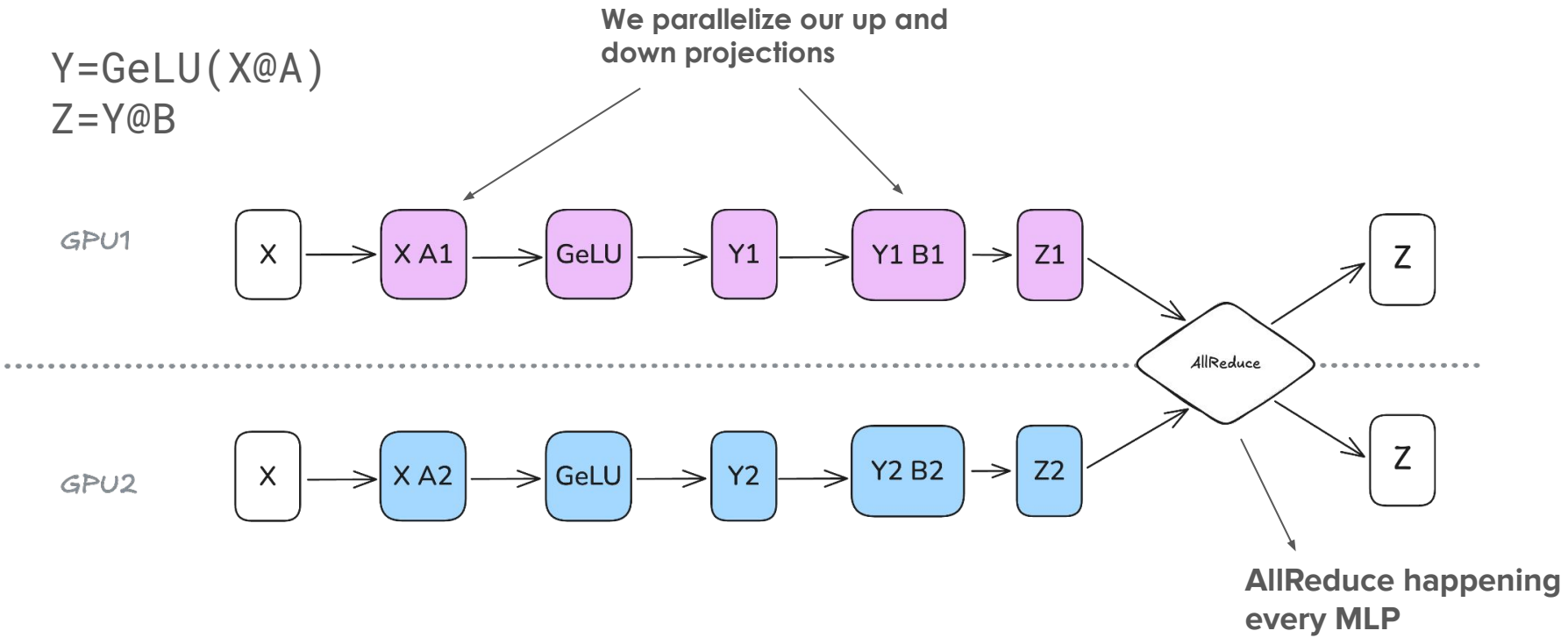
Still works nicely with **2 matrix multiplications !** Note that we assume **same inputs**

TP: 2 matmuls example (bwd pass)

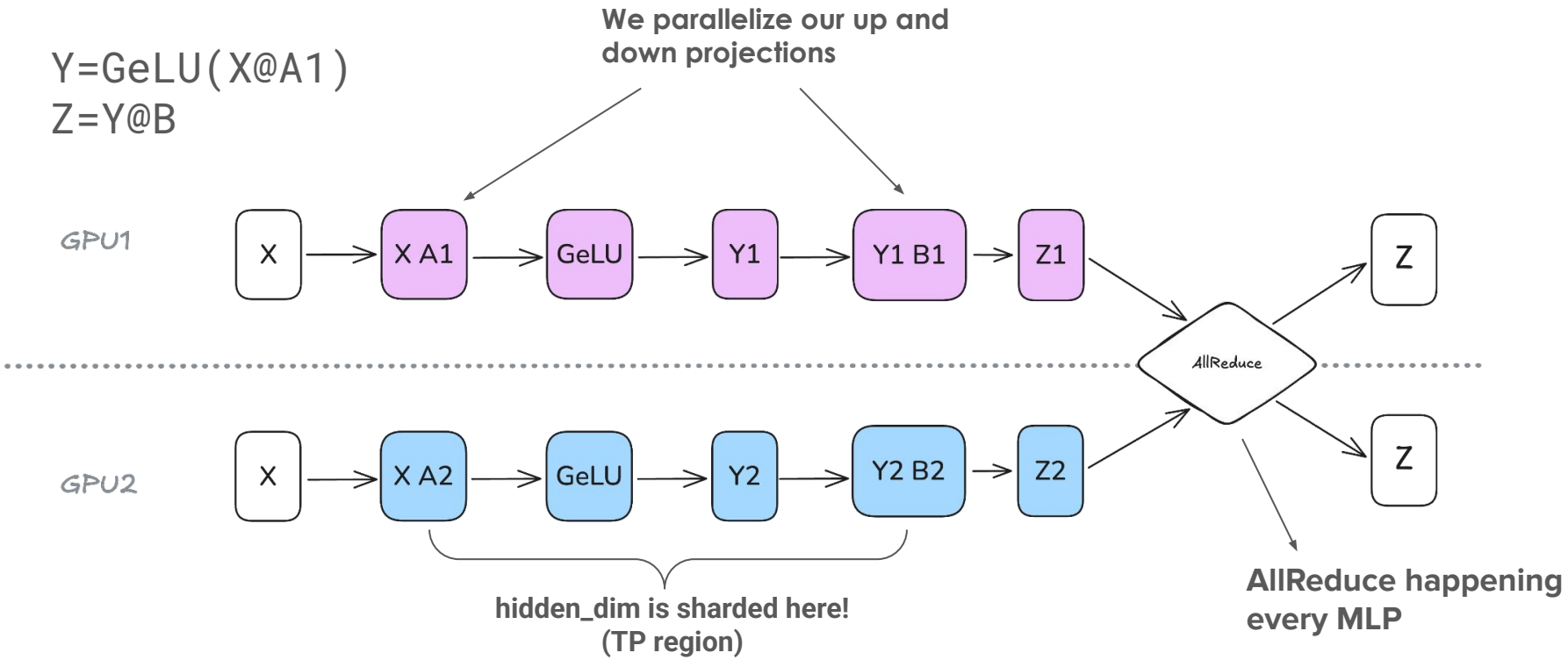


For backward we inverse the order but still assume **same upstream grad**

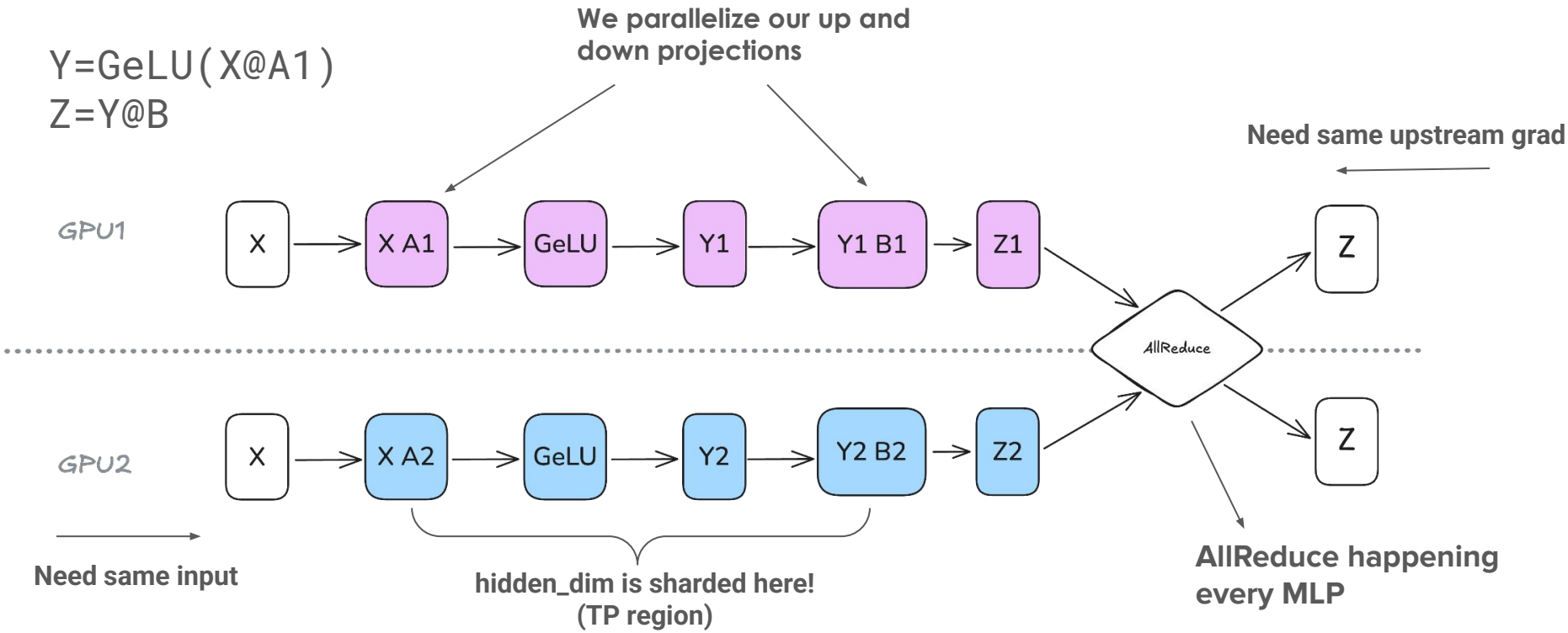
TP: MLP Example



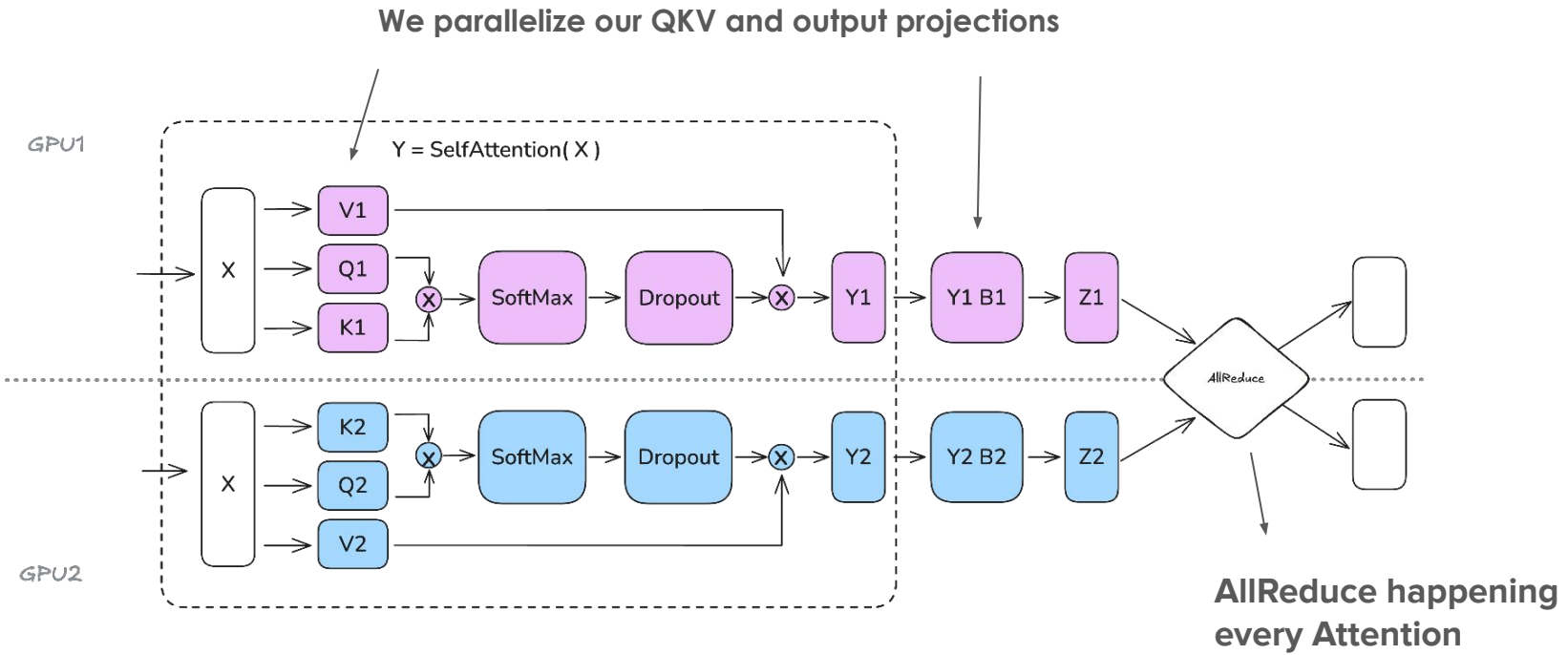
TP: MLP Example



TP: MLP Example



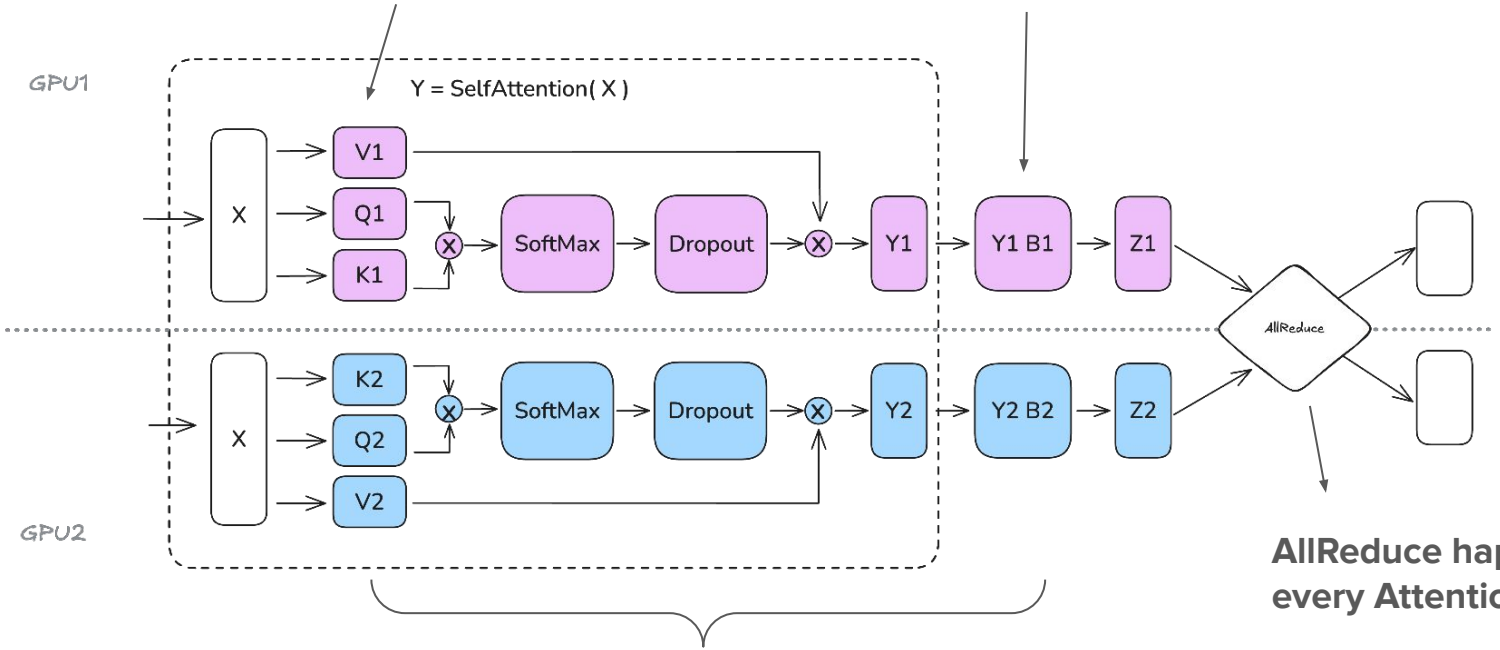
TP: Attention Example



Should we shard along head_dim or num_heads or both?
Would GPUs compute the same activations as without TP?

TP: Attention Example

We parallelize our QKV and output projections

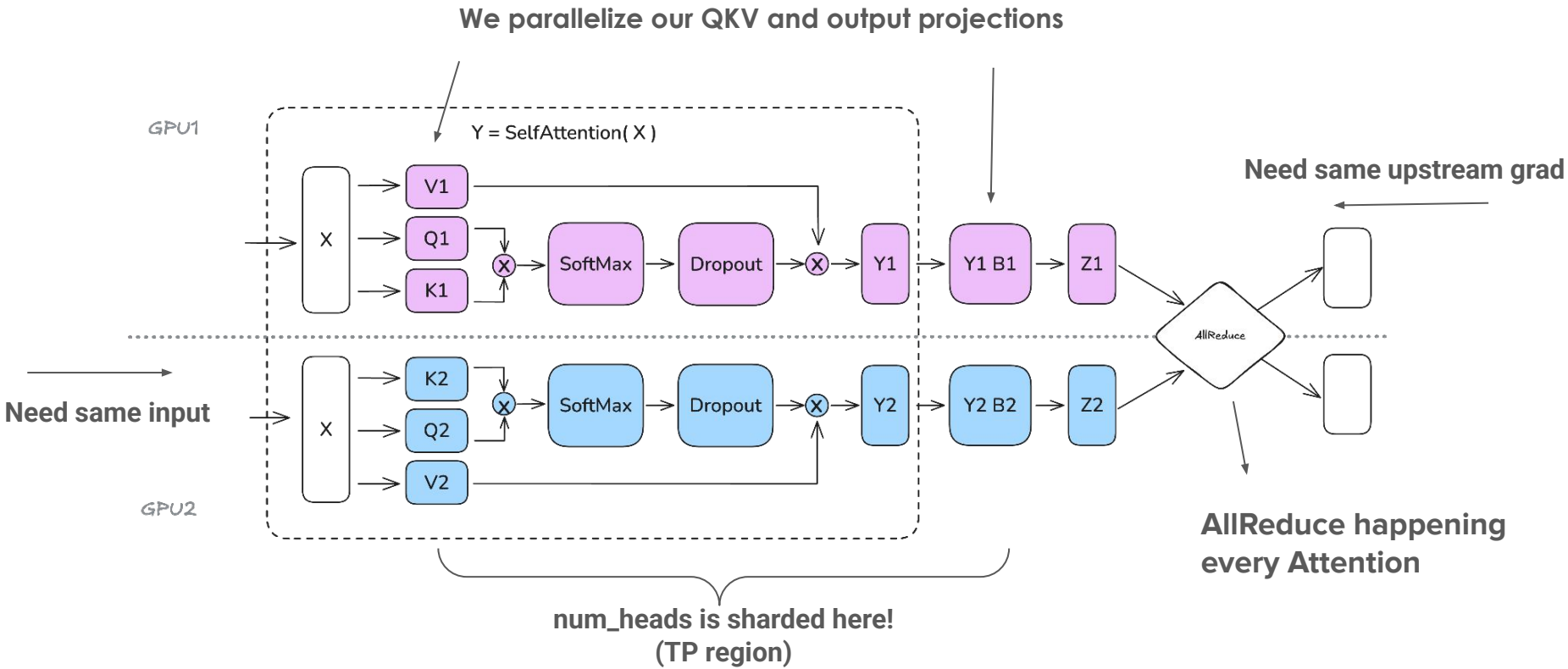


AllReduce happening every Attention

num_heads is sharded here!
(TP region)

We need to shard along num_heads dim to keep Attention correct !

TP: Attention Example



We need to shard along num_heads dim to keep Attention correct !

TP: Pros/Cons

+ Shards model and compute across GPUs -> **memory & sample efficient**
+ No need for grad AllReduce because GPUs are essentially doing the same work

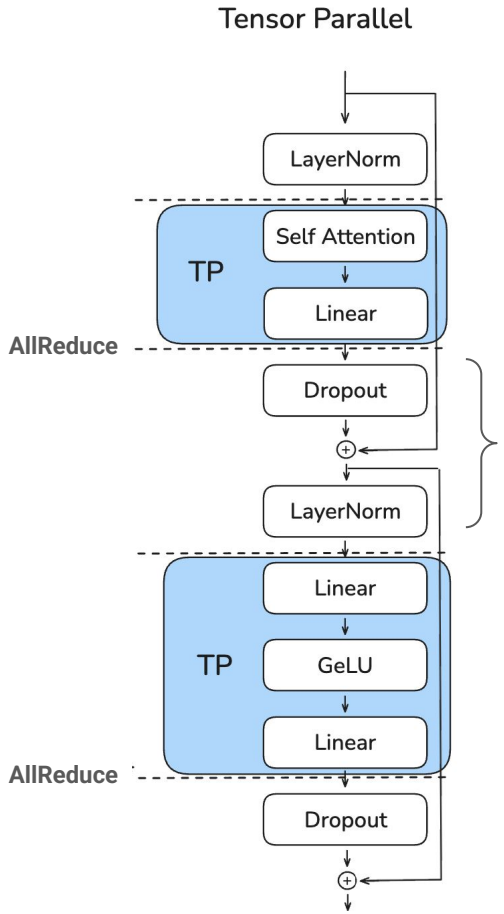
- **Communication heavy** (inside every block, part of critical path)
- Parameters (e.g LayerNorm) outside the TP region should **stay synchronized**. In practice we **AllReduce** their grads to make sure they don't drift apart
- **Complex implementation** and assumes operations to be parallelizable

Let's look at a slightly different version of TP called **Sequence Parallelism**!

TP&SP: Motivation

Activations stored here are of shape (s, b, h/TP)

Activations stored here are of shape (s, b, h)



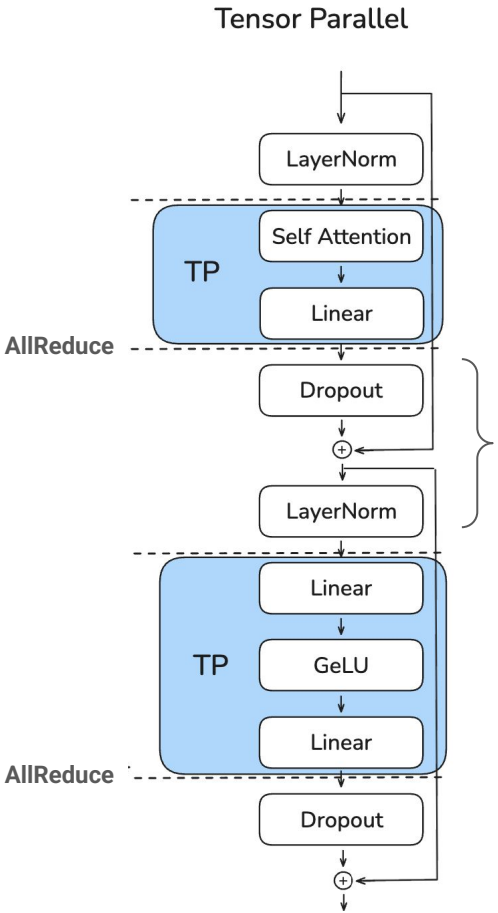
These ops are replicated across GPUs, can we distribute them too?

Note that LayerNorm needs hidden_dim to be full

TP&SP: Motivation

Activations stored here are of shape (s, b, h/TP)

Activations stored here are of shape (s, b, h)

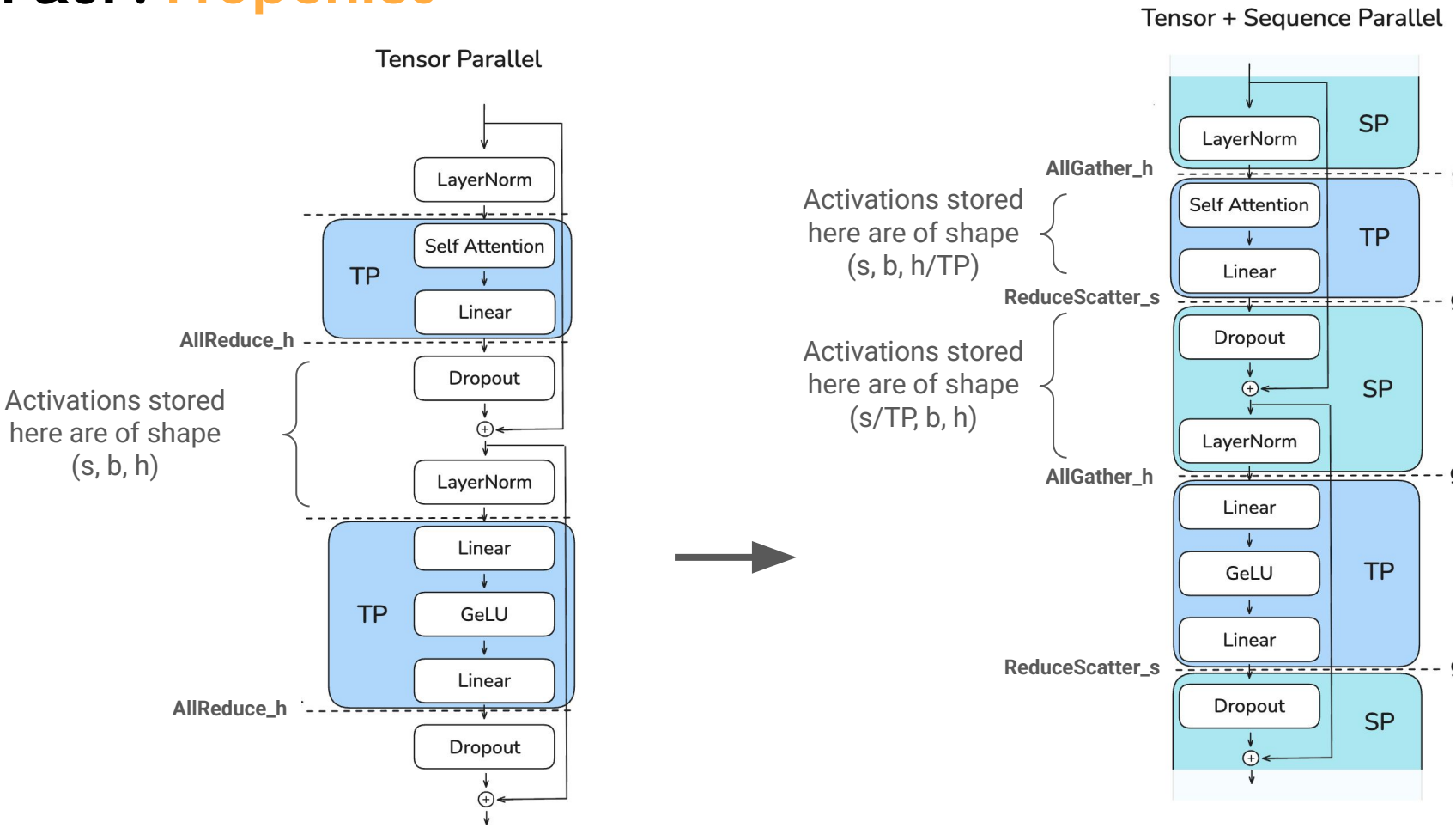


These ops are replicated across GPUs, can we distribute them too?

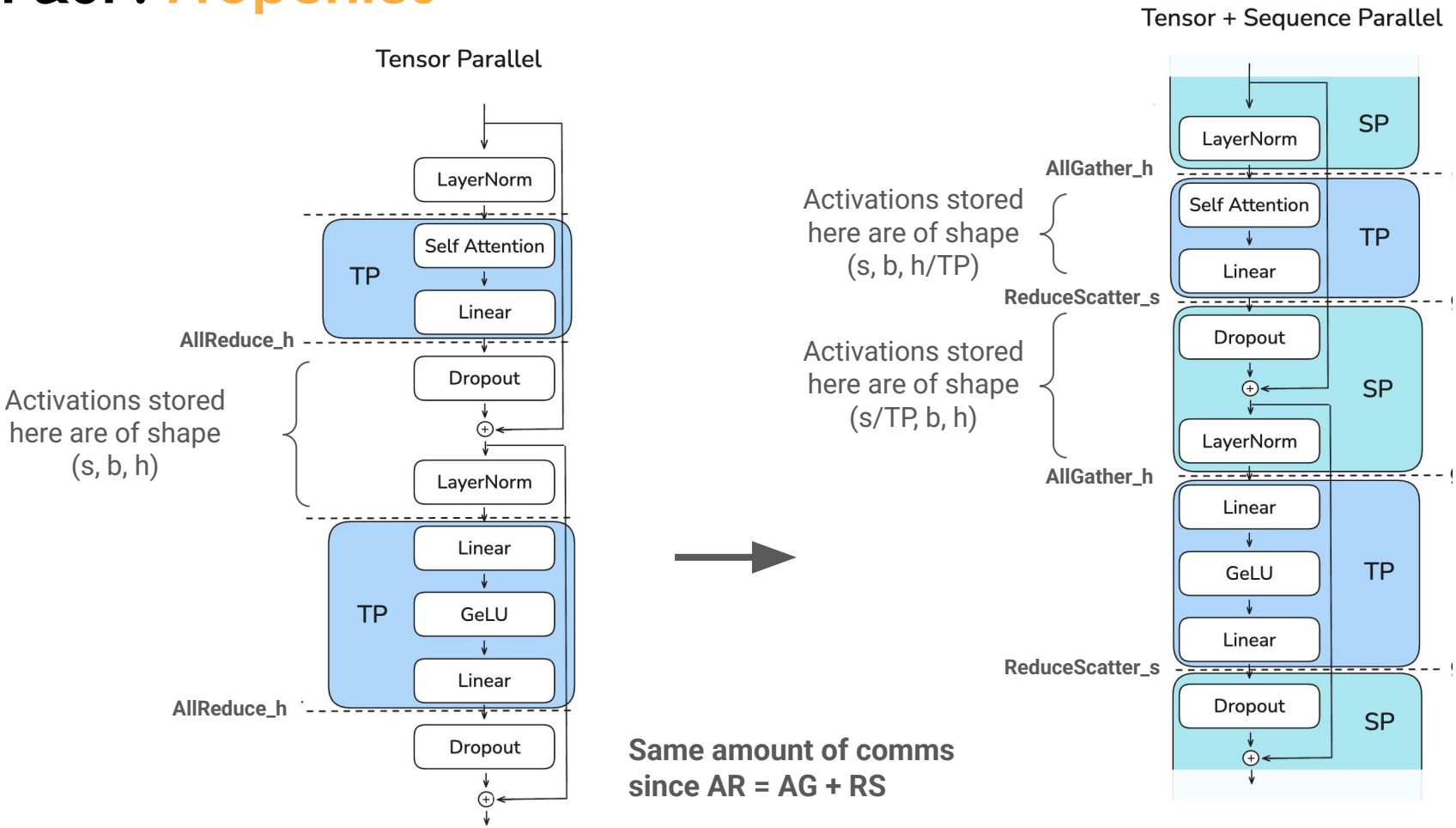
Instead of **AllReduce** to get back full hidden_dim let's **ReduceScatter** on sequence_dim and **AllGather** before linear

Note that LayerNorm needs hidden_dim to be full

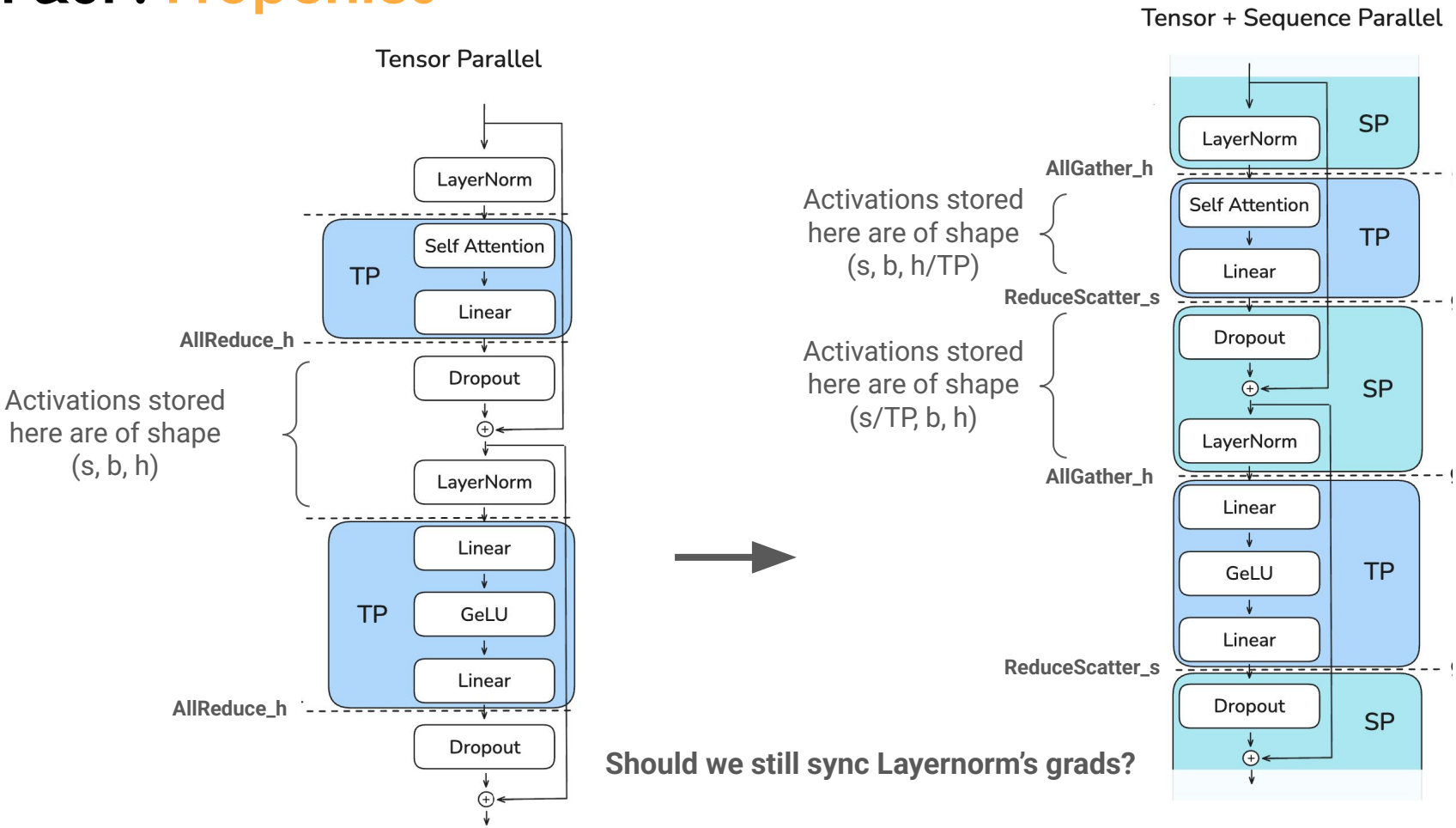
TP&SP: Properties



TP&SP: Properties

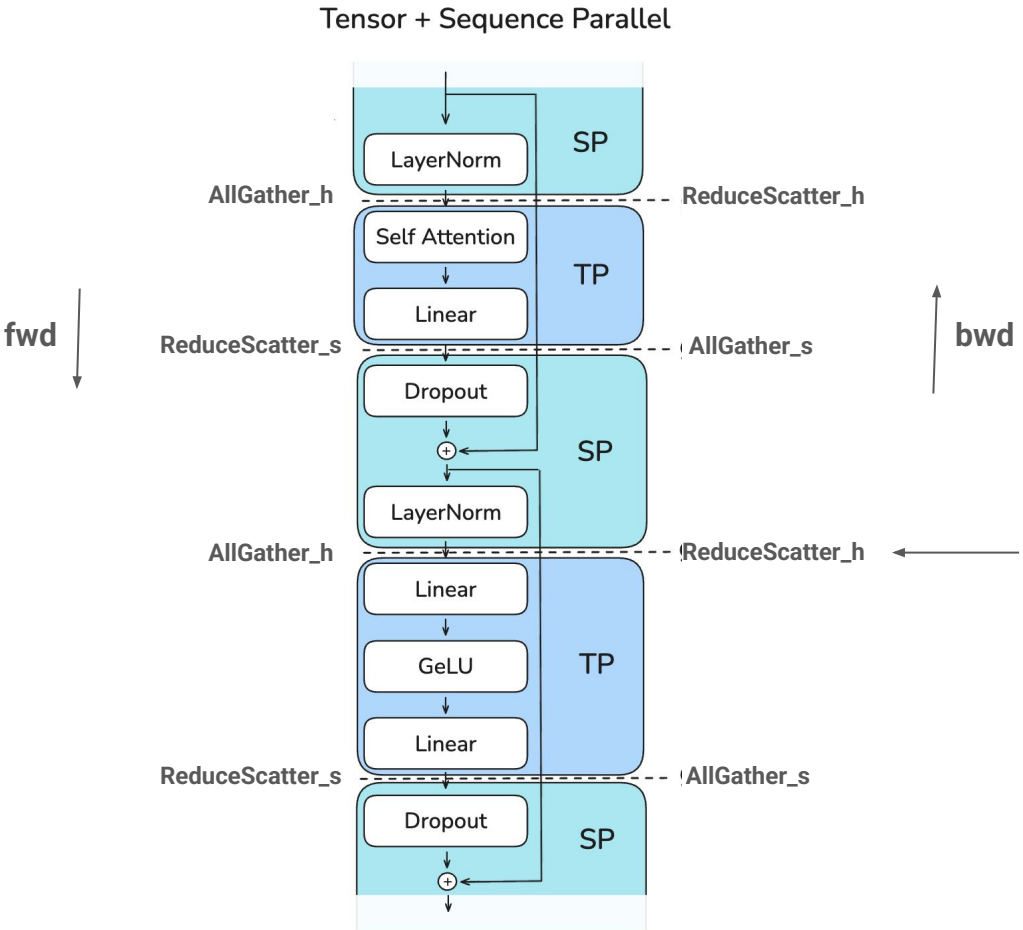


TP&SP: Properties



Should we still sync Layernorm's grads?

TP&SP: Backward of AllGather/ReduceScatter



ReduceScatter before **SP regions** help keep Layernorms synchronized

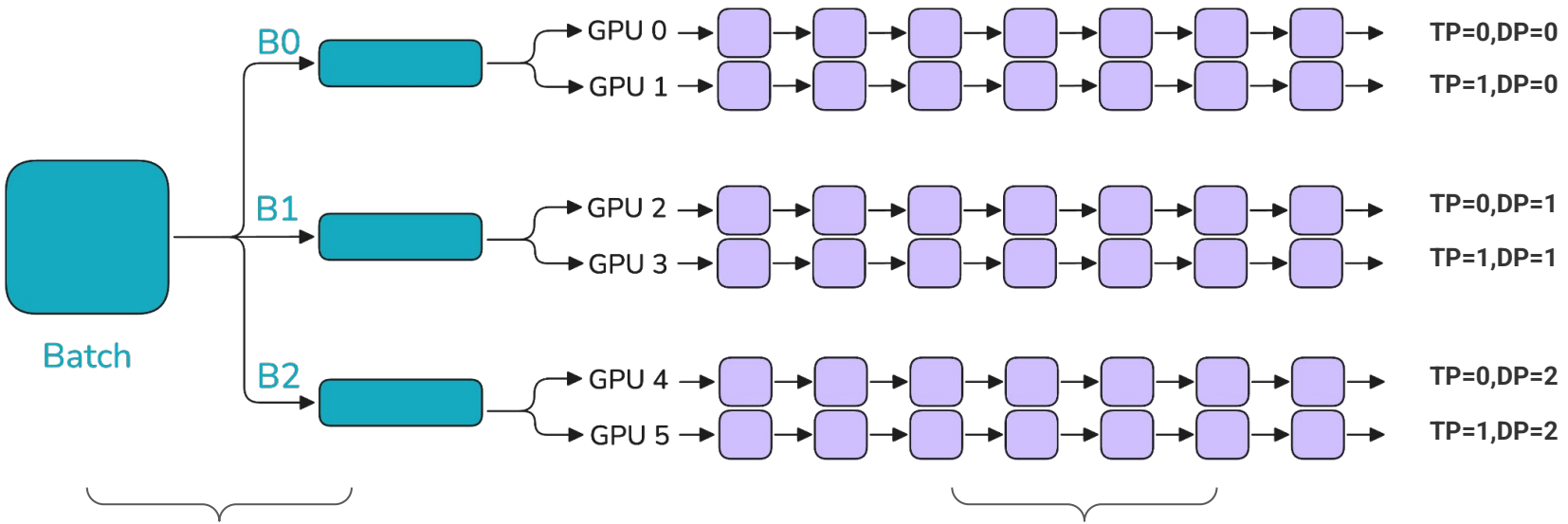
No need to sync Layernorm's grads

TP&SP: Pros/Cons

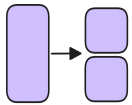
- \+ Shards model and compute across GPUs -> **memory & sample efficient**
- \+ No need for grad AllReduce because GPUs are essentially doing the same work

- \- **Communication heavy** (inside every block, part of critical path)
- \- **Complex implementation** and assumes operations to be parallelizable

Combining parallelisms?

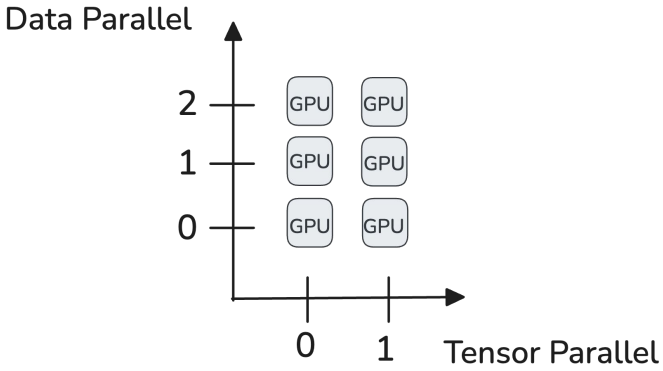


We shard our **data** along **Data Parallel axis** (e.g. DP, CP) and duplicate along other axes



We shard our **model** along **Model Parallel axis** (e.g. TP, PP) and duplicate along other axes

Example code



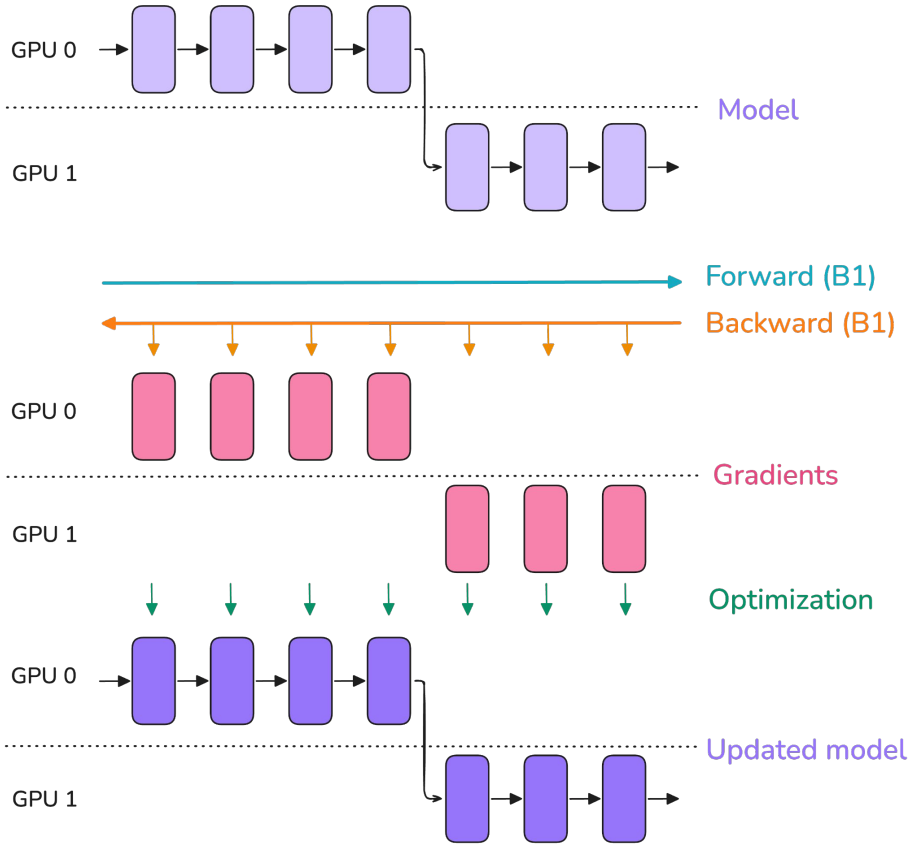
We pick which parallelism axis to AllReduce over

```python
import os
import torch
import torch.distributed as dist

def main():
    # torchrun sets these env vars; launch with: torchrun --nproc_per_node=6 script.py
    rank = int(os.environ["RANK"])
    world_size = int(os.environ["WORLD_SIZE"])  # must be 6 (DP=3 * TP=2)

    dist.init_process_group(backend="nccl")
    torch.cuda.set_device(rank % torch.cuda.device_count())

    DP, TP = 3, 2
    dp_idx = rank // TP
    tp_idx = rank %  TP

    tp_groups = [dist.new_group(ranks=[d * TP + t for t in range(TP)]) for d in range(DP)]
    tp_group  = tp_groups[dp_idx]

    dp_groups = [dist.new_group(ranks=[d * TP + t for d in range(DP)]) for t in range(TP)]
    dp_group  = dp_groups[tp_idx]

    # ── All-reduce along ONE dimension ──
    x = torch.full((4,), float(rank), device="cuda")

    # (a) all-reduce across TP only (e.g. summing partial activations from row-parallel linear)
    dist.all_reduce(x, op=dist.ReduceOp.SUM, group=tp_group)

    # (b) all-reduce across DP only (e.g. gradient averaging)
    dist.all_reduce(x, op=dist.ReduceOp.SUM, group=dp_group)
```

Pipeline Parallelism (PP)

Sharding the model layers

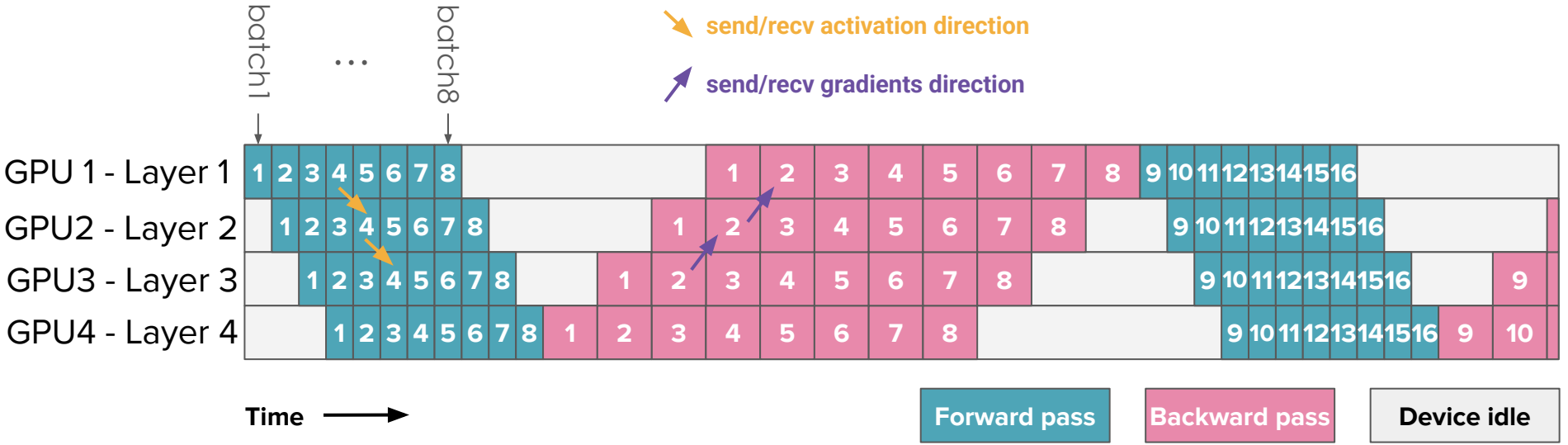
Intuition behind Pipeline Parallelism



We want to **shard layers** across GPUs

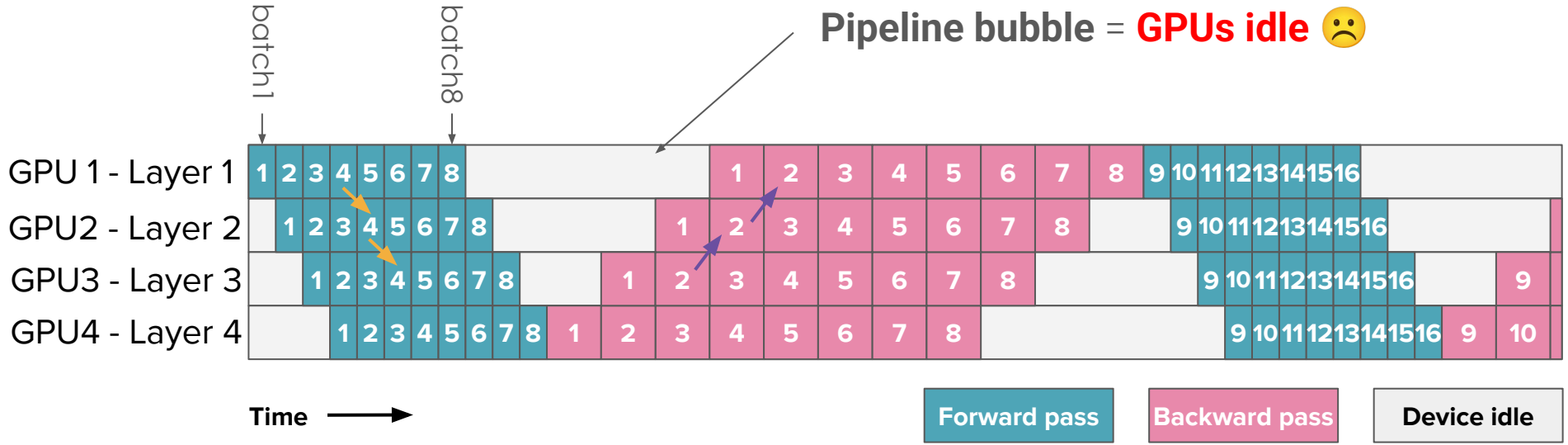
.. But what does GPU do when it **doesn't have activations yet**?

PP: AFAB scheduler (All-Forward All-Backward)



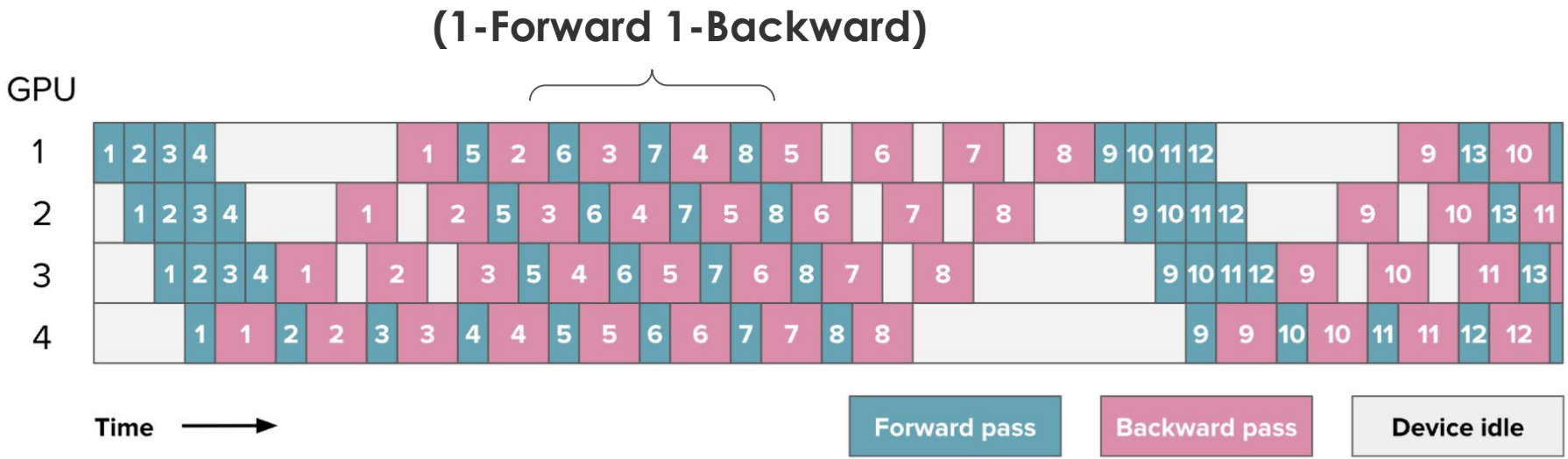
- Let's assume each GPU has 1 layer
- We communicate **activations/grads** between **pairs of GPUs**
- We schedule many microbatches to keep GPU busy

PP: Pipeline bubble



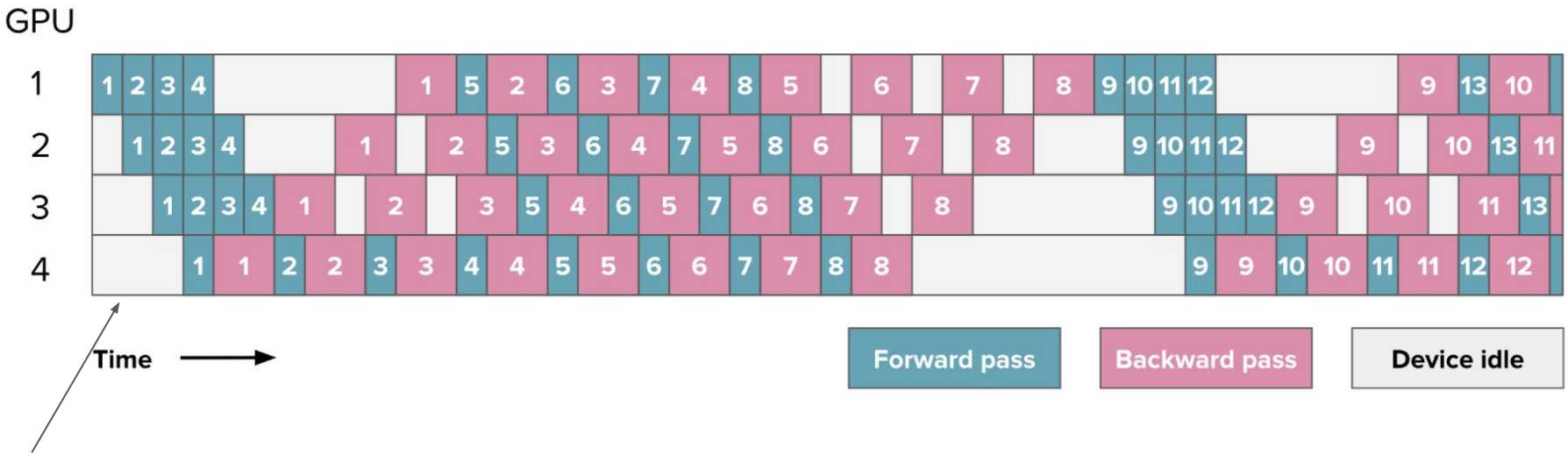
- Let's assume each GPU has 1 layer
- We communicate **activations/grads** between **pairs of GPUs**
- We schedule many microbatches to keep GPU busy

PP: 1F1B Scheduler



- We prioritize backwards over forwards
- Still same pipeline bubble

PP: 1F1B Scheduler



If only we could start doing **forwards** from this side too..

PP: Deepseek's pipeline scheduler

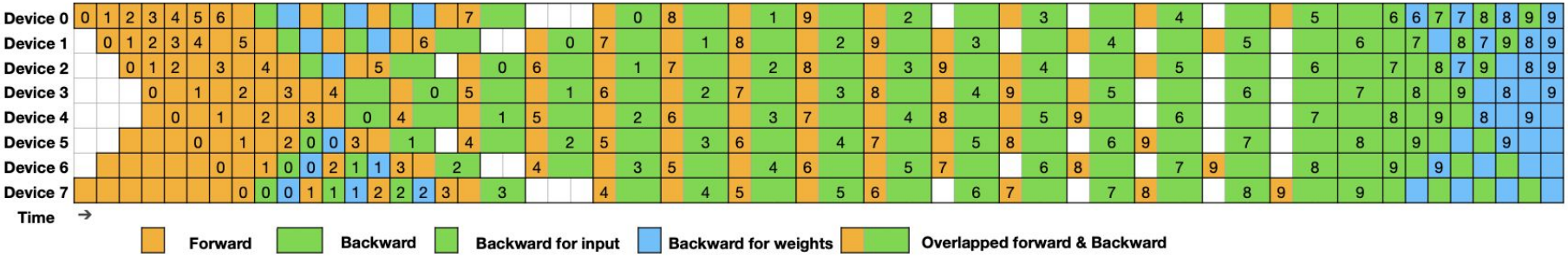


Figure 5 | Example DualPipe scheduling for 8 PP ranks and 20 micro-batches in two directions. The micro-batches in the reverse direction are symmetric to those in the forward direction, so we omit their batch ID for illustration simplicity. Two cells enclosed by a shared black border have mutually overlapped computation and communication.

DualPipe is an advanced **PP schedule** where we **inject data** from **both PP ends** to **minimize** PP bubble.. Easier said than done :)

PP: Pros/Cons

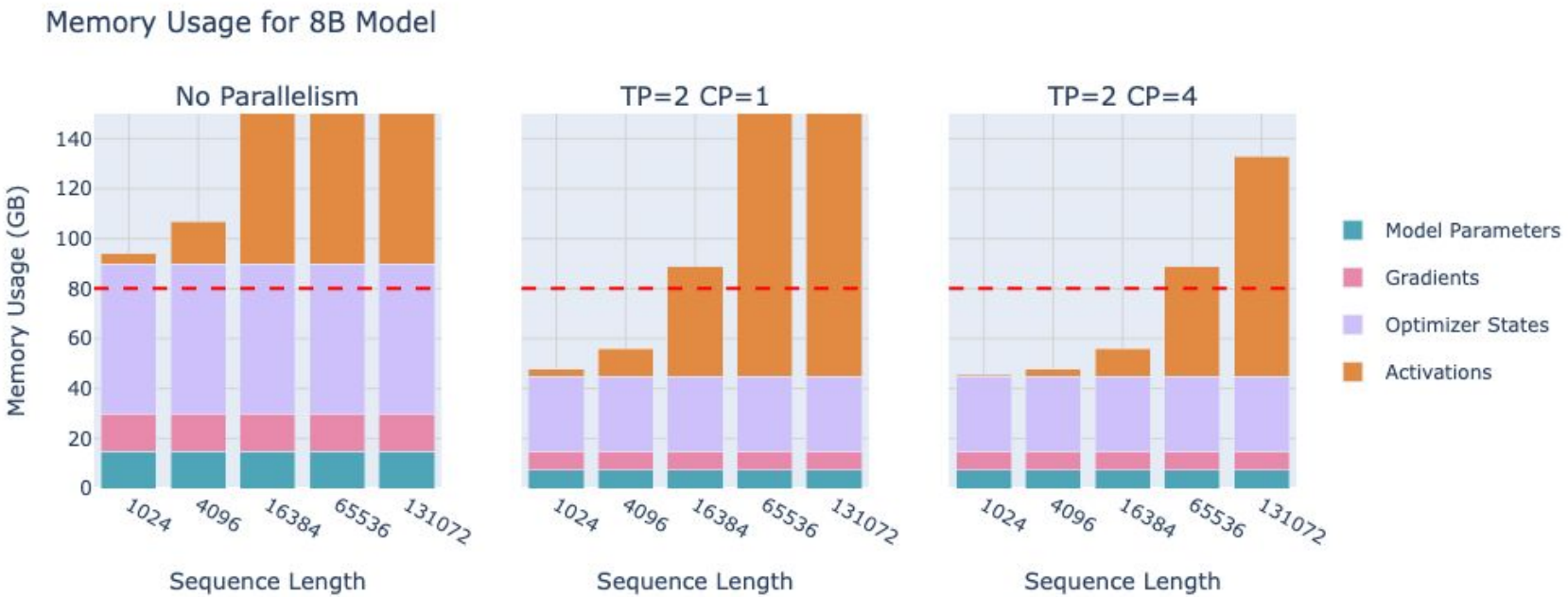
- \+ **Cheap communications** (activs/grads every pipeline stage)
- \+ Shards model across GPUs -> memory efficient
- \+ **Model-agnostic**
- \+ No need to AllReduce grads

- \- Need to save activations for multiple microbatches before backward
- \- **Complex implementation** for advanced pipeline schedulers
- \- Needs **multiple microbatches** to hide PP bubble

Context Parallelism (CP)

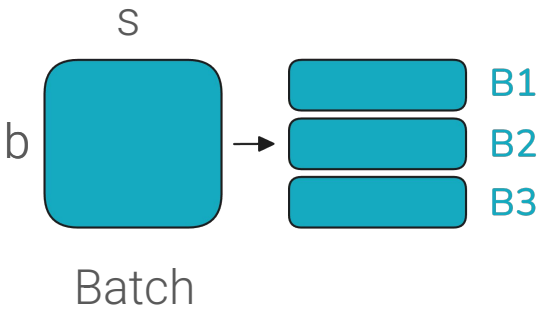
Sharding the sequence length dimension

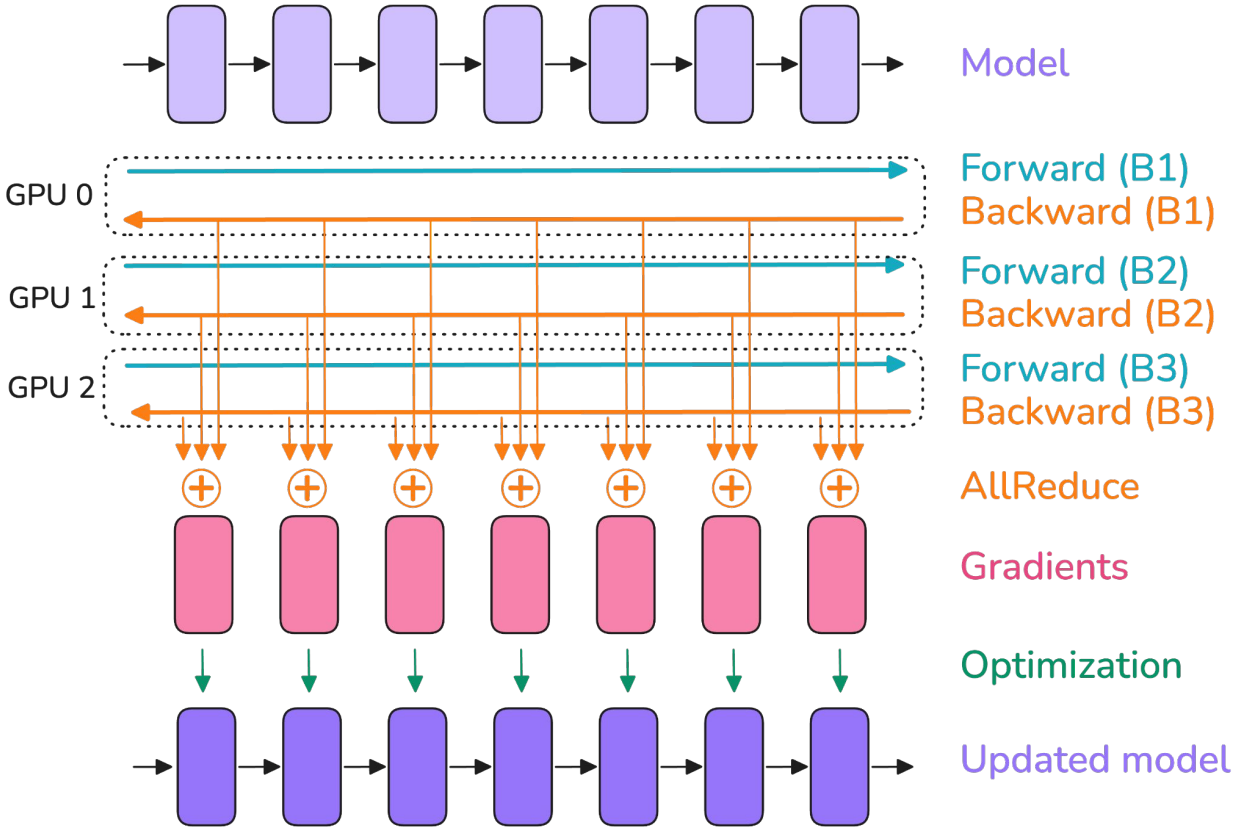
Motivation for Context Parallelism



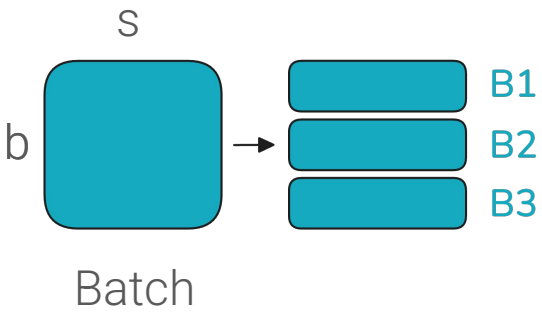
Scaling sequence length is no joke...

Previously in Data Parallelism

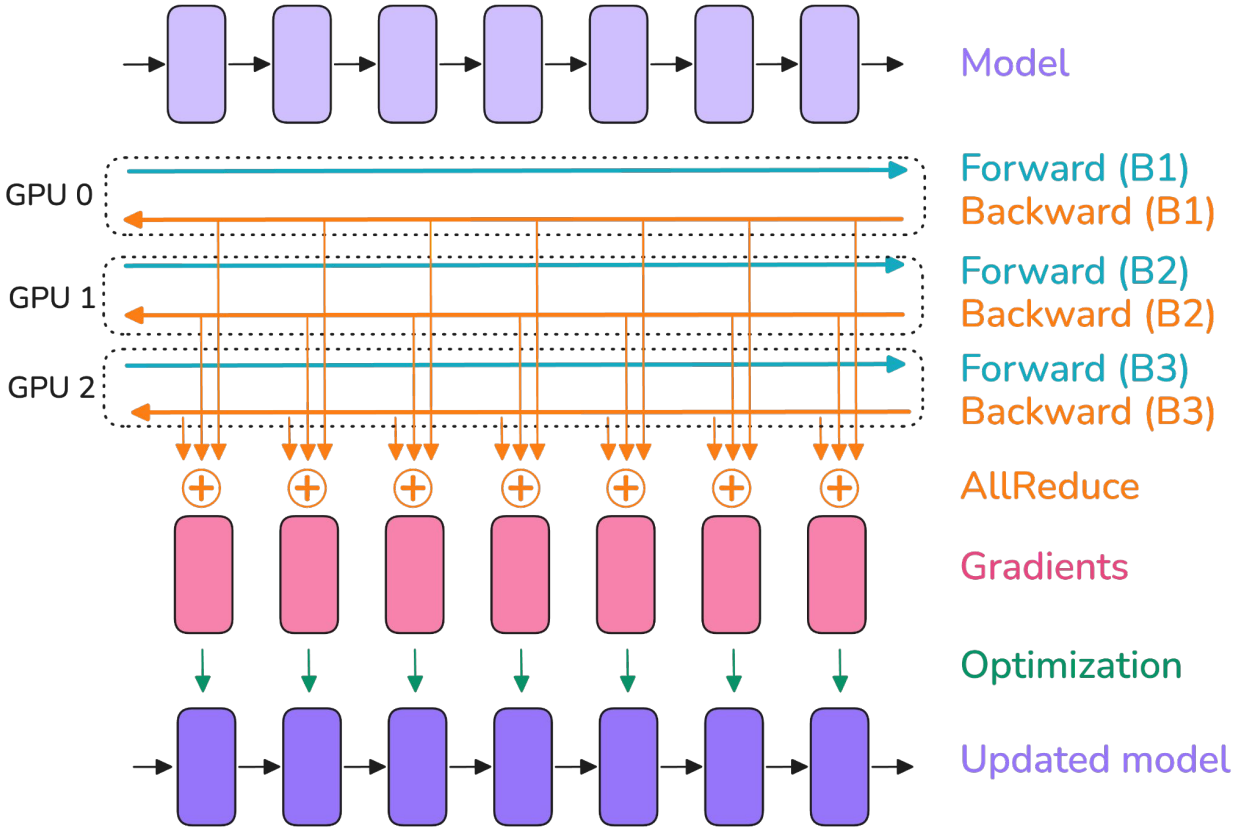




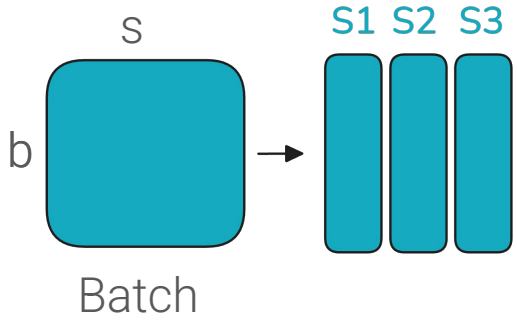
Previously in Data Parallelism



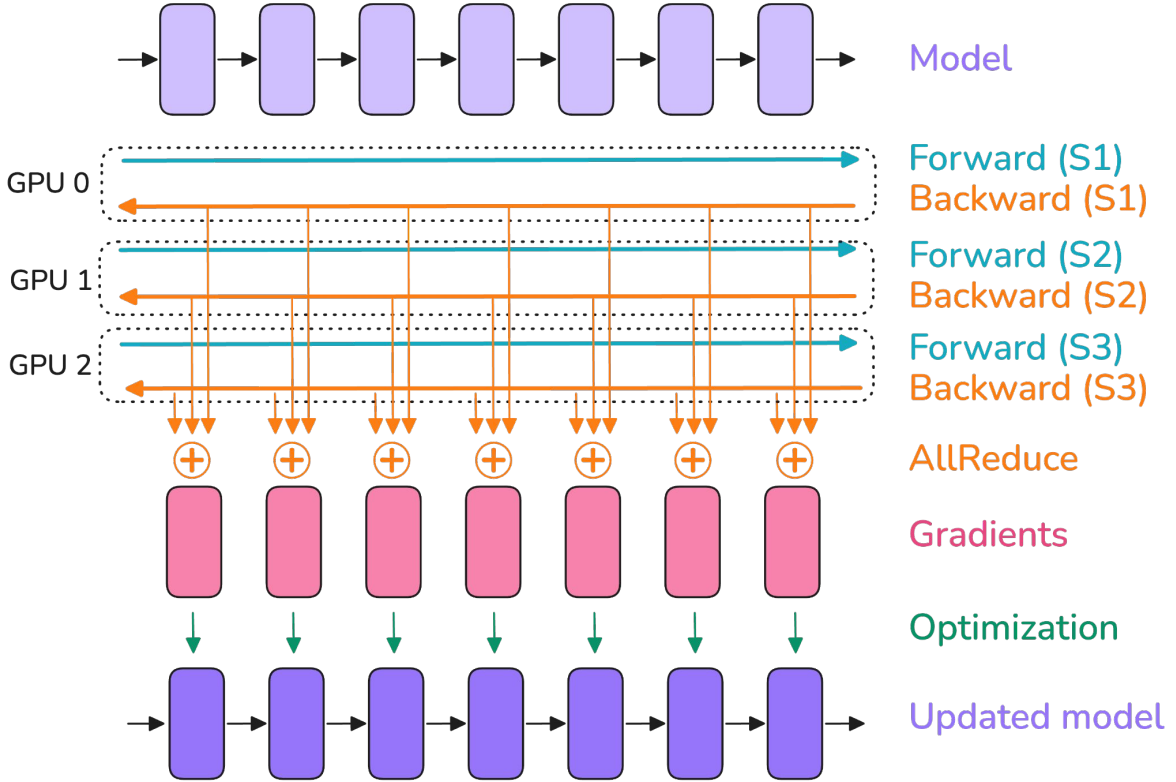
What if **sequence length** is **too large**?



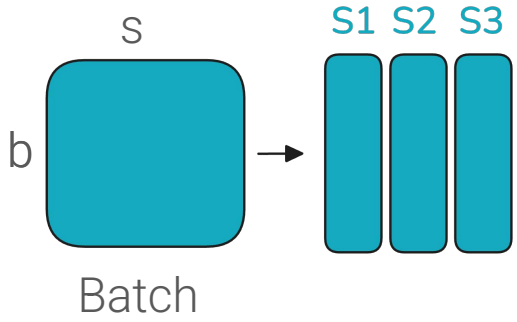
Intuition behind Context Parallelism



In **Context Parallelism** we shard data along **sequence length**



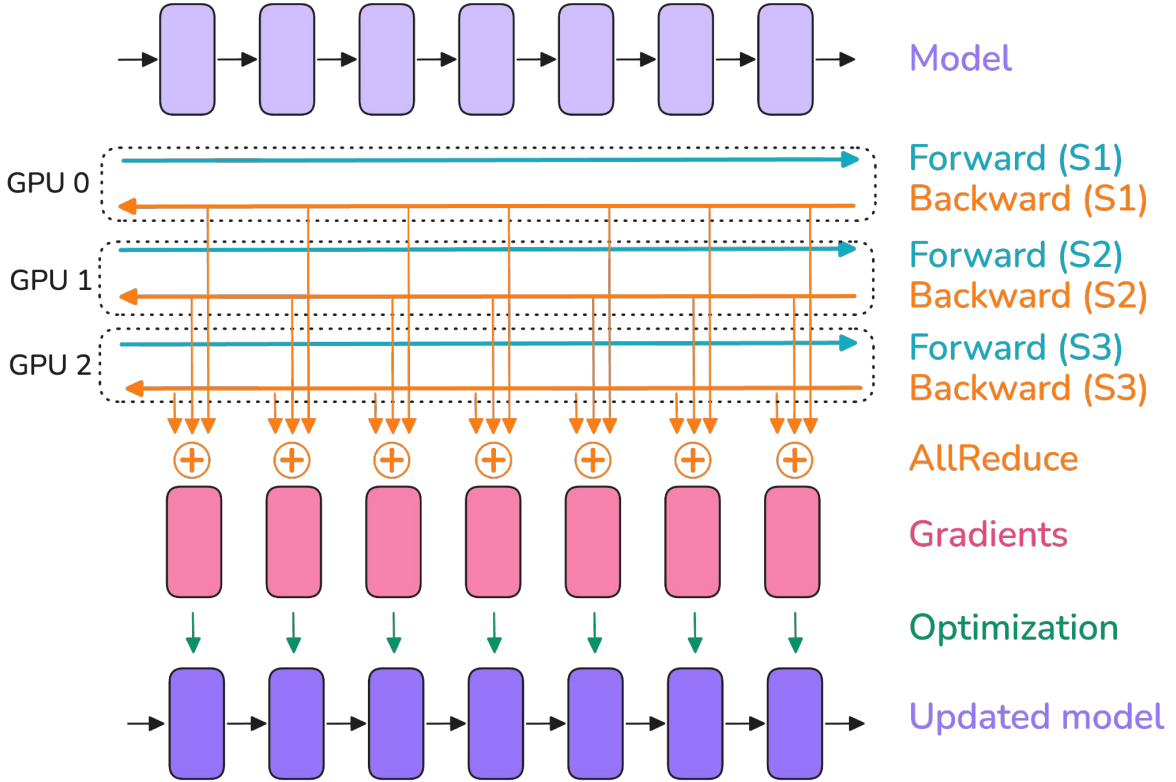
Intuition behind Context Parallelism



In Context Parallelism we shard data along sequence length

⇩

But what about attention?



CP: Ring Attention

We use **Ring Attention** which exchanges **K** and **V** of each GPU to compute **attention** using **online softmax**

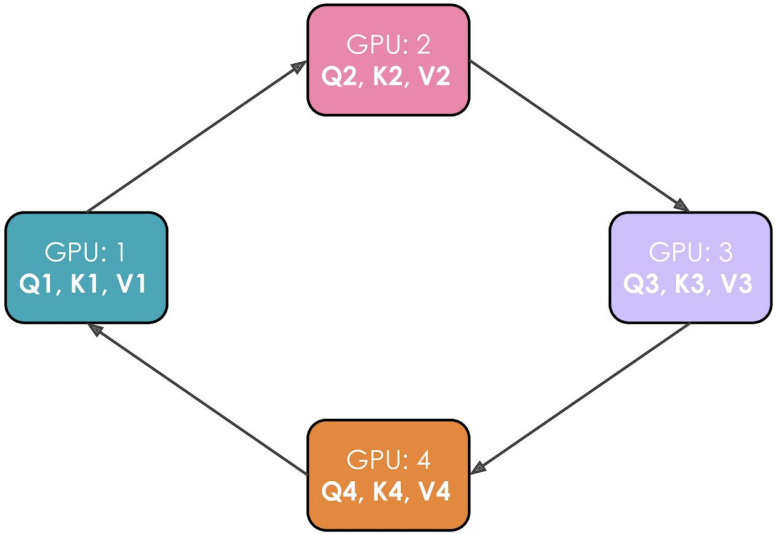
Online Softmax:

for $i \leftarrow 1, N$ **do**

$$
\begin{aligned}
x_i &\leftarrow Q[k,:]\,K^T[:,i] \\
m_i &\leftarrow \max\left(m_{i-1}, x_i\right) \\
d'_i &\leftarrow d'_{i-1}\,e^{m_{i-1}-m_i} + e^{x_i-m_i} \\
\boldsymbol{o}'_i &\leftarrow \boldsymbol{o}'_{i-1}\frac{d'_{i-1}\,e^{m_{i-1}-m_i}}{d'_i} + \frac{e^{x_i-m_i}}{d'_i}V[i,:]
\end{aligned}
$$

end

$$O[k,:] \leftarrow \boldsymbol{o}'_N$$



Ring Attention

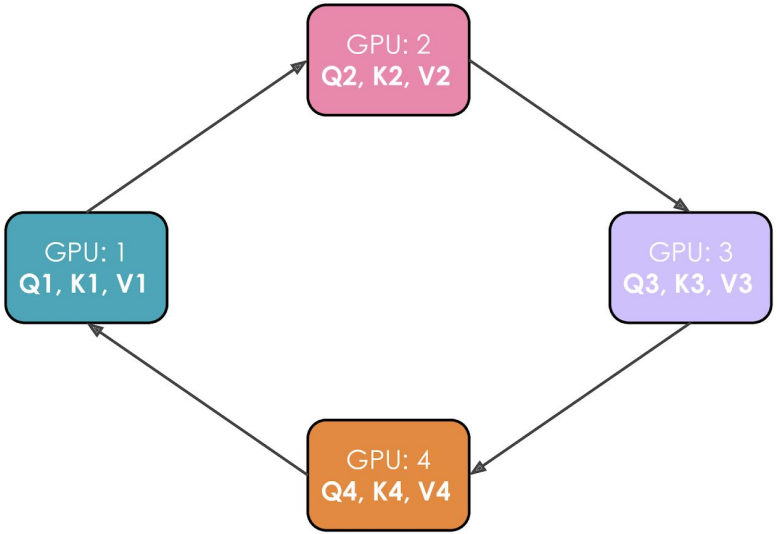
CP: Ring Attention

We use **Ring Attention** which exchanges **K** and **V** of each GPU to compute **attention** using **online softmax**

⇩

We need to communicate K and V each attention block

Note: we can use SendRecv ops to send (`K_i`, `V_i`) and recv (`K_i-1`, `V_i-1`)



Ring Attention

CP: Pros/Cons

\+ **Only parallelism** to efficiently partition **large sequences** memory

\- Communication **heavy** (every attention block, critical path)

\- Need to **allreduce** grads since each GPU sees different data

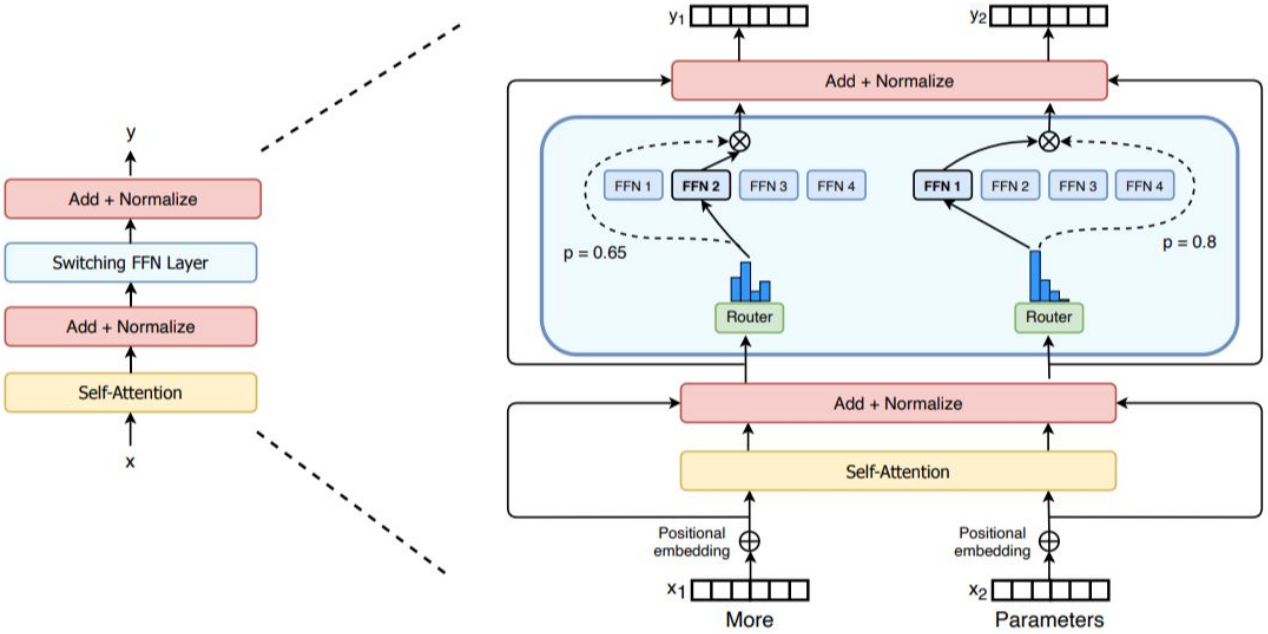
\- Doesn't help much for **short sequences**

To be used if and only if **long context training**

Expert Parallelism (EP)

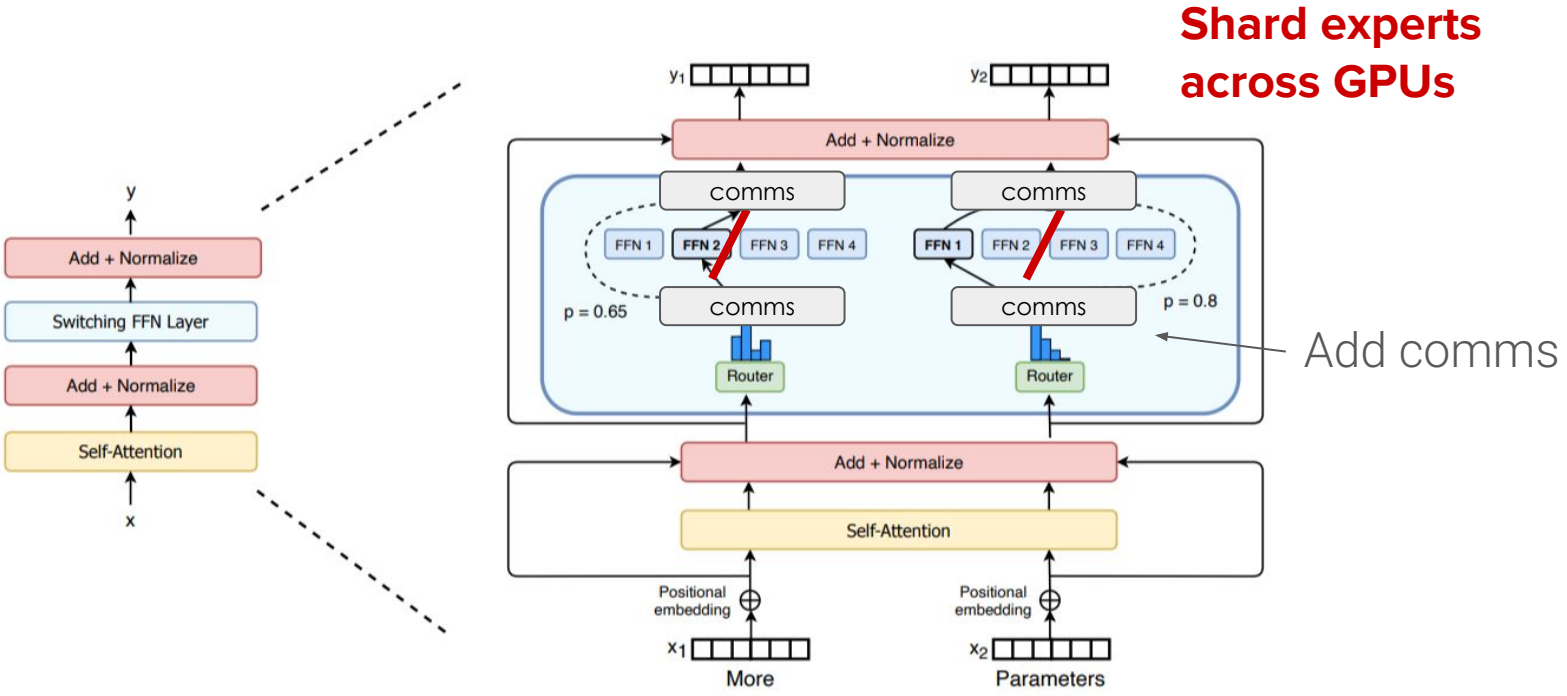
Sharding the model experts

Intuition behind Expert Parallelism



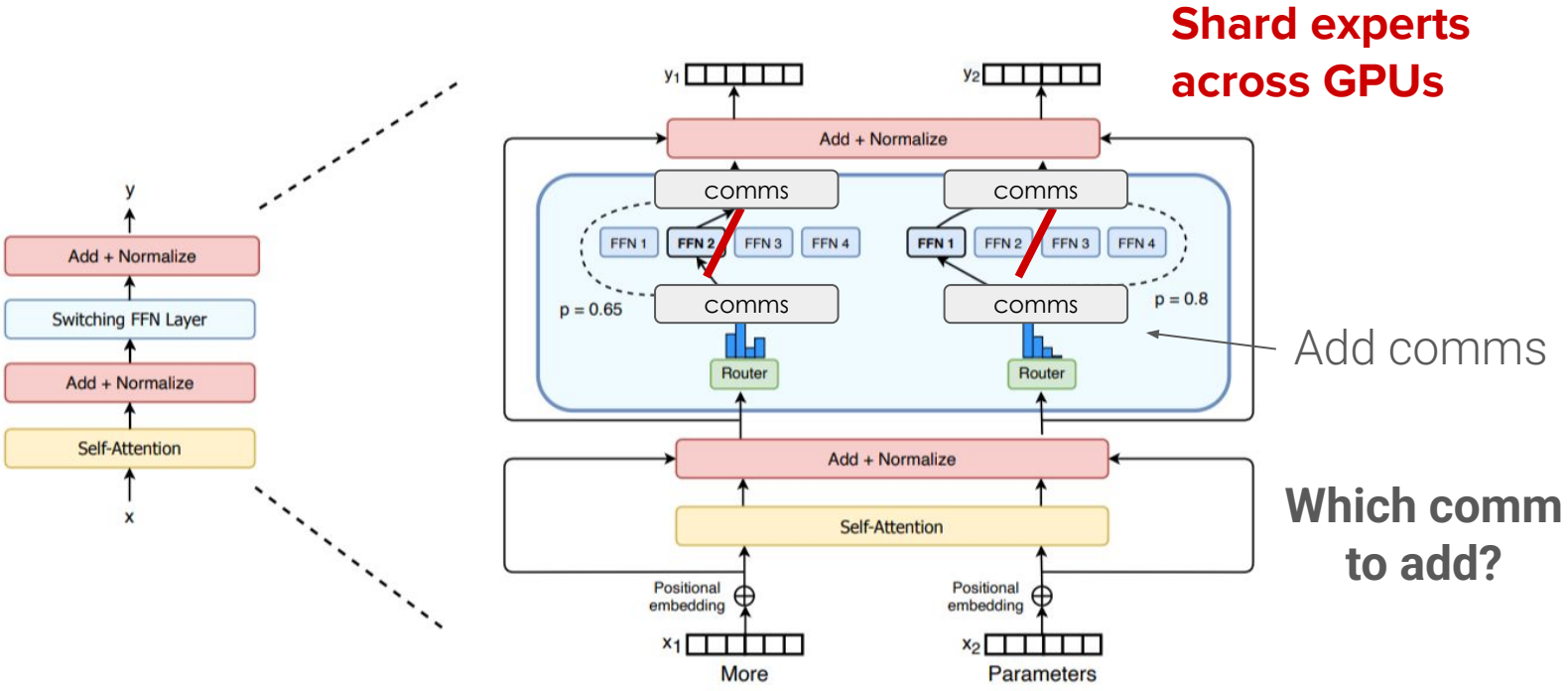
MoEs are cool.. But how do we **parallelize** them?

Intuition behind Expert Parallelism



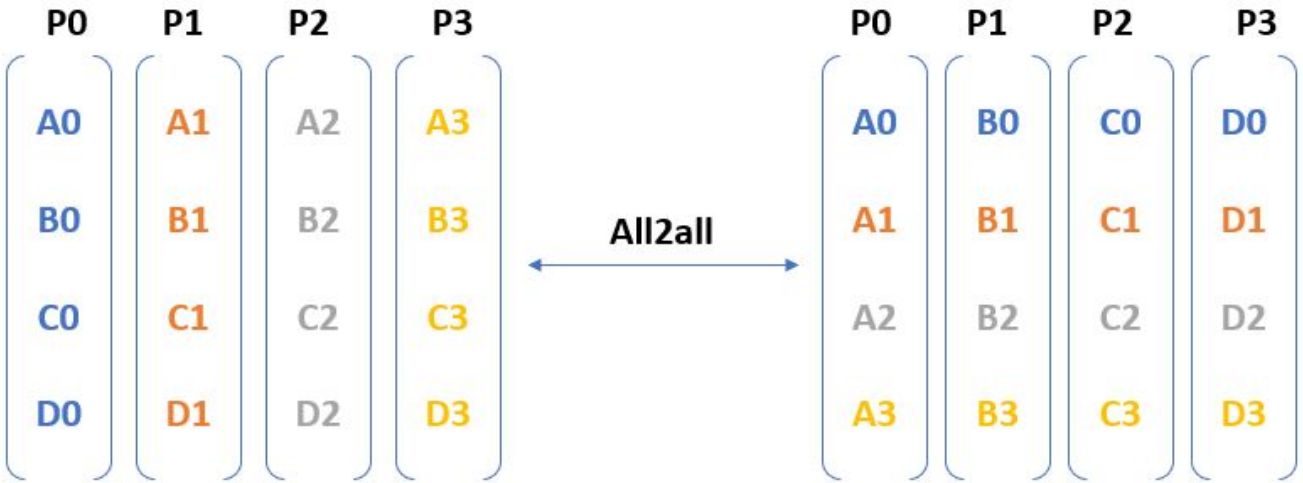
MoEs are cool.. But how do we **parallelize** them?

Intuition behind Expert Parallelism



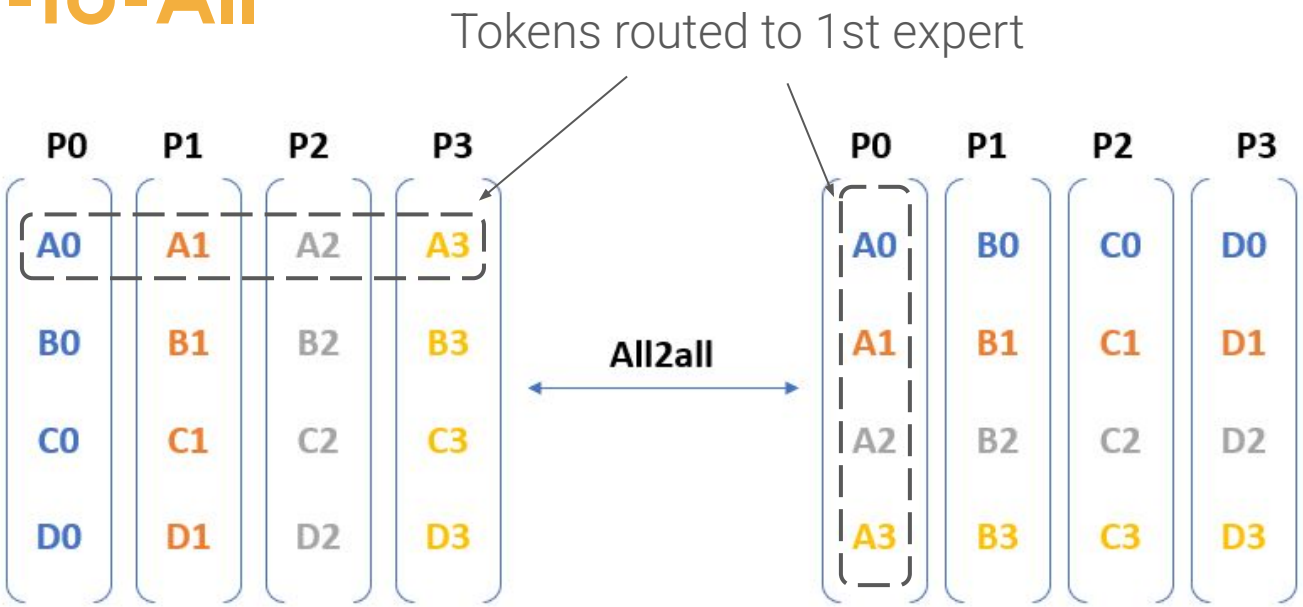
MoEs are cool.. But how do we **parallelize** them?

EP: All-to-All



All-to-All is communicating **distinct messages** from **each process** to every other process.

EP: All-to-All

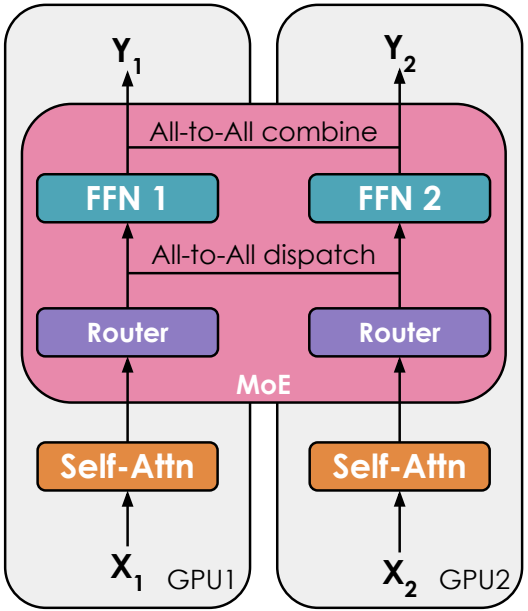


All-to-All is communicating **distinct messages** from **each process** to every other process.

Intuition behind Expert Parallelism

In **Expert Parallelism** we:

- Shard the **experts** across GPUs
- Shard **data** across GPUs
- We need **All-to-All** comms to route tokens to their respective experts



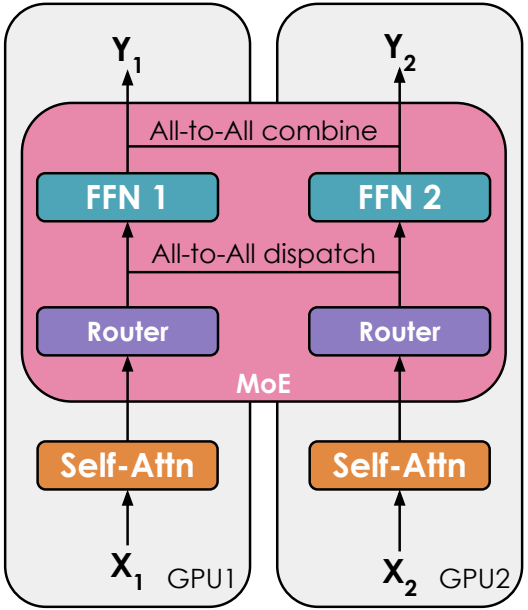
Expert Parallel

Intuition behind Expert Parallelism

In **Expert Parallelism** we:

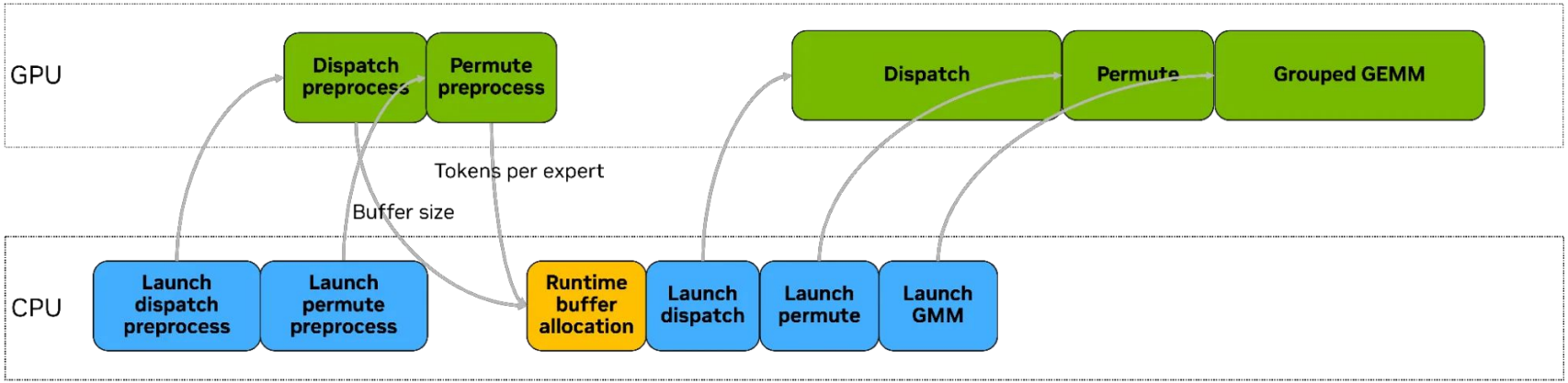
- Shard the **experts** across GPUs
- Shard **data** across GPUs
- We need **All-to-All** comms to route tokens to their respective experts

Problem: Router decides on how many tokens each rank gets, so CPU needs to wait for GPU to decide



Expert Parallel

Expert Parallelism: Difficulty



Source: NVIDIA's HybridEP blog

Problem: Router decides on how many tokens each rank gets, so CPU needs to wait for GPU to get buffer sizes

Most solutions to avoid cpu-gpu sync currently involve using **recent hardware** (IBGDA/RDMA)

EP: Pros/Cons

\+ **Only way** to distribute MoEs efficiently

\- Communication heavy (**AlltoAll** every MoE block, critical path)

\- Need to allreduce grads (outside MoE) since each GPU sees **different data**

\- **CPU-GPU sync** to launch Dispatch operation

To be used if and only if **MoE training**

EP: Practical Solution

Combine **EP** with **PP** with **1F1B scheduler** and hide comms from a batch with compute from another batch

Flags to activate this on Megatron:

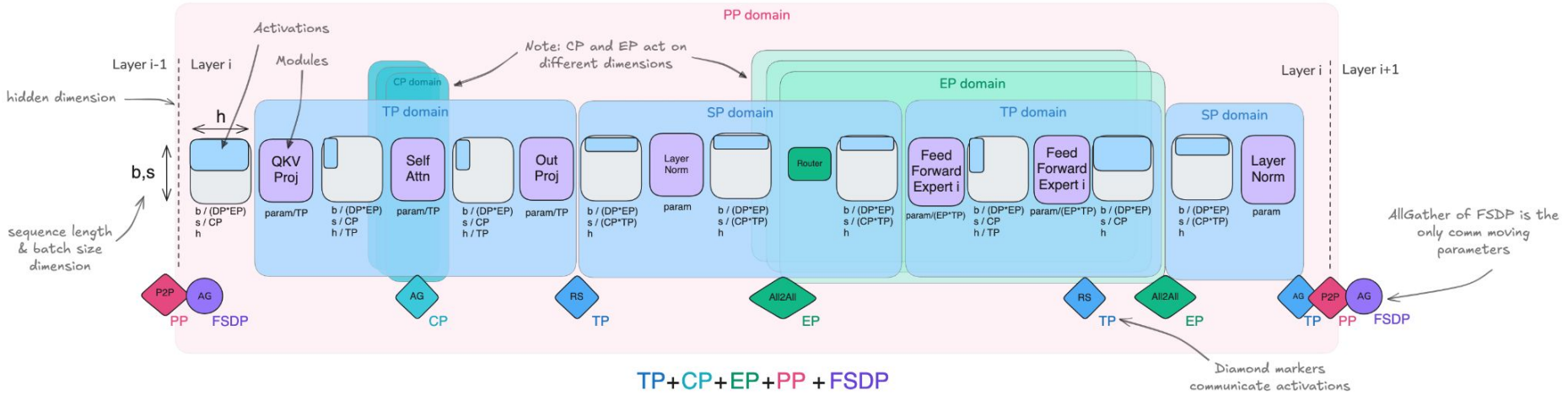
```
cfg.comm_overlap.overlap_moe_expert_parallel_comm = True
cfg.comm_overlap.delay_wgrad_compute = True
```

5D Parallelism

In a nutshell

5D parallelism

Each parallelism works **orthogonal** to the others, and **all 5 can be combined**!



Recap

What we've seen today:

1. Data Parallelism *(with ZeRO variants)*
2. Tensor Parallelism *(with SP variant)*
3. Pipeline Parallelism
4. Context Parallelism
5. Expert Parallelism

Learn more

The Ultra-Playbook Cheatsheet

Step 1: Fit model into memory

GPU rich case: 🤑

- **Small models (<10B):** use a single parallelism technique, e.g. TP or ZeRO-3/DP with Full Recompute across 8 GPUs.
- **Large models (10B+):** requires more than 8 GPUs, you have several options:
  - Combining Tensor Parallelism (TP=8) with Pipeline Parallelism
  - Combining Tensor Parallelism (TP=8) with Data Parallelism (ZeRO-3)
  - Using only ZeRO-3 (i.e. only pure Data Parallelism)
- **512+ GPU scale**: pure DP/ZeRO-3 becomes inefficient due to communication cost - better to then combine DP with either TP or PP
- **1024+ GPU scale**, a recommended setup can be TP=8 with DP (ZeRO-2) and PP
- Special cases: for **long context** consider CP and for **MoE** arch use EP

GPU poor case 😭**:**

- **Reduce memory:** use full activation checkpointing and/or gradient accumulation

Step 2: Satisfy target global batch size

Experiments tell us which batch size is ideal for training (4-40M tokens). So we either have to increase or decrease the batch size based on step 1 to meet it.

Increase Global Batch Size:

- Scale up DP or CP or gradient accumulation steps

Decrease Global Batch Size:

- Reduce DP or CP in favor of other parallelization strategies

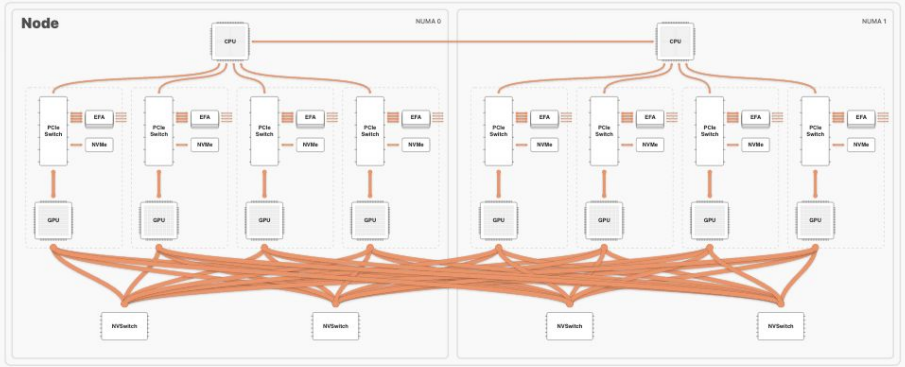
Step 3: Optimizing Training Throughput

There is no general recipe for the best configuration so at this point we should experiment:

- **Scale up TP** up to the node size to reduce other parallel strategies
- **Increase DP** with ZeRO-3 while keeping target GBS
- **Use PP** if communication becomes a bottleneck for DP
- Play with **micro batch size** to balance max GBS, model size, compute/comms

Learn more

How infra works when training LLMs



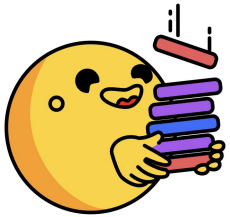
Simplified diagram of the key components and communication links in our AWS P5 instance setup.

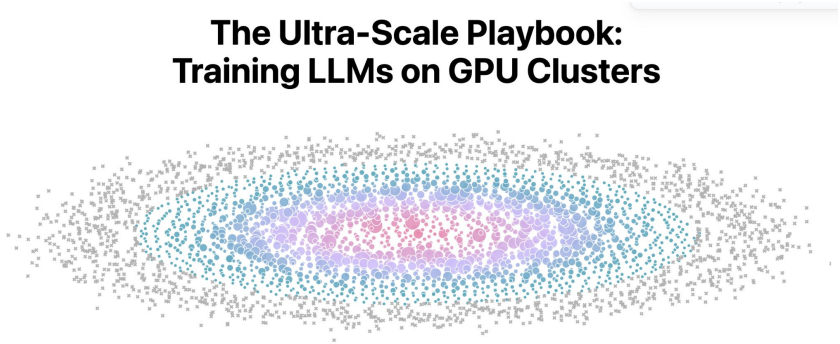
EFA Link - **12.5** GB/s
PCIe Gen4 - **16** GB/s
PCIe Gen5 - **64** GB/s
NVLink 4.0 - **900** GB/s

A practical journey through the challenges, decisions, and messy reality behind training state-of-the-art language models

And much more..

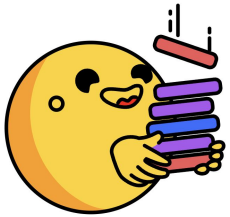
- Ultra-Scale Playbook huggingface.co/spaces/nanotron/ultrascale-playbook
- Smol-Training Playbook huggingface.co/spaces/HuggingFaceTB/smol-training-playbook
- Jax Scaling Book https://jax-ml.github.io/scaling-book/tpus/

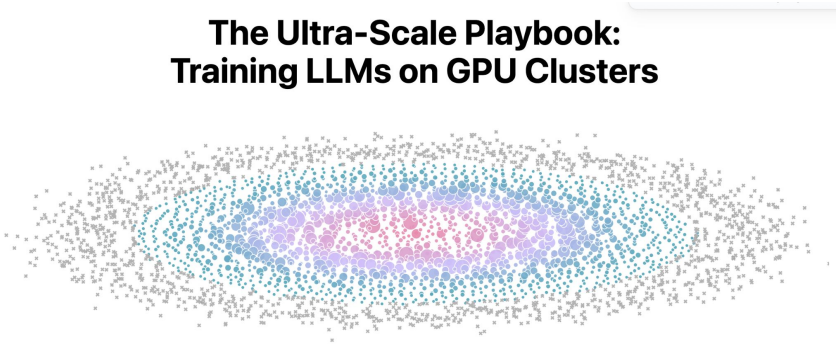




Contributions are welcome!

- Nanotron: https://github.com/huggingface/nanotron
- Torchtitan: https://github.com/pytorch/torchtitan
- Megatron-LM: https://github.com/NVIDIA/Megatron-LM





Finally..

As we scale to thousand of GPUs let's be mindful of their energy impact, and try to use them *responsibly* 🤗

Thank you

GitHub/HF Hub/X: **nouamanetazi**